

mcat

Computer-Assisted Translation Tool

Version 1.0

Codebase Comprehensive Report

Generated: 2026-07-16 00:51

Total: 7728 lines across 31 Python files

Repository: <https://github.com/skarmohdi2006/mcat>

Table of Contents

1. Overview & Statistics
2. Entry Point
3. Core Library Modules
4. Format Handlers
5. Test Suites

Files (alphabetical within each category):

- mcat.py (21 lines)
- cat_tool__init__.py (1 lines)
- cat_toolui.py (2636 lines)
- cat_tooltm.py (89 lines)
- cat_toolglossary.py (360 lines)
- cat_toolsegmentation.py (380 lines)
- cat_toolsuggest.py (138 lines)
- cat_toolspellcheck.py (135 lines)
- cat_toolqa.py (179 lines)
- cat_toolsettings.py (46 lines)
- cat_toolproject.py (217 lines)
- cat_toolfile_handler.py (113 lines)
- cat_toolmcat_format_manager.py (151 lines)
- cat_toolformats__init__.py (30 lines)
- cat_toolformats\factory.py (55 lines)
- cat_toolformats_text_base.py (27 lines)
- cat_toolformats>tag_utils.py (34 lines)
- cat_toolformats\text.py (16 lines)
- cat_toolformats\pdf.py (77 lines)
- cat_toolformats\docx.py (47 lines)
- cat_toolformats\pptx.py (43 lines)
- cat_toolformats\xlsx.py (33 lines)
- cat_toolformats\xls.py (39 lines)
- cat_toolformats\odf.py (102 lines)
- cat_toolformats\mqxliif.py (64 lines)
- cat_toolformats\sdxliiff.py (111 lines)
- cat_toolformats\mcatsby.py (125 lines)
- tests\test_cat_logic.py (101 lines)

- tests\test_comprehensive.py (610 lines)
- tests\test_remaining.py (722 lines)
- tests\test_more.py (1026 lines)

1. Overview & Statistics

Metric	Value
Total Python Files	31
Total Lines of Code	7728
Total Classes Defined	63
Total Functions Defined	517
Application Classes	20 (in cat_tool package)
Format Handler Classes	11
Test Classes	32
Core Library Modules	12
Format Handlers	14
Test Files	4
Lines per File (avg)	249
GUI Framework	PyQt5
Database Engine	SQLite3
Fuzzy Matching	rapidfuzz
Spell Checking	spyls (Hunspell)

Supported File Formats

Format	Handler Class	Libraries Used
Plain Text (.txt)	TextHandler	builtins
PDF (.pdf)	PdfHandler	pdfplumber, pypdf, reportlab
Word (.docx)	DocxHandler	python-docx
PowerPoint (.pptx/.ppsx)	PptxHandler	python-pptx
Excel (.xlsx)	XlsxHandler	openpyxl
Excel (.xls)	XlsHandler	xlrd, xlutils
OpenDocument (.odt/.ods/.odp)	OdfHandler	odfpy
memoQ XLIFF (.mqxliff)	MqxliffHandler	xml.etree
SDL Trados XLIFF (.sdlxliff)	SdlxliffHandler	xml.etree
Native MCAT (.mcat.db)	McatDbHandler	sqlite3
PO / XLIFF / TMX / TBX / CSV	FactoryHandler	translate-toolkit

2. Entry Point

mcat.py — 21 lines

- Function **main** (line 7)

```
import os
import sys
from PyQt5.QtWidgets import QApplication
from cat_tool.ui import MainWindow

def main():
    app = QApplication(sys.argv)
    win = MainWindow()
    win.show()

    if len(sys.argv) > 1:
        file_path = sys.argv[1]
        if os.path.exists(file_path):
            win.open_file_path(file_path)

    sys.exit(app.exec_())

if __name__ == "__main__":
    main()
```

3. Core Library

cat_tool__init__.py — 1 lines

```
# Package marker for cat_tool
```

cat_toolui.py — 2636 lines

- Class **GlossaryDialog** (line 147)
- Class **LangDialog** (line 192)
- Class **NewProjectDialog** (line 273)
- Class **ProjectSettingsDialog** (line 355)
- Class **ShortcutsDialog** (line 430)
- Class **QADialog** (line 483)
- Class **SpellCheckDialog** (line 563)
- Class **SettingsDialog** (line 729)
- Class **TagViewDialog** (line 772)
- Class **MainWindow** (line 853)
- Function **_get_data_dir** (line 9)
- Function **_fill_lang_combo** (line 242)
- Function **__init__** (line 148)
- Function **init_ui** (line 156)
- Function **get_data** (line 182)
- Function **__init__** (line 193)
- Function **__init__** (line 274)
- Function **init_ui** (line 281)
- Function **browse_files** (line 333)
- Function **clear_files** (line 342)
- Function **get_data** (line 346)
- Function **__init__** (line 356)
- Function **init_ui** (line 363)
- Function **add_source_file** (line 410)
- Function **remove_source_file** (line 418)
- Function **apply** (line 424)
- Function **__init__** (line 431)
- Function **init_ui** (line 437)
- Function **__init__** (line 484)
- Function **init_ui** (line 491)
- Function **_go_to_segment** (line 549)
- Function **_on_double_click** (line 559)
- Function **__init__** (line 564)
- Function **init_ui** (line 572)
- Function **_populate** (line 627)
- Function **_show_details** (line 637)
- Function **_clear_suggestions** (line 659)
- Function **_show_suggestion_buttons** (line 665)
- Function **_replace_word** (line 683)
- Function **_load_dict** (line 711)
- Function **__init__** (line 730)
- Function **init_ui** (line 737)
- Function **_save** (line 763)
- Function **__init__** (line 773)
- Function **init_ui** (line 786)
- Function **_insert_selected_tag** (line 834)
- Function **__init__** (line 854)

- Function **init_menu** (line 881)
- Function **init_ui** (line 1072)
- Function **open_file** (line 1161)
- Function **populate_table** (line 1190)
- Function **get_status_text** (line 1211)
- Function **set_source_cell_style** (line 1219)
- Function **set_target_cell_style** (line 1230)
- Function **_on_target_cell_changed** (line 1241)
- Function **save_file** (line 1276)
- Function **save_file_as** (line 1287)
- Function **render_translated_file** (line 1337)
- Function **segment_selected** (line 1356)
- Function **fuzzy_state_changed** (line 1371)
- Function **update_table_row_status** (line 1382)
- Function **confirm_active_segment** (line 1393)
- Function **_edit_target_cell** (line 1440)
- Function **_go_next_segment** (line 1444)
- Function **_go_prev_segment** (line 1454)
- Function **_go_next_untranslated** (line 1464)
- Function **_table_context_menu** (line 1477)
- Function **_show_tag_dialog_for_target** (line 1499)
- Function **_spellcheck_segment** (line 1503)
- Function **show_options** (line 1508)
- Function **open_file_path** (line 1514)
- Function **_apply_settings** (line 1528)
- Function **update_fuzzy_matches** (line 1539)
- Function **apply_tm_suggestion** (line 1548)
- Function **update_glossary_matches** (line 1556)
- Function **insert_glossary_term** (line 1576)
- Function **add_glossary_term** (line 1588)
- Function **import_tmx_to_glossary** (line 1603)
- Function **export_glossary_as_tmx** (line 1634)
- Function **create_collection_dialog** (line 1656)
- Function **manage_collections** (line 1683)
- Function **import_collection_tmx** (line 1860)
- Function **export_collection_tmx** (line 1881)
- Function **view_glossary** (line 1925)
- Function **import_po_to_tm** (line 1982)
- Function **clear_tm** (line 2007)
- Function **_toggle_fuzzy** (line 2019)
- Function **_go_first_segment** (line 2023)
- Function **_go_last_segment** (line 2028)
- Function **show_shortcuts** (line 2034)
- Function **_open_dicts_page** (line 2038)
- Function **navigate_to_segment** (line 2042)
- Function **run_qa** (line 2047)
- Function **import_dictionary** (line 2055)
- Function **_load_spelling_dict_for_language** (line 2076)
- Function **run_spellcheck** (line 2110)
- Function **update_stats** (line 2119)
- Function **close_project** (line 2161)
- Function **new_project** (line 2185)
- Function **open_project** (line 2219)
- Function **_load_project** (line 2263)
- Function **_load_omegat** (line 2283)

- Function **project_settings** (line 2303)
- Function **_open_project_source_file** (line 2318)
- Function **save_project** (line 2347)
- Function **save_project_as** (line 2364)
- Function **_current_lang_code** (line 2441)
- Function **_load_segmentation_rules** (line 2446)
- Function **_tm_sl** (line 2459)
- Function **_tm_tl** (line 2463)
- Function **open_segmentation_rules** (line 2466)
- Function **import_omegat_project** (line 2475)
- Function **export_omegat_project** (line 2502)
- Function **import_memoq_project** (line 2525)
- Function **export_memoq_project** (line 2538)
- Function **_load_memoq** (line 2560)
- Function **import_sdlxliff** (line 2586)
- Function **export_sdlxliff** (line 2603)
- Function **_update_recent_menu** (line 2624)
- Function **refresh_collection_list** (line 1749)
- Function **refresh_terms** (line 1756)
- Function **on_coll_selected** (line 1770)
- Function **do_create** (line 1773)
- Function **do_delete** (line 1778)
- Function **do_add_term** (line 1793)
- Function **do_remove_term** (line 1832)
- Function **delete_selected** (line 1951)

```
import os
import sys

# Ensure parent directory is in sys.path to allow running this script directly
parent_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if parent_dir not in sys.path:
    sys.path.insert(0, parent_dir)

def _get_data_dir():
    d = os.path.join(os.environ["APPDATA"], "mcat")
    os.makedirs(d, exist_ok=True)
    return d

from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QTableWidgetItem, QSplitter, QTextEdit,
                             QPushButton, QLabel, QListWidget, QListWidgetItem,
                             QFileDialog, QStatusBar, QProgressBar, QMessageBox,
                             QCheckBox, QDialog, QFormLayout, QLineEdit, QHeaderView,
                             QComboBox, QDialogButtonBox, QMenu, QScrollArea, QSpinBox)

from PyQt5.QtCore import Qt, pyqtSignal
from PyQt5.QtGui import QColor, QFont, QKeySequence

from cat_tool.file_handler import CATFileHandler, convert_to_po
from cat_tool.formats.tag_utils import PH_L, PH_R, extract_all_tags, apply_tags, restore_tags
from cat_tool.tm import TranslationMemory
from cat_tool.glossary import Glossary
from cat_tool.project import Project
from cat_tool.segmentation import AdvancedRulesDialog, advanced_segmenter, DEFAULT_RULES, load_rules
from cat_tool.suggest import CompletionDelegate
from cat_tool import qa
from cat_tool.spellcheck import SpellChecker
from cat_tool.settings import AppSettings

DARK_STYLESHEET = """
QMainWindow {
    background-color: #121214;
}
QWidget {
    background-color: #1e1e24;
    color: #ffffff;
}
```

```

    font-family: "Segoe UI", Arial, sans-serif;
    font-size: 13px;
}
QTableWidget {
    background-color: #1a1a1e;
    gridline-color: #2c2c35;
    border: 1px solid #2c2c35;
    border-radius: 4px;
}
QTableWidget::item {
    padding: 8px;
    border-bottom: 1px solid #2c2c35;
}
QTableWidget::item:selected {
    background-color: #0078d4;
    color: white;
}
QHeaderView::section {
    background-color: #25252b;
    color: #b0b0b8;
    padding: 6px;
    border: 1px solid #2c2c35;
    font-weight: bold;
}
QTextEdit {
    background-color: #1a1a1e;
    border: 1px solid #2c2c35;
    border-radius: 4px;
    padding: 6px;
    font-size: 14px;
    color: #ffffff;
}
QTextEdit[readOnly="true"] {
    background-color: #151518;
    color: #a0a0a8;
}
QPushButton {
    background-color: #0078d4;
    color: white;
    border: none;
    padding: 8px 16px;
    border-radius: 4px;
    font-weight: bold;
}
QPushButton:hover {
    background-color: #1085e0;
}
QPushButton:pressed {
    background-color: #0066b3;
}
QPushButton:disabled {
    background-color: #3a3a42;
    color: #808088;
}
QListWidget {
    background-color: #1a1a1e;
    border: 1px solid #2c2c35;
    border-radius: 4px;
}
QListWidget::item {
    padding: 8px;
    border-bottom: 1px solid #2c2c35;
}
QListWidget::item:hover {
    background-color: #25252b;
}
QListWidget::item:selected {
    background-color: #0078d4;
    color: white;
}
QCheckBox {
    spacing: 5px;
}
QCheckBox::indicator {
    width: 18px;
    height: 18px;
}
QStatusBar {
    background-color: #121214;

```

```

        color: #b0b0b8;
    }
    QProgressBar {
        border: 1px solid #2c2c35;
        border-radius: 4px;
        text-align: center;
        background-color: #1a1a1e;
        color: white;
    }
    QProgressBar::chunk {
        background-color: #20c997;
        border-radius: 3px;
    }
    QLineEdit {
        background-color: #1a1a1e;
        border: 1px solid #2c2c35;
        border-radius: 4px;
        padding: 6px;
        color: white;
    }
    QLabel {
        font-weight: bold;
        color: #e0e0e8;
    }
    """

class GlossaryDialog(QDialog):
    def __init__(self, parent=None, source_lang="", target_lang=""):
        super().__init__(parent)
        self.setWindowTitle("Add Glossary Term")
        self.setMinimumWidth(500)
        self.source_lang = source_lang
        self.target_lang = target_lang
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout(self)
        form = QFormLayout()

        self.src_edit = QLineEdit(self)
        self.tgt_edit = QLineEdit(self)
        self.desc_edit = QLineEdit(self)

        self.src_lang_combo = QComboBox(self)
        _fill_lang_combo(self.src_lang_combo, self.source_lang)
        self.tgt_lang_combo = QComboBox(self)
        _fill_lang_combo(self.tgt_lang_combo, self.target_lang)

        form.addRow("Source Term:", self.src_edit)
        form.addRow("Source Language:", self.src_lang_combo)
        form.addRow("Target Translation:", self.tgt_edit)
        form.addRow("Target Language:", self.tgt_lang_combo)
        form.addRow("Description/Context:", self.desc_edit)

        layout.addLayout(form)

        btn_box = QDialogButtonBox(QDialogButtonBox.Save | QDialogButtonBox.Cancel, self)
        btn_box.accepted.connect(self.accept)
        btn_box.rejected.connect(self.reject)
        layout.addWidget(btn_box)

    def get_data(self):
        return (
            self.src_edit.text().strip(),
            self.tgt_edit.text().strip(),
            self.desc_edit.text().strip(),
            self.src_lang_combo.currentData() or "",
            self.tgt_lang_combo.currentData() or "",
        )

class LangDialog(QDialog):
    def __init__(self, sl, tl, count, parent=None):
        super().__init__(parent)
        self.setWindowTitle("TMX Language Detection")
        layout = QVBoxLayout(self)
        layout.addWidget(QLabel(f"Found {count} term pairs in TMX."))
        layout.addWidget(QLabel("Detected languages:"))
        f = QFormLayout()

```

```

        self.src_combo = QComboBox()
        _fill_lang_combo(self.src_combo, sl)
        self.tgt_combo = QComboBox()
        _fill_lang_combo(self.tgt_combo, tl)
        f.addRow("Source:", self.src_combo)
        f.addRow("Target:", self.tgt_combo)
        layout.addLayout(f)
        btns = QHBoxLayout()
        ok = QPushButton("Import")
        ok.setToolTip("Import the TMX terms into the glossary")
        ok.clicked.connect(self.accept)
        cancel = QPushButton("Cancel")
        cancel.setToolTip("Cancel TMX import")
        cancel.clicked.connect(self.reject)
        btns.addWidget(ok)
        btns.addWidget(cancel)
        layout.addLayout(btns)

LANGUAGES = [
    ("af", "Afrikaans"), ("sq", "Albanian"), ("ar", "Arabic"), ("hy", "Armenian"),
    ("az", "Azerbaijani"), ("eu", "Basque"), ("be", "Belarusian"), ("bn", "Bengali"),
    ("bs", "Bosnian"), ("bg", "Bulgarian"), ("ca", "Catalan"), ("zh", "Chinese (Simplified)"),
    ("zh-TW", "Chinese (Traditional)"), ("hr", "Croatian"), ("cs", "Czech"), ("da", "Danish"),
    ("nl", "Dutch"), ("en", "English"), ("eo", "Esperanto"), ("et", "Estonian"),
    ("fi", "Finnish"), ("fr", "French"), ("gl", "Galician"), ("ka", "Georgian"),
    ("de", "German"), ("el", "Greek"), ("gu", "Gujarati"), ("he", "Hebrew"),
    ("hi", "Hindi"), ("hu", "Hungarian"), ("is", "Icelandic"), ("id", "Indonesian"),
    ("ga", "Irish"), ("it", "Italian"), ("ja", "Japanese"), ("kn", "Kannada"),
    ("kk", "Kazakh"), ("ko", "Korean"), ("ky", "Kyrgyz"), ("lv", "Latvian"),
    ("lt", "Lithuanian"), ("mk", "Macedonian"), ("ms", "Malay"), ("ml", "Malayalam"),
    ("mt", "Maltese"), ("mr", "Marathi"), ("mn", "Mongolian"), ("ne", "Nepali"),
    ("no", "Norwegian"), ("fa", "Persian"), ("pl", "Polish"), ("pt", "Portuguese"),
    ("pt-BR", "Portuguese (Brazil)"), ("pa", "Punjabi"), ("ro", "Romanian"),
    ("ru", "Russian"), ("sr", "Serbian"), ("sk", "Slovak"), ("sl", "Slovenian"),
    ("es", "Spanish"), ("sw", "Swahili"), ("sv", "Swedish"), ("tl", "Tagalog"),
    ("ta", "Tamil"), ("te", "Telugu"), ("th", "Thai"), ("tr", "Turkish"),
    ("uk", "Ukrainian"), ("ur", "Urdu"), ("uz", "Uzbek"), ("vi", "Vietnamese"),
    ("cy", "Welsh"), ("xh", "Xhosa"), ("yi", "Yiddish"), ("zu", "Zulu"),
]

def _fill_lang_combo(combo, selected_code=""):
    combo.clear()
    for code, name in LANGUAGES:
        combo.addItem(f"{name} [{code}]", code)
    if selected_code:
        idx = combo.findData(selected_code)
        if idx >= 0:
            combo.setCurrentIndex(idx)

FILE_FILTER = (
    "All Supported Files (*.po *.pot *.xlf *.xliff *.mqxliff *.sdlxiff *.ts *.tmx *.tbx *.csv "
    "*.txt *.pdf *.docx *.pptx *.ppsx *.xlsx *.xls *.odt *.ods *.odp);;"
    "PO Files (*.po *.pot);;"
    "XLIFF (*.xlf *.xliff);;"
    "memoQ MQXLIFF (*.mqxliff);;"
    "SDL Trados SDLXLIFF (*.sdlxiff);;"
    "Qt TS (*.ts);;"
    "TMX (*.tmx);;"
    "TBX (*.tbx);;"
    "CSV (*.csv);;"
    "Text (*.txt);;"
    "PDF (*.pdf);;"
    "Word (*.docx);;"
    "PowerPoint (*.pptx *.ppsx);;"
    "Excel (*.xlsx *.xls);;"
    "OpenDocument (*.odt *.ods *.odp);;"
    "All Files (*)"
)

class NewProjectDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("New Project")
        self.setMinimumWidth(520)
        self.source_files = []

```

```

        self.init_ui()

def init_ui(self):
    layout = QVBoxLayout(self)
    form = QFormLayout()

    self.name_edit = QLineEdit(self)
    self.name_edit.setPlaceholderText("My Translation Project")
    name_label = QLabel("Project Name:")
    form.addRow(name_label, self.name_edit)

    self.src_lang_combo = QComboBox(self)
    _fill_lang_combo(self.src_lang_combo, "en")
    self.src_lang_combo.setMinimumWidth(250)
    src_label = QLabel("Source Language:")
    form.addRow(src_label, self.src_lang_combo)

    self.tgt_lang_combo = QComboBox(self)
    _fill_lang_combo(self.tgt_lang_combo, "es")
    self.tgt_lang_combo.setMinimumWidth(250)
    tgt_label = QLabel("Target Language:")
    form.addRow(tgt_label, self.tgt_lang_combo)

    layout.addLayout(form)
    layout.addSpacing(10)

    # Source files section
    files_label = QLabel("Files to Translate:")
    layout.addWidget(files_label)
    self.file_list = QListWidget(self)
    layout.addWidget(self.file_list)

    file_btn_layout = QHBoxLayout()
    browse_btn = QPushButton("Browse for Files...", self)
    browse_btn.clicked.connect(self.browse_files)
    clear_btn = QPushButton("Clear", self)
    clear_btn.setStyleSheet("background-color: #4a4a52;")
    clear_btn.clicked.connect(self.clear_files)
    file_btn_layout.addWidget(browse_btn)
    file_btn_layout.addWidget(clear_btn)
    layout.addLayout(file_btn_layout)

    layout.addSpacing(10)
    buttons_layout = QHBoxLayout()
    self.create_btn = QPushButton("Create Project", self)
    self.create_btn.clicked.connect(self.accept)
    self.cancel_btn = QPushButton("Cancel", self)
    self.cancel_btn.clicked.connect(self.reject)
    self.cancel_btn.setStyleSheet("background-color: #4a4a52;")
    buttons_layout.addStretch()
    buttons_layout.addWidget(self.create_btn)
    buttons_layout.addWidget(self.cancel_btn)
    layout.addLayout(buttons_layout)

def browse_files(self):
    files, _ = QFileDialog.getOpenFileNames(
        self, "Select Files to Translate", "", FILE_FILTER
    )
    for f in files:
        if f not in self.source_files:
            self.source_files.append(f)
            self.file_list.addItem(f"{os.path.basename(f)} ({os.path.dirname(f)})")

def clear_files(self):
    self.source_files.clear()
    self.file_list.clear()

def get_data(self):
    return (
        self.name_edit.text().strip(),
        self.src_lang_combo.currentData(),
        self.tgt_lang_combo.currentData(),
        list(self.source_files),
    )

class ProjectSettingsDialog(QDialog):
    def __init__(self, project, parent=None):
        super().__init__(parent)

```

```

        self.setWindowTitle("Project Settings")
        self.setMinimumWidth(500)
        self.project = project
        self.init_ui()

def init_ui(self):
    layout = QVBoxLayout(self)
    form = QFormLayout()

    self.name_edit = QLineEdit(self.project.name, self)
    name_label = QLabel("Project Name:")
    form.addRow(name_label, self.name_edit)

    self.src_lang_combo = QComboBox(self)
    _fill_lang_combo(self.src_lang_combo, self.project.source_lang)
    self.src_lang_combo.setMinimumWidth(250)
    src_label = QLabel("Source Language:")
    form.addRow(src_label, self.src_lang_combo)

    self.tgt_lang_combo = QComboBox(self)
    _fill_lang_combo(self.tgt_lang_combo, self.project.target_lang)
    self.tgt_lang_combo.setMinimumWidth(250)
    tgt_label = QLabel("Target Language:")
    form.addRow(tgt_label, self.tgt_lang_combo)

    layout.addLayout(form)

    # Source files list
    layout.addSpacing(10)
    files_label = QLabel("Source Files:")
    layout.addWidget(files_label)
    self.file_list = QListWidget(self)
    for f in self.project.source_files:
        self.file_list.addItem(os.path.basename(f))
    layout.addWidget(self.file_list)

    file_btn_layout = QHBoxLayout()
    add_btn = QPushButton("Add PO File...", self)
    add_btn.clicked.connect(self.add_source_file)
    rem_btn = QPushButton("Remove", self)
    rem_btn.setStyleSheet("background-color: #4a4a52;")
    rem_btn.clicked.connect(self.remove_source_file)
    file_btn_layout.addWidget(add_btn)
    file_btn_layout.addWidget(rem_btn)
    layout.addLayout(file_btn_layout)

    layout.addSpacing(10)
    btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel, self)
    btn_box.accepted.connect(self.accept)
    btn_box.rejected.connect(self.reject)
    layout.addWidget(btn_box)

def add_source_file(self):
    files, _ = QFileDialog.getOpenFileName(
        self, "Add Source File", "", "PO Files (*.po);;All Files (*)"
    )
    if files:
        self.project.source_files.append(files)
        self.file_list.addItem(os.path.basename(files))

def remove_source_file(self):
    row = self.file_list.currentRow()
    if row >= 0:
        self.file_list.takeItem(row)
        self.project.source_files.pop(row)

def apply(self):
    self.project.name = self.name_edit.text().strip()
    self.project.source_lang = self.src_lang_combo.currentData()
    self.project.target_lang = self.tgt_lang_combo.currentData()

class ShortcutsDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Keyboard Shortcuts")
        self.setMinimumSize(500, 400)
        self.init_ui()
    def init_ui(self):

```

```

layout = QVBoxLayout(self)
table = QTableWidgetItem()
table.setColumnCount(2)
table.setHorizontalHeaderLabels(["Shortcut", "Action"])
table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
table.setEditTriggers(QTableWidgetItem.NoEditTriggers)
table.setSelectionBehavior(QTableWidgetItem.SelectRows)

shortcuts = [
    ("Ctrl+N", "New Project"),
    ("Ctrl+Shift+O", "Open Project"),
    ("Ctrl+Shift+S", "Save Project"),
    ("Ctrl+O", "Open File"),
    ("Ctrl+S", "Save File"),
    ("Ctrl+R", "Render Translated File"),
    ("Ctrl+Down", "Next Segment"),
    ("Ctrl+Up", "Previous Segment"),
    ("Ctrl+U", "Next Untranslated Segment"),
    ("Ctrl+Enter", "Confirm and Next"),
    ("Ctrl+F", "Toggle Fuzzy / Needs Review"),
    ("Ctrl+G", "Add Glossary Term"),
    ("Ctrl+Shift+G", "View/Manage Glossary"),
    ("Ctrl+M", "Import PO into TM"),
    ("Ctrl+Shift+T", "Segmentation Rules"),
    ("Ctrl+Home", "Go to First Segment"),
    ("Ctrl+End", "Go to Last Segment"),
    ("Ctrl+Shift+Q", "Quality Assurance Check"),
    ("F1 / Ctrl+Shift+H", "Show Keyboard Shortcuts"),
]

table.setRowCount(len(shortcuts))
for i, (shortcut, action) in enumerate(shortcuts):
    table.setItem(i, 0, QTableWidgetItem(shortcut))
    table.setItem(i, 1, QTableWidgetItem(action))

layout.addWidget(table)

close_btn = QPushButton("Close")
close_btn.clicked.connect(self.accept)
btn_layout = QHBoxLayout()
btn_layout.addStretch()
btn_layout.addWidget(close_btn)
layout.addLayout(btn_layout)

class QADialog(QDialog):
    def __init__(self, issues, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Quality Assurance Results")
        self.resize(900, 500)
        self._issues = issues
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout(self)

        summary = QLabel(
            f"Found {len(self._issues)} issue(s)"
            if self._issues else "No issues found."
        )
        summary.setStyleSheet("font-weight: bold; font-size: 14px; padding: 4px;")
        layout.addWidget(summary)

        self.qa_table = QTableWidgetItem()
        self.qa_table.setColumnCount(6)
        self.qa_table.setHorizontalHeaderLabels(["", "Severity", "Check", "#", "Source", "Target", "Description"])
        self.qa_table.setColumnHidden(0, True)
        self.qa_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeToContents)
        self.qa_table.horizontalHeader().setSectionResizeMode(4, QHeaderView.Stretch)
        self.qa_table.setEditTriggers(QTableWidgetItem.NoEditTriggers)
        self.qa_table.setSelectionBehavior(QTableWidgetItem.SelectRows)
        self.qa_table.setSelectionMode(QTableWidgetItem.SingleSelection)
        self.qa_table.cellDoubleClicked.connect(self._on_double_click)
        self.qa_table.setRowCount(len(self._issues))

        for i, issue in enumerate(self._issues):
            sev_item = QTableWidgetItem(issue.severity.upper())
            if issue.severity == "error":
                sev_item.setForeground(QColor("#ff4444"))

```

```

else:
    sev_item.setForeground(QColor("#ffaa00"))

    idx_item = QTableWidgetItem(str(issue.segment_index))
    idx_item.setTextAlignment(Qt.AlignCenter)

    self.qa_table.setItem(i, 1, sev_item)
    self.qa_table.setItem(i, 2, QTableWidgetItem(issue.check_name))
    self.qa_table.setItem(i, 3, idx_item)
    self.qa_table.setItem(i, 4, QTableWidgetItem(issue.source[:120] if issue.source else ""))
    self.qa_table.setItem(i, 5, QTableWidgetItem(issue.target[:120] if issue.target else ""))
    self.qa_table.setItem(i, 6, QTableWidgetItem(issue.description))

    # Hidden column 0 stores the segment index for navigation
    hidden_item = QTableWidgetItem(str(issue.segment_index))
    self.qa_table.setItem(i, 0, hidden_item)

layout.addWidget(self.qa_table)

btn_layout = QHBoxLayout()
go_btn = QPushButton("Go to Selected Segment")
go_btn.setStyleSheet("background-color: #0078d4;")
go_btn.clicked.connect(self._go_to_segment)
btn_layout.addWidget(go_btn)

close_btn = QPushButton("Close")
close_btn.clicked.connect(self.accept)
btn_layout.addStretch()
btn_layout.addWidget(close_btn)

layout.addLayout(btn_layout)

def _go_to_segment(self):
    row = self.qa_table.currentRow()
    if row < 0:
        return
    seg_idx = int(self.qa_table.item(row, 0).text())
    main_win = self.parent()
    if main_win and hasattr(main_win, "navigate_to_segment"):
        main_win.navigate_to_segment(seg_idx)
    self.accept()

def _on_double_click(self, row, col):
    self._go_to_segment()

class SpellCheckDialog(QDialog):
    def __init__(self, segments, spellchecker, parent=None):
        super().__init__(parent)
        self.segments = segments
        self.spellchecker = spellchecker
        self.setWindowTitle("Spell Check")
        self.setMinimumSize(700, 450)
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout(self)

        names = self.spellchecker.loaded_names
        dict_label = QLabel(f"Dictionaries: {'', '.join(names) if names else 'None'}")
        dict_label.setStyleSheet("font-weight: bold;")
        layout.addWidget(dict_label)

        if not self.spellchecker.has_dict:
            layout.addWidget(QLabel("No dictionary loaded. Load a LibreOffice .oxf / .zip / .dic file:"))
            btn = QPushButton("Load Dictionary")
            btn.clicked.connect(self._load_dict)
            layout.addWidget(btn)
            close_btn = QPushButton("Close")
            close_btn.clicked.connect(self.accept)
            layout.addWidget(close_btn)
            return

        self.results = []
        for idx, seg in enumerate(self.segments):
            target = seg.get("target_clean", seg.get("target", ""))
            if not target:
                continue
            errors = self.spellchecker.check_text(target)

```

```

        if errors:
            self.results.append((idx, seg, errors))

status_label = QLabel(f"Found errors in {len(self.results)} of {len(self.segments)} segments")
layout.addWidget(status_label)

self.list_widget = QListWidget()
self.list_widget.currentRowChanged.connect(self._show_details)
layout.addWidget(self.list_widget)

self.detail_label = QLabel("Select an error above to see suggestions")
self.detail_label.setWordWrap(True)
layout.addWidget(self.detail_label)

self.suggestions_widget = QWidget()
self.suggest_layout = QHBoxLayout(self.suggestions_widget)
layout.addWidget(self.suggestions_widget)
self.suggestions_widget.setVisible(False)

btn_layout = QHBoxLayout()
load_btn = QPushButton("Load Dictionary")
load_btn.clicked.connect(self._load_dict)
close_btn = QPushButton("Close")
close_btn.clicked.connect(self.accept)
btn_layout.addWidget(load_btn)
btn_layout.addStretch()
btn_layout.addWidget(close_btn)
layout.addLayout(btn_layout)

self._populate()

def _populate(self):
    self.list_widget.blockSignals(True)
    self.list_widget.clear()
    for seg_idx, seg, errors in self.results:
        text = f"Segment {seg_idx + 1}: {' '.join(w for w, _ in errors[:5])}"
        item = QListWidgetItem(text)
        item.setData(Qt.UserRole, (seg_idx, seg, errors))
        self.list_widget.addItem(item)
    self.list_widget.blockSignals(False)

def _show_details(self, row):
    self._clear_suggestions()
    if row < 0 or row >= self.list_widget.count():
        self.detail_label.setText("Select an error above to see suggestions")
        self.suggestions_widget.setVisible(False)
        return
    item = self.list_widget.item(row)
    seg_idx, seg, errors = item.data(Qt.UserRole)

    parts = [f"Segment {seg_idx + 1}:"]
    parts.append(f"Source: {seg.get('source_clean', seg['source'])[:100]}")
    parts.append(f"Target: {seg.get('target_clean', seg.get('target', ''))[:100]}")
    parts.append("")
    for word, suggs in errors:
        if suggs:
            parts.append(f"  {word}" + "> " + ", ".join(suggs[:5]))
        else:
            parts.append(f"  {word}" + " (no suggestions)")
    self.detail_label.setText("\n".join(parts))

    self._show_suggestion_buttons(errors)

def _clear_suggestions(self):
    while self.suggest_layout.count():
        w = self.suggest_layout.takeAt(0).widget()
        if w:
            w.deleteLater()

def _show_suggestion_buttons(self, errors):
    self._clear_suggestions()
    if not errors:
        self.suggestions_widget.setVisible(False)
        return
    first_word, first_suggs = errors[0]
    if not first_suggs:
        self.suggestions_widget.setVisible(False)
        return
    lbl = QLabel(f'Replace "{first_word}":')

```

```

self.suggest_layout.addWidget(lbl)
for s in first_suggs[:8]:
    btn = QPushButton(s)
    btn.clicked.connect(lambda checked, w=first_word, r=s: self._replace_word(w, r))
    self.suggest_layout.addWidget(btn)
self.suggest_layout.addStretch()
self.suggestions_widget.setVisible(True)

def _replace_word(self, old, new):
    row = self.list_widget.currentRow()
    if row < 0:
        return
    item = self.list_widget.item(row)
    seg_idx, seg, errors = item.data(Qt.UserRole)
    target = seg.get("target_clean", seg.get("target", ""))
    new_target = target.replace(old, new, 1)
    if "target_clean" in seg:
        seg["target_clean"] = new_target
    else:
        seg["target"] = new_target
    new_errors = self.spellchecker.check_text(new_target)
    new_errors = [e for e in new_errors if e[0] != old]
    if new_errors:
        self.results[row] = (seg_idx, seg, new_errors)
        self._populate()
        self.list_widget.setCurrentRow(row)
        self._show_details(row)
    else:
        self.results.pop(row)
        self._populate()
        if self.results:
            self.list_widget.setCurrentRow(min(row, len(self.results) - 1))
        else:
            self.detail_label.setText("All errors corrected!")
            self.suggestions_widget.setVisible(False)

def _load_dict(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Open Dictionary", "",
        "Dictionary (*.txt *.zip *.dic);;All Files (*)"
    )
    if not path:
        return
    if path.endswith(".dic"):
        ok = self.spellchecker.load_dictionary(path)
    else:
        ok = self.spellchecker.load_from_archive(path)
    if ok:
        QMessageBox.information(self, "Success", "Dictionary loaded.")
        self.init_ui()
    else:
        QMessageBox.warning(self, "Error", "Failed to load dictionary.")

class SettingsDialog(QDialog):
    def __init__(self, settings, parent=None):
        super().__init__(parent)
        self.settings = settings
        self.setWindowTitle("Options")
        self.setMinimumWidth(500)
        self.init_ui()

    def init_ui(self):
        layout = QFormLayout(self)

        self.src_font_spin = QSpinBox()
        self.src_font_spin.setRange(8, 30)
        self.src_font_spin.setValue(self.settings["source_font_size"])
        layout.addRow("Source font size:", self.src_font_spin)

        self.tgt_font_spin = QSpinBox()
        self.tgt_font_spin.setRange(8, 30)
        self.tgt_font_spin.setValue(self.settings["target_font_size"])
        layout.addRow("Target font size:", self.tgt_font_spin)

        self.auto_save_cb = QCheckBox()
        self.auto_save_cb.setChecked(self.settings["auto_save_on_segment_change"])
        layout.addRow("Auto-save on segment change:", self.auto_save_cb)
        self.show_tags_cb = QCheckBox()

```

```

self.show_tags_cb.setChecked(self.settings["show_tag_placeholders"])
layout.addRow("Show tag placeholders:", self.show_tags_cb)

btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
btn_box.accepted.connect(self._save)
btn_box.rejected.connect(self.reject)
layout.addRow(btn_box)

def _save(self):
    self.settings["source_font_size"] = self.src_font_spin.value()
    self.settings["target_font_size"] = self.tgt_font_spin.value()
    self.settings["auto_save_on_segment_change"] = self.auto_save_cb.isChecked()
    self.settings["show_tag_placeholders"] = self.show_tags_cb.isChecked()
    self.settings.save()
    self.accept()

class TagViewDialog(QDialog):
    def __init__(self, segment, column, parent=None):
        super().__init__(parent)
        self.segment = segment
        self.column = column
        self.setWindowTitle("View Tags")
        self.setAccessibleName("Tag Viewer")
        self.setAccessibleDescription(
            "Lists all tags in this segment. Select a tag and press Insert to add its placeholder to the translation." if column == 1
            else "Lists all tags in this segment for reference.")
        self.setMinimumWidth(500)
        self.setMinimumHeight(350)
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout(self)

        tags = self.segment.get("all_tags", [])
        label = QLabel(f"Tags in this segment ({len(tags)}):")
        label.setAccessibleName("Tags heading")
        layout.addWidget(label)

        self.tag_list = QListWidget()
        self.tag_list.setAccessibleName("Tag list")
        self.tag_list.setAccessibleDescription(
            "Select a tag from the list, then press the Insert button to add it to your translation." if self.column == 1
            else "List of tags found in the source segment.")
        if not tags:
            item = QListWidgetItem("No tags in this segment")
            self.tag_list.addItem(item)
            self.tag_list.item(0).setFlags(Qt.NoItemFlags)
        else:
            for i, tag in enumerate(tags):
                ph = f"{PH_L}{i}{PH_R}"
                display = f"{ph} = {tag}"
                item = QListWidgetItem(display)
                item.setData(Qt.UserRole, i)
                item.setToolTip(f"Placeholder: {ph}" if self.column == 0 else f"Click to select, then press Insert to add {ph} to the target")
                self.tag_list.addItem(item)

        layout.addWidget(self.tag_list)

        btn_layout = QHBoxLayout()
        if self.column == 1 and tags:
            self.insert_btn = QPushButton("Insert")
            self.insert_btn.setAccessibleName("Insert tag")
            self.insert_btn.setAccessibleDescription("Inserts the selected tag placeholder into your translation.")
            self.insert_btn.setToolTip("Insert selected tag at cursor in the target cell")
            self.insert_btn.clicked.connect(self._insert_selected_tag)
            self.insert_btn.setEnabled(False)
            self.tag_list.itemClicked.connect(lambda _: self.insert_btn.setEnabled(True))
            self.tag_list.itemDoubleClicked.connect(self._insert_selected_tag)
            btn_layout.addWidget(self.insert_btn)

        close_btn = QPushButton("Close")
        close_btn.setAccessibleName("Close tag viewer")
        close_btn.setAccessibleDescription("Closes this tag viewer dialog.")
        close_btn.clicked.connect(self.accept)
        btn_layout.addStretch()
        btn_layout.addWidget(close_btn)
        layout.addLayout(btn_layout)

    def _insert_selected_tag(self):

```

```

        item = self.tag_list.currentItem()
        if item is None:
            return
        tag_idx = item.data(Qt.UserRole)
        if tag_idx is None:
            return
        parent = self.parent()
        if not parent or not hasattr(parent, 'table') or parent.current_segment_index < 0:
            return
        cell_item = parent.table.item(parent.current_segment_index, 1)
        if cell_item is None:
            return
        ph = f"{PH_L}{tag_idx}{PH_R}"
        old = cell_item.text()
        new = old + ph
        cell_item.setText(new)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("mcat")
        self.resize(1100, 750)
        self.setStyleSheet(DARK_STYLESHEET)

        # Initialize backends
        data_dir = _get_data_dir()
        self.file_handler = CATFileHandler()
        self.tm = TranslationMemory(os.path.join(data_dir, "translation_memory.db"))
        self.glossary = Glossary(os.path.join(data_dir, "glossary.db"))
        self.project = None

        self.current_segment_index = -1
        self.modified = False
        self.segmentation_rules = [dict(r) for r in DEFAULT_RULES]
        self._load_segmentation_rules()

        self.settings = AppSettings().load()
        self._spellchecker = SpellChecker()
        self._spellchecker.auto_load()

        self.init_menu()
        self.init_ui()
        self._apply_settings()
        self.update_stats()

    def init_menu(self):
        menubar = self.menuBar()

        # File Menu
        file_menu = menubar.addMenu("&File")

        self.new_project_action = file_menu.addAction("&New Project...")
        self.new_project_action.setShortcut(QKeySequence("Ctrl+N"))
        self.new_project_action.setStatusTip("Create a new translation project")
        self.new_project_action.triggered.connect(self.new_project)

        self.open_project_action = file_menu.addAction("&Open Project...")
        self.open_project_action.setShortcut(QKeySequence("Ctrl+Shift+O"))
        self.open_project_action.setStatusTip("Open an existing project")
        self.open_project_action.triggered.connect(self.open_project)

        self.save_project_action = file_menu.addAction("&Save Project")
        self.save_project_action.setShortcut(QKeySequence("Ctrl+Shift+S"))
        self.save_project_action.setStatusTip("Save the current project")
        self.save_project_action.triggered.connect(self.save_project)

        self.save_project_as_action = file_menu.addAction("Save Project &As...")
        self.save_project_as_action.setStatusTip("Save the project with a new name")
        self.save_project_as_action.triggered.connect(self.save_project_as)

        self.close_project_action = file_menu.addAction("&Close Project")
        self.close_project_action.setStatusTip("Close the current project")
        self.close_project_action.triggered.connect(self.close_project)

        file_menu.addSeparator()

        self.project_settings_action = file_menu.addAction("Project &Settings...")
        self.project_settings_action.triggered.connect(self.project_settings)

```

```

file_menu.addSeparator()

self.recent_menu = file_menu.addMenu("&Recent Projects")

file_menu.addSeparator()

import_menu = file_menu.addMenu("&Import Project")

import_omegat_action = import_menu.addAction("Import Omega&T Project...")
import_omegat_action.triggered.connect(self.import_omegat_project)

import_menu.addSeparator()

import_memoq_action = import_menu.addAction("Import &memoQ Project...")
import_memoq_action.triggered.connect(self.import_memoq_project)

import_menu.addSeparator()

import_sdl_action = import_menu.addAction("Import &SDL Trados SDLXLIFF...")
import_sdl_action.triggered.connect(self.import_sdlxliff)

file_menu.addSeparator()

open_action = file_menu.addAction("&Open File...")
open_action.setShortcut(QKeySequence.Open)
open_action.setStatusTip("Open a translation file (PO, MQXLIFF, SDLXLIFF, etc.)")
open_action.triggered.connect(self.open_file)

save_action = file_menu.addAction("&Save File")
save_action.setShortcut(QKeySequence.Save)
save_action.setStatusTip("Save the current translation file")
save_action.triggered.connect(self.save_file)

save_as_action = file_menu.addAction("Save File &As...")
save_as_action.setStatusTip("Save the translation file in a different format")
save_as_action.triggered.connect(self.save_file_as)

file_menu.addSeparator()

self.render_action = file_menu.addAction("&Render Translated File...")
self.render_action.setShortcut(QKeySequence("Ctrl+R"))
self.render_action.triggered.connect(self.render_translated_file)

file_menu.addSeparator()

exit_action = file_menu.addAction("E&xit")
exit_action.triggered.connect(self.close)

# Go / Navigation Menu
go_menu = menubar.addMenu("&Go")

self.next_seg_action = go_menu.addAction("&Next Segment")
self.next_seg_action.setShortcut(QKeySequence("Ctrl+Down"))
self.next_seg_action.triggered.connect(self._go_next_segment)

self.prev_seg_action = go_menu.addAction("&Previous Segment")
self.prev_seg_action.setShortcut(QKeySequence("Ctrl+Up"))
self.prev_seg_action.triggered.connect(self._go_prev_segment)

go_menu.addSeparator()

self.next_untranslated_action = go_menu.addAction("Next &Untranslated")
self.next_untranslated_action.setShortcut(QKeySequence("Ctrl+U"))
self.next_untranslated_action.triggered.connect(self._go_next_untranslated)

go_menu.addSeparator()

self.toggle_fuzzy_action = go_menu.addAction("Toggle &Fuzzy")
self.toggle_fuzzy_action.setShortcut(QKeySequence("Ctrl+F"))
self.toggle_fuzzy_action.triggered.connect(self._toggle_fuzzy)

go_menu.addSeparator()

self.first_seg_action = go_menu.addAction("&First Segment")
self.first_seg_action.setShortcut(QKeySequence("Ctrl+Home"))
self.first_seg_action.triggered.connect(self._go_first_segment)

self.last_seg_action = go_menu.addAction("&Last Segment")
self.last_seg_action.setShortcut(QKeySequence("Ctrl+End"))

```

```

self.last_seg_action.triggered.connect(self._go_last_segment)

# TM Menu
tm_menu = menubar.addMenu("&Translation Memory")
import_tm_action = tm_menu.addAction("&Import PO into TM...")
import_tm_action.setShortcut(QKeySequence("Ctrl+M"))
import_tm_action.triggered.connect(self.import_po_to_tm)

clear_tm_action = tm_menu.addAction("&Clear TM Database")
clear_tm_action.triggered.connect(self.clear_tm)

# Glossary Menu
glossary_menu = menubar.addMenu("&Glossary")
add_term_action = glossary_menu.addAction("&Add Term...")
add_term_action.setShortcut(QKeySequence("Ctrl+G"))
add_term_action.triggered.connect(self.add_glossary_term)

view_all_action = glossary_menu.addAction("&View/Manage Glossary")
view_all_action.setShortcut(QKeySequence("Ctrl+Shift+G"))
view_all_action.triggered.connect(self.view_glossary)

glossary_menu.addSeparator()
import_tm_action = glossary_menu.addAction("Import &TMX into Glossary...")
import_tm_action.triggered.connect(self.import_tm_to_glossary)
export_tm_action = glossary_menu.addAction("E&xport Glossary as TMX...")
export_tm_action.triggered.connect(self.export_glossary_as_tm)

glossary_menu.addSeparator()
collections_menu = glossary_menu.addMenu("&Collections")
create_coll_action = collections_menu.addAction("&Create Collection...")
create_coll_action.triggered.connect(self.create_collection_dialog)
manage_coll_action = collections_menu.addAction("&Manage Collections...")
manage_coll_action.triggered.connect(self.manage_collections)
collections_menu.addSeparator()
import_coll_tm_action = collections_menu.addAction("&Import Collection as TMX...")
import_coll_tm_action.triggered.connect(self.import_collection_tm)
export_coll_tm_action = collections_menu.addAction("E&xport Collection as TMX...")
export_coll_tm_action.triggered.connect(self.export_collection_tm)

# Tools Menu
tools_menu = menubar.addMenu("&Tools")
seg_rules_action = tools_menu.addAction("&Segmentation Rules...")
seg_rules_action.setShortcut(QKeySequence("Ctrl+Shift+T"))
seg_rules_action.triggered.connect(self.open_segmentation_rules)

tools_menu.addSeparator()

self.qa_action = tools_menu.addAction("&Quality Assurance...")
self.qa_action.setShortcut(QKeySequence("Ctrl+Shift+Q"))
self.qa_action.setStatusTip("Run quality assurance checks on the translation")
self.qa_action.triggered.connect(self.run_qa)

tools_menu.addSeparator()
self.import_dict_action = tools_menu.addAction("&Import Dictionary...")
self.import_dict_action.triggered.connect(self.import_dictionary)

tools_menu.addSeparator()
self.spellcheck_action = tools_menu.addAction("&Spell Check...")
self.spellcheck_action.setShortcut(QKeySequence("F7"))
self.spellcheck_action.triggered.connect(self.run_spellcheck)

tools_menu.addSeparator()
self.options_action = tools_menu.addAction("&Options...")
self.options_action.setShortcut(QKeySequence("Ctrl+,"))
self.options_action.triggered.connect(self.show_options)

# Help Menu
help_menu = menubar.addMenu("&Help")
self.shortcuts_action = help_menu.addAction("&Keyboard Shortcuts...")
self.shortcuts_action.setShortcuts(["F1", "Ctrl+Shift+H"])
self.shortcuts_action.triggered.connect(self.show_shortcuts)

help_menu.addSeparator()
get_dicts_action = help_menu.addAction("&Get Dictionaries...")
get_dicts_action.triggered.connect(self._open_dicts_page)

self._update_recent_menu()

def init_ui(self):

```

```

main_splitter = QSplitter(Qt.Horizontal)

# ----- LEFT PANEL: 2-Column Editable Table -----
left_widget = QWidget()
left_layout = QVBoxLayout(left_widget)
left_layout.setContentsMargins(0, 0, 0, 0)

self.table = QTableWidgetItem()
self.table.setColumnCount(2)
self.table.setHorizontalHeaderLabels(["Source Segment", "Translation"])
self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
self.table.horizontalHeader().setSectionResizeMode(0, QHeaderView.Stretch)
self.table.horizontalHeader().setSectionResizeMode(1, QHeaderView.Stretch)
self.table.setSelectionBehavior(QTableWidgetItem.SelectRows)
self.table.setSelectionMode(QTableWidgetItem.SingleSelection)
self.table.itemSelectionChanged.connect(self.segment_selected)
self.table.cellChanged.connect(self._on_target_cell_changed)
self.table.setToolTip("Main translation table: source segments (left) and your translations (right). Select a row to edit.")
self.table.setContextMenuPolicy(Qt.CustomContextMenu)
self.table.customContextMenuRequested.connect(self._table_context_menu)

sl = self.project.source_lang if self.project else None
tl = self.project.target_lang if self.project else None
self.completion_delegate = CompletionDelegate(self.table, self.tm, self.glossary, self.spellchecker, self.settings, source_lang=sl,
self.table.setItemDelegateForColumn(1, self.completion_delegate)

seg_label = QLabel("Segments")
left_layout.addWidget(seg_label)
left_layout.addWidget(self.table)

# Bottom button bar
btn_layout = QHBoxLayout()
self.fuzzy_checkbox = QCheckBox("Fuzzy / Needs Review")
self.fuzzy_checkbox.setToolTip("Mark this segment as fuzzy or needing review")
self.fuzzy_checkbox.stateChanged.connect(self.fuzzy_state_changed)

self.confirm_btn = QPushButton("Confirm & Next (Ctrl+Enter)")
self.confirm_btn.setToolTip("Confirm the current translation and move to the next segment")
self.confirm_btn.clicked.connect(self.confirm_active_segment)
self.confirm_btn.setShortcut(QKeySequence("Ctrl+Return"))

btn_layout.addWidget(self.fuzzy_checkbox)
btn_layout.addStretch()
btn_layout.addWidget(self.confirm_btn)
left_layout.addLayout(btn_layout)

# ----- RIGHT PANEL: Sidebars -----
right_widget = QWidget()
right_layout = QVBoxLayout(right_widget)
right_layout.setContentsMargins(0, 0, 0, 0)

tm_label = QLabel("TM Fuzzy Matches (Double-click to apply)")
right_layout.addWidget(tm_label)
self.tm_list = QListWidget()
self.tm_list.itemDoubleClicked.connect(self.apply_tm_suggestion)
right_layout.addWidget(self.tm_list)

glossary_label = QLabel("Detected Terminology / Glossary")
right_layout.addWidget(glossary_label)
self.glossary_table = QTableWidgetItem()
self.glossary_table.setColumnCount(2)
self.glossary_table.setHorizontalHeaderLabels(["Source Term", "Translation"])
self.glossary_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
self.glossary_table.setSelectionBehavior(QTableWidgetItem.SelectRows)
self.glossary_table.setSelectionMode(QTableWidgetItem.SingleSelection)
self.glossary_table.itemDoubleClicked.connect(self.insert_glossary_term)
right_layout.addWidget(self.glossary_table)

main_splitter.addWidget(left_widget)
main_splitter.addWidget(right_widget)
main_splitter.setStretchFactor(0, 2)
main_splitter.setStretchFactor(1, 1)

self.setCentralWidget(main_splitter)

# ----- STATUS BAR -----
self.status_bar = QStatusBar()
self.setStatusBar(self.status_bar)
self.progress_bar = QProgressBar()

```

```

self.progress_bar.setMaximumWidth(200)
self.progress_bar.setValue(0)
self.status_bar.addPermanentWidget(self.progress_bar)

self.status_label = QLabel("Ready")
self.status_bar.addWidget(self.status_label)

# ----- FILE OPERATIONS -----
def open_file(self):
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Open a new file anyway?",
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.No:
            return

    file_path, _ = QFileDialog.getOpenFileName(
        self, "Open Translation File", "", FILE_FILTER
    )
    if not file_path:
        return

    try:
        segments = self.file_handler.load_file(file_path)
        self.populate_table(segments)
        self.modified = False
        self.update_stats()
        self.status_label.setText(f"Loaded file: {os.path.basename(file_path)}")

        # Select first segment
        if segments:
            self.table.selectRow(0)
    except Exception as e:
        QMessageBox.critical(self, "Error Loading File", str(e))

def populate_table(self, segments):
    self.table.blockSignals(True)
    self.table.setRowCount(len(segments))
    for idx, seg in enumerate(segments):
        src_text = seg.get("source_clean", seg["source"])
        src_item = QTableWidgetItem(src_text)
        src_item.setFlags(src_item.flags() & ~Qt.ItemIsEditable)
        self.set_source_cell_style(src_item, seg)
        self.table.setItem(idx, 0, src_item)

        tgt_text = seg.get("target_clean", seg["target"])
        tgt_item = QTableWidgetItem(tgt_text)
        tgt_item.setFlags(tgt_item.flags() | Qt.ItemIsEditable)
        self.set_target_cell_style(tgt_item, seg)
        self.table.setItem(idx, 1, tgt_item)

    self.table.blockSignals(False)

    source_texts = [seg.get("source_clean", seg["source"]) for seg in segments]
    self.completion_delegate.set_source_texts(source_texts)

def get_status_text(self, seg):
    if seg["fuzzy"]:
        return "Fuzzy"
    elif seg["target"] and seg["target"].strip():
        return "Translated"
    else:
        return "Untranslated"

def set_source_cell_style(self, item, seg):
    if seg["fuzzy"]:
        item.setBackground(QColor("#3d2e00"))
        item.setForeground(QColor("#ff9800"))
    elif seg.get("translated", False) or (seg["target"] and seg["target"].strip()):
        item.setBackground(QColor("#002b00"))
        item.setForeground(QColor("#28a745"))
    else:
        item.setBackground(QColor("#1a1a1a"))
        item.setForeground(QColor("#a0a0a8"))

def set_target_cell_style(self, item, seg):
    if seg["fuzzy"]:

```

```

        item.setBackground(QColor("#3d2e00"))
        item.setForeground(QColor("#ff9800"))
    elif seg.get("translated", False) or (seg["target"] and seg["target"].strip()):
        item.setBackground(QColor("#003300"))
        item.setForeground(QColor("#ffffff"))
    else:
        item.setBackground(QColor("#1a1a1e"))
        item.setForeground(QColor("#808080"))

def _on_target_cell_changed(self, row, col):
    if col != 1:
        return
    seg = self.file_handler.segments[row]
    item = self.table.item(row, 1)
    if item is None:
        return

    new_text = item.text().strip()
    tags = seg.get("all_tags", [])
    restored = restore_tags(new_text, tags)

    if seg["target"] == restored:
        return

    seg["target"] = restored
    seg["translated"] = bool(restored)
    new_tgt_tags = extract_all_tags(restored)
    all_tags = list(tags)
    for t in new_tgt_tags:
        if t not in all_tags:
            all_tags.append(t)
    seg["all_tags"] = all_tags
    seg["target_clean"] = apply_tags(restored, all_tags)

    if restored:
        self.tm.add_translation(seg["source"], restored, self._tm_sl, self._tm_tl)

    item.setText(seg["target_clean"])
    self.set_target_cell_style(item, seg)
    self.set_source_cell_style(self.table.item(row, 0), seg)

    self.modified = True
    self.update_stats()

def save_file(self):
    if not self.file_handler.current_file_path:
        self.save_file_as()
        return
    try:
        self.file_handler.save_file()
        self.modified = False
        self.status_label.setText(f"Saved: {os.path.basename(self.file_handler.current_file_path)}")
    except Exception as e:
        QMessageBox.critical(self, "Error Saving File", str(e))

def save_file_as(self):
    file_path, selected_filter = QFileDialog.getSaveFileName(
        self, "Save Translation File As", "",
        "MCAT Project (*.mcats.db);;PO Files (*.po);;memoQ MQXLIFF (*.mqxliff);;SDL Trados SDLXLIFF (*.sdlxliiff);;OmegaT Project folder;*"
    )
    if not file_path:
        return
    try:
        src_lang = self.project.source_lang if self.project else "en"
        tgt_lang = self.project.target_lang if self.project else "es"

        if "OmegaT" in selected_filter:
            dir_path = QFileDialog.getExistingDirectory(
                self, "Select folder to create OmegaT project in",
                os.path.dirname(file_path) if os.path.dirname(file_path) else ""
            )
            if not dir_path:
                return
            target_dir = os.path.join(dir_path, os.path.splitext(os.path.basename(file_path))[0])
            p = self.project if self.project else Project()
            if not self.project:
                p.new(os.path.basename(target_dir), src_lang, tgt_lang, [])
            p.save_as_omegat(target_dir)
            self.status_label.setText(f"OmegaT project saved: {target_dir}")
    
```

```

        QMessageBox.information(self, "Success",
                                f"Exported to OmegaT format:\n{target_dir}")
        return
    elif "MCAT" in selected_filter:
        if not file_path.lower().endswith(".mcat.db"):
            file_path += ".mcat.db"
        self.file_handler.save_file(file_path)
    elif "MQXLIFF" in selected_filter:
        if not file_path.endswith(".mqxliff"):
            file_path += ".mqxliff"
        self.file_handler.save_as_mqxliff(file_path, src_lang, tgt_lang)
    elif "SDLXLIFF" in selected_filter:
        if not file_path.endswith(".sdlxliiff"):
            file_path += ".sdlxliiff"
        from cat_tool.formats.sdlxliiff import SdlxliiffHandler
        SdlxliiffHandler().save(self.file_handler.segments, file_path, src_lang, tgt_lang)
        self.file_handler.current_file_path = file_path
    else:
        self.file_handler.save_file(file_path)
        self.modified = False
        self.status_label.setText(f"Saved: {os.path.basename(file_path)}")
        QMessageBox.information(self, "Success", "File saved successfully.")
except Exception as e:
    QMessageBox.critical(self, "Error Saving File", str(e))

# ----- RENDER TRANSLATED -----
def render_translated_file(self):
    if not self.file_handler.segments:
        QMessageBox.warning(self, "No Data", "No file loaded. Open a file first.")
        return
    ext = os.path.splitext(self.file_handler.current_file_path or "")[1].lower()
    ext_filter = f"Translated {ext.upper()} (*{ext})" if ext else "All Files (*)"
    file_path, _ = QFileDialog.getSaveFileName(
        self, "Save Translated File As", f"translated{ext}", ext_filter
    )
    if not file_path:
        return
    try:
        self.file_handler.render_translated(file_path)
        self.status_label.setText(f"Rendered: {os.path.basename(file_path)}")
        QMessageBox.information(self, "Success", f"Translated file saved:\n{file_path}")
    except Exception as e:
        QMessageBox.critical(self, "Render Error", str(e))

# ----- SELECTION & CONFIRMATION -----
def segment_selected(self):
    row = self.table.currentRow()
    if row < 0:
        return

    self.current_segment_index = row
    seg = self.file_handler.segments[row]

    self.fuzzy_checkbox.blockSignals(True)
    self.fuzzy_checkbox.setChecked(seg["fuzzy"])
    self.fuzzy_checkbox.blockSignals(False)

    self.update_fuzzy_matches(seg["source"])
    self.update_glossary_matches(seg["source"])

def fuzzy_state_changed(self, state):
    if self.current_segment_index < 0:
        return
    seg = self.file_handler.segments[self.current_segment_index]
    seg["fuzzy"] = (state == Qt.Checked)
    self.modified = True

    # Update table status cell
    self.update_table_row_status(self.current_segment_index, seg)
    self.update_stats()

def update_table_row_status(self, row, seg):
    self.table.blockSignals(True)
    item = self.table.item(row, 1)
    if item:
        item.setText(seg.get("target_clean", seg["target"]))
        self.set_target_cell_style(item, seg)
    src_item = self.table.item(row, 0)
    if src_item:

```

```

        self.set_source_cell_style(src_item, seg)
self.table.blockSignals(False)

def confirm_active_segment(self):
    if self.current_segment_index < 0:
        return

    row = self.current_segment_index
    item = self.table.item(row, 1)
    cell_text = item.text().strip() if item else ""

    seg = self.file_handler.segments[row]
    tags = seg.get("all_tags", [])
    restored = restore_tags(cell_text, tags)
    if seg["target"] == restored and seg["translated"] == bool(restored):
        pass
    else:
        seg["target"] = restored
        seg["translated"] = bool(restored)
        new_tgt_tags = extract_all_tags(restored)
        all_tags = list(tags)
        for t in new_tgt_tags:
            if t not in all_tags:
                all_tags.append(t)
        seg["all_tags"] = all_tags
        seg["target_clean"] = apply_tags(restored, all_tags)
        if restored:
            self.tm.add_translation(seg["source"], restored, self._tm_sl, self._tm_tl)

    self.modified = True
    if item:
        item.setText(seg["target_clean"])
    self.update_table_row_status(row, seg)

self.update_stats()

if self.modified and self.file_handler.current_file_path:
    try:
        self.file_handler.save_file()
        self.modified = False
    except Exception as e:
        self.status_label.setText(f"Auto-save failed: {e}")

if row < len(self.file_handler.segments) - 1:
    self.table.selectRow(row + 1)
    self._edit_target_cell(row + 1)
else:
    self.status_label.setText("Reached the end of the file!")

# ----- SEGMENT NAVIGATION -----
def _edit_target_cell(self, row):
    idx = self.table.model().index(row, 1)
    self.table.edit(idx)

def _go_next_segment(self):
    if not self.file_handler.segments:
        return
    next_idx = self.current_segment_index + 1
    if next_idx < len(self.file_handler.segments):
        self.table.selectRow(next_idx)
        self._edit_target_cell(next_idx)
    else:
        self.status_label.setText("Already at the last segment.")

def _go_prev_segment(self):
    if not self.file_handler.segments:
        return
    prev_idx = self.current_segment_index - 1
    if prev_idx >= 0:
        self.table.selectRow(prev_idx)
        self._edit_target_cell(prev_idx)
    else:
        self.status_label.setText("Already at the first segment.")

def _go_next_untranslated(self):
    if not self.file_handler.segments:
        return
    start = self.current_segment_index + 1
    for i in range(start, len(self.file_handler.segments)):

```

```

        seg = self.file_handler.segments[i]
        if not seg.get("translated", False) or not seg.get("target", "").strip():
            self.table.selectRow(i)
            self._edit_target_cell(i)
            return
        self.status_label.setText("No more untranslated segments.")

# ----- CONTEXT MENU -----
def _table_context_menu(self, pos):
    index = self.table.indexAt(pos)
    if not index.isValid():
        return
    row = index.row()
    col = index.column()
    if row < 0 or row >= len(self.file_handler.segments):
        return
    seg = self.file_handler.segments[row]
    self.table.selectRow(row)

    menu = QMenu(self)
    view_action = menu.addAction("View Tags")
    if col == 0:
        view_action.triggered.connect(lambda: TagViewDialog(seg, 0, self).exec_())
    else:
        view_action.triggered.connect(lambda: self._show_tag_dialog_for_target(seg))
    menu.addSeparator()
    spell_action = menu.addAction("Spell Check Segment")
    spell_action.triggered.connect(lambda: self._spellcheck_segment(seg))
    menu.exec_(self.table.viewport().mapToGlobal(pos))

def _show_tag_dialog_for_target(self, seg):
    dlg = TagViewDialog(seg, 1, self)
    dlg.exec_()

def _spellcheck_segment(self, seg):
    dlg = SpellCheckDialog([seg], self._spellchecker, self)
    dlg.exec_()
    self.populate_table(self.file_handler.segments)

def show_options(self):
    dlg = SettingsDialog(self.settings, self)
    if dlg.exec_() == QDialog.Accepted:
        self._apply_settings()

def open_file_path(self, file_path):
    if not os.path.exists(file_path):
        return
    try:
        segments = self.file_handler.load_file(file_path)
        self.populate_table(segments)
        self.modified = False
        self.update_stats()
        self.status_label.setText(f"Loaded file: {os.path.basename(file_path)}")
        if segments:
            self.table.selectRow(0)
    except Exception as e:
        QMessageBox.critical(self, "Error Loading File", str(e))

def _apply_settings(self):
    font = self.table.font()
    font.setPointSize(self.settings["source_font_size"])
    self.table.horizontalHeaderItem(0).setFont(font) if self.table.horizontalHeaderItem(0) else None
    if self.settings["source_font_size"] != self.settings["target_font_size"]:
        tgt_font = QFont(font)
        tgt_font.setPointSize(self.settings["target_font_size"])
        self.table.horizontalHeaderItem(1).setFont(tgt_font) if self.table.horizontalHeaderItem(1) else None
    self.table.setFont(font)

# ----- SIDEBAR SUGGESTIONS & INTERACTIONS -----
def update_fuzzy_matches(self, source_text):
    self.tm_list.clear()
    matches = self.tm.get_fuzzy_matches(source_text, min_score=40.0, source_lang=self._tm_sl or None, target_lang=self._tm_tl or None)

    for m in matches:
        item = QListWidgetItem(f"[{m['score']}]% Match] {m['target']}\n(From: \"{m['source']}\")")
        item.setData(Qt.UserRole, m["target"])
        self.tm_list.addItem(item)
def apply_tm_suggestion(self, item):

```

```

target_suggestion = item.data(Qt.UserRole)
if target_suggestion and self.current_segment_index >= 0:
    tgt_item = self.table.item(self.current_segment_index, 1)
    if tgt_item:
        tgt_item.setText(target_suggestion)
        self._edit_target_cell(self.current_segment_index)

def update_glossary_matches(self, source_text):
    self.glossary_table.setRowCount(0)
    matches = self.glossary.check_segment(source_text)

    self.glossary_table.setRowCount(len(matches))
    for idx, m in enumerate(matches):
        src_item = QTableWidgetItem(m["source"])
        tgt_item = QTableWidgetItem(m["target"])

        src_item.setFlags(src_item.flags() & ~Qt.ItemIsEditable)
        tgt_item.setFlags(tgt_item.flags() & ~Qt.ItemIsEditable)

        # Tooltip shows details/description
        if m["description"]:
            src_item.setToolTip(m["description"])
            tgt_item.setToolTip(m["description"])

        self.glossary_table.setItem(idx, 0, src_item)
        self.glossary_table.setItem(idx, 1, tgt_item)

def insert_glossary_term(self, item):
    glossary_row = item.row()
    term_item = self.glossary_table.item(glossary_row, 1)
    if not term_item or self.current_segment_index < 0:
        return
    tgt_item = self.table.item(self.current_segment_index, 1)
    if tgt_item:
        current = tgt_item.text()
        tgt_item.setText(current + term_item.text())
        self._edit_target_cell(self.current_segment_index)

# ----- GLOSSARY MANAGEMENT -----
def add_glossary_term(self):
    src_lang = self.project.source_lang if self.project else "en"
    tgt_lang = self.project.target_lang if self.project else ""
    dial = GlossaryDialog(self, source_lang=src_lang, target_lang=tgt_lang)
    if dial.exec_() == QDialog.Accepted:
        src, tgt, desc, sl, tl = dial.get_data()
        if src and tgt:
            self.glossary.add_term(src, tgt, desc, source_lang=sl, target_lang=tl)
            self.status_label.setText(f"Added glossary term: '{src}'")
            if self.current_segment_index >= 0:
                seg = self.file_handler.segments[self.current_segment_index]
                self.update_glossary_matches(seg["source"])
        else:
            QMessageBox.warning(self, "Invalid Input", "Source and Target cannot be empty.")

def import_tmj_to_glossary(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Import TMJ into Glossary", "",
        "TMJ Files (*.tmj);;All Files (*)")
    if not path:
        return
    try:
        info = self.glossary.peak_tmj(path)
        if info["count"] == 0 and not info.get("source_lang"):
            QMessageBox.information(self, "No Terms",
                "No translatable term pairs found in this TMJ file.")
            return

        sl = info["source_lang"] or (self.project.source_lang if self.project else "en")
        tl = info["target_lang"] or ""

        dlg = LangDialog(sl, tl, info["count"], self)
        if dlg.exec_() != QDialog.Accepted:
            return

        source_lang = dlg.src_combo.currentData() or sl
        target_lang = dlg.tgt_combo.currentData() or tl

        count = self.glossary.import_tmj(path, source_lang=source_lang)
        QMessageBox.information(

```

```

        self, "Import Complete",
        f"Imported {count} terms from TMX into glossary.\n"
        f"Source: {source_lang} | Target: {target_lang}")
except Exception as e:
    QMessageBox.critical(self, "Import Error", f"Failed to import TMX:\n{e}")

def export_glossary_as_tmx(self):
    path, _ = QFileDialog.getSaveFileName(
        self, "Export Glossary as TMX", "",
        "TMX Files (*.tmx);;All Files (*)")
    if not path:
        return
    if not path.lower().endswith(".tmx"):
        path += ".tmx"
    try:
        count = self.glossary.export_tmx(path)
        if count:
            QMessageBox.information(
                self, "Export Complete",
                f"Exported {count} terms to TMX.")
        else:
            QMessageBox.information(
                self, "Export Complete",
                "Glossary is empty - no terms exported.")
    except Exception as e:
        QMessageBox.critical(self, "Export Error", f"Failed to export TMX:\n{e}")

# ----- COLLECTIONS MANAGEMENT -----
def create_collection_dialog(self):
    dialog = QDialog(self)
    dialog.setWindowTitle("Create Collection")
    dialog.setMinimumWidth(400)
    layout = QFormLayout(dialog)
    name_edit = QLineEdit()
    name_edit.setPlaceholderText("My Collection")
    desc_edit = QLineEdit()
    desc_edit.setPlaceholderText("Optional description")
    layout.addRow("Collection Name:", name_edit)
    layout.addRow("Description:", desc_edit)
    btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
    btn_box.accepted.connect(dialog.accept)
    btn_box.rejected.connect(dialog.reject)
    layout.addRow(btn_box)
    if dialog.exec_() != QDialog.Accepted:
        return
    name = name_edit.text().strip()
    if not name:
        QMessageBox.warning(self, "Invalid", "Collection name cannot be empty.")
        return
    coll_id = self.glossary.create_collection(name, desc_edit.text().strip())
    if coll_id is None:
        QMessageBox.warning(self, "Duplicate", f"Collection '{name}' already exists.")
        return
    self.status_label.setText(f"Created collection: '{name}'")

def manage_collections(self):
    collections = self.glossary.get_all_collections()
    if not collections:
        if QMessageBox.question(self, "No Collections",
            "No collections yet. Create one now?",
            QMessageBox.Yes | QMessageBox.No) == QMessageBox.Yes:
            self.create_collection_dialog()
            collections = self.glossary.get_all_collections()
    if not collections:
        return

    dialog = QDialog(self)
    dialog.setWindowTitle("Manage Collections")
    dialog.setMinimumSize(700, 450)
    layout = QVBoxLayout(dialog)
    hsplit = QSplitter(Qt.Horizontal)

    left_widget = QWidget()
    left_layout = QVBoxLayout(left_widget)
    left_layout.setContentsMargins(0, 0, 0, 0)
    left_layout.addWidget(QLabel("Collections:"))
    coll_list = QListWidget()
    left_layout.addWidget(coll_list)
    coll_btn_layout = QHBoxLayout()

```

```

create_btn = QPushButton("Create")
create_btn.setToolTip("Create a new collection")
delete_btn = QPushButton("Delete")
delete_btn.setToolTip("Delete the selected collection")
coll_btn_layout.addWidget(create_btn)
coll_btn_layout.addWidget(delete_btn)
left_layout.addLayout(coll_btn_layout)

right_widget = QWidget()
right_layout = QVBoxLayout(right_widget)
right_layout.setContentsMargins(0, 0, 0, 0)
right_layout.addWidget(QLabel("Terms in collection:"))
terms_tbl = QTableWidget()
terms_tbl.setColumnCount(4)
terms_tbl.setHorizontalHeaderLabels(["Source", "Translation", "SL", "TL"])
terms_tbl.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
right_layout.addWidget(terms_tbl)
term_btn_layout = QHBoxLayout()
add_term_btn = QPushButton("Add Term...")
add_term_btn.setToolTip("Select a glossary term to add to this collection")
rem_term_btn = QPushButton("Remove Selected")
rem_term_btn.setToolTip("Remove the selected term from this collection")
term_btn_layout.addWidget(add_term_btn)
term_btn_layout.addWidget(rem_term_btn)
term_btn_layout.addStretch()
right_layout.addLayout(term_btn_layout)

hsplit.addWidget(left_widget)
hsplit.addWidget(right_widget)
hsplit.setStretchFactor(0, 1)
hsplit.setStretchFactor(1, 2)
layout.addWidget(hsplit)

btn_layout = QHBoxLayout()
close_btn = QPushButton("Close")
close_btn.setToolTip("Close collections manager")
close_btn.clicked.connect(dialog.accept)
btn_layout.addStretch()
btn_layout.addWidget(close_btn)
layout.addLayout(btn_layout)

coll_map = {}
def refresh_collection_list():
    coll_list.clear()
    coll_map.clear()
    for c in self.glossary.get_all_collections():
        coll_list.addItem(f"{c['name']} ({c['description']})" if c['description'] else c['name'])
        coll_map[coll_list.count() - 1] = c["id"]

def refresh_terms():
    terms_tbl.setRowCount(0)
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        return
    cid = coll_map[sel]
    terms = self.glossary.get_collection_terms(cid)
    terms_tbl.setRowCount(len(terms))
    for idx, t in enumerate(terms):
        terms_tbl.setItem(idx, 0, QTableWidgetItem(t["source"]))
        terms_tbl.setItem(idx, 1, QTableWidgetItem(t["target"]))
        terms_tbl.setItem(idx, 2, QTableWidgetItem(t.get("source_lang", "")))
        terms_tbl.setItem(idx, 3, QTableWidgetItem(t.get("target_lang", "")))

def on_coll_selected():
    refresh_terms()

def do_create():
    dialog.close()
    self.create_collection_dialog()
    refresh_collection_list()

def do_delete():
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        return
    cid = coll_map[sel]
    name = coll_list.currentItem().text().split(" ")[0] if coll_list.currentItem() else ""
    if QMessageBox.question(dialog, "Confirm Delete",
        f"Delete collection '{name}' and all its term associations?",

```

```

        QMessageBox.Yes | QMessageBox.No) != QMessageBox.Yes:
            return
        self.glossary.delete_collection(cid)
        self.status_label.setText(f"Deleted collection: '{name}'")
        refresh_collection_list()
        terms_tbl.setRowCount(0)

def do_add_term():
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        QMessageBox.information(dialog, "No Collection", "Select a collection first.")
        return
    cid = coll_map[sel]
    all_terms = self.glossary.get_all_terms()
    if not all_terms:
        QMessageBox.information(dialog, "No Terms", "The glossary is empty.")
        return
    existing = {t["source"] for t in self.glossary.get_collection_terms(cid)}
    available = [t for t in all_terms if t["source"] not in existing]
    if not available:
        QMessageBox.information(dialog, "All Added", "All glossary terms are already in this collection.")
        return
    picker = QDialog(dialog)
    picker.setWindowTitle("Add Term to Collection")
    picker.setMinimumWidth(450)
    ply = QVBoxLayout(picker)
    ply.addWidget(QLabel("Select a term to add:"))
    pick_list = QListWidget()
    for t in available:
        pick_list.addItem(f"{t['source']} → {t['target']}")
    ply.addWidget(pick_list)
    pb = QHBoxLayout()
    ok_btn = QPushButton("Add")
    ok_btn.clicked.connect(picker.accept)
    cancel_btn = QPushButton("Cancel")
    cancel_btn.clicked.connect(picker.reject)
    pb.addWidget(ok_btn)
    pb.addWidget(cancel_btn)
    ply.addLayout(pb)
    if picker.exec_() != QDialog.Accepted or pick_list.currentRow() < 0:
        return
    selected_term = available[pick_list.currentRow()]["source"]
    self.glossary.add_term_to_collection(cid, selected_term)
    self.status_label.setText(f"Added '{selected_term}' to collection")
    refresh_terms()

def do_remove_term():
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        return
    cid = coll_map[sel]
    row = terms_tbl.currentRow()
    if row < 0:
        return
    src = terms_tbl.item(row, 0).text()
    if QMessageBox.question(dialog, "Confirm Remove",
        f"Remove '{src}' from this collection? (Term stays in glossary)",
        QMessageBox.Yes | QMessageBox.No) != QMessageBox.Yes:
        return
    self.glossary.remove_term_from_collection(cid, src)
    self.status_label.setText(f"Removed '{src}' from collection")
    refresh_terms()

coll_list.currentRowChanged.connect(on_coll_selected)
create_btn.clicked.connect(do_create)
delete_btn.clicked.connect(do_delete)
add_term_btn.clicked.connect(do_add_term)
rem_term_btn.clicked.connect(do_remove_term)

refresh_collection_list()
if coll_list.count() > 0:
    coll_list.setCurrentRow(0)
dialog.exec_()

def import_collection_tmx(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Import Collection as TMX", "",
        "TMX Files (*.tmx);;All Files (*)")
    if not path:

```

```

        return
    try:
        count, coll_id = self.glossary.import_collection_tmx(path)
        if coll_id is None:
            QMessageBox.warning(self, "Import Failed",
                                "Could not create collection. Name may already exist.")
            return
        if count:
            QMessageBox.information(self, "Import Complete",
                                    f"Imported {count} terms into a new collection.")
        else:
            QMessageBox.information(self, "No Terms",
                                    "No translatable term pairs found in that TMX.")
    except Exception as e:
        QMessageBox.critical(self, "Import Error", str(e))

def export_collection_tmx(self):
    collections = self.glossary.get_all_collections()
    if not collections:
        QMessageBox.information(self, "No Collections", "No collections to export.")
        return
    coll_names = [c["name"] for c in collections]
    picker = QDialog(self)
    picker.setWindowTitle("Export Collection as TMX")
    picker.setMinimumWidth(400)
    ply = QVBoxLayout(picker)
    ply.addWidget(QLabel("Select a collection to export:"))
    pick_list = QListWidget()
    for n in coll_names:
        pick_list.addItem(n)
    ply.addWidget(pick_list)
    pb = QHBoxLayout()
    ok_btn = QPushButton("Export")
    ok_btn.clicked.connect(picker.accept)
    cancel_btn = QPushButton("Cancel")
    cancel_btn.clicked.connect(picker.reject)
    pb.addWidget(ok_btn)
    pb.addWidget(cancel_btn)
    ply.addLayout(pb)
    if picker.exec_() != QDialog.Accepted or pick_list.currentRow() < 0:
        return
    coll = collections[pick_list.currentRow()]
    path, _ = QFileDialog.getSaveFileName(
        self, "Save Collection TMX", f"{coll['name']}.tmx",
        "TMX Files (*.tmx);;All Files (*)")
    if not path:
        return
    if not path.lower().endswith(".tmx"):
        path += ".tmx"
    try:
        count = self.glossary.export_collection_tmx(coll["id"], path)
        if count:
            QMessageBox.information(self, "Export Complete",
                                    f"Exported {count} terms from '{coll['name']}' to TMX.")
        else:
            QMessageBox.information(self, "Empty Collection",
                                    "That collection has no terms.")
    except Exception as e:
        QMessageBox.critical(self, "Export Error", str(e))

def view_glossary(self):
    terms = self.glossary.get_all_terms()
    dialog = QDialog(self)
    dialog.setWindowTitle("Glossary Terms List")
    dialog.setMinimumSize(650, 400)

    layout = QVBoxLayout(dialog)
    tbl = QTableWidget()
    tbl.setColumnCount(5)
    tbl.setHorizontalHeaderLabels(["Source Term", "SL", "Target Translation", "TL", "Description"])
    tbl.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    tbl.setRowCount(len(terms))

    for idx, t in enumerate(terms):
        tbl.setItem(idx, 0, QTableWidgetItem(t["source"]))
        tbl.setItem(idx, 1, QTableWidgetItem(t.get("source_lang", "")))
        tbl.setItem(idx, 2, QTableWidgetItem(t["target"]))
        tbl.setItem(idx, 3, QTableWidgetItem(t.get("target_lang", "")))
        tbl.setItem(idx, 4, QTableWidgetItem(t["description"]))

```

```

layout.addWidget(tbl)

btn_layout = QHBoxLayout()
del_btn = QPushButton("Delete Selected", dialog)
del_btn.setToolTip("Remove the selected glossary term")

def delete_selected():
    selected_items = tbl.selectedItems()
    if not selected_items:
        return
    row = selected_items[0].row()
    src = tbl.item(row, 0).text()
    confirm = QMessageBox.question(
        dialog, "Confirm Delete",
        f"Delete glossary term '{src}'?",
        QMessageBox.Yes | QMessageBox.No)
    if confirm != QMessageBox.Yes:
        return
    self.glossary.remove_term(src)
    tbl.removeRow(row)
    self.status_label.setText(f"Deleted term: '{src}'")
    if self.current_segment_index >= 0:
        seg = self.file_handler.segments[self.current_segment_index]
        self.update_glossary_matches(seg["source"])

del_btn.clicked.connect(delete_selected)
btn_layout.addWidget(del_btn)
close_btn = QPushButton("Close", dialog)
close_btn.setToolTip("Close the glossary list")
close_btn.clicked.connect(dialog.accept)
btn_layout.addWidget(close_btn)
btn_layout.addStretch()
layout.addLayout(btn_layout)

dialog.exec_()

# ----- TM MANAGEMENT -----
def import_po_to_tm(self):
    file_path, _ = QFileDialog.getOpenFileName(
        self, "Import PO for Translation Memory", "", "PO Files (*.po);;All Files (*)"
    )
    if not file_path:
        return
    try:
        handler = CATFileHandler()
        segments = handler.load_file(file_path)
        imported_count = 0
        for seg in segments:
            if seg["target"] and seg["target"].strip() and not seg["fuzzy"]:
                self.tm.add_translation(seg["source"], seg["target"], self.tm_sl, self.tm_tl)
                imported_count += 1

        QMessageBox.information(
            self, "Success", f"Imported {imported_count} translations into translation memory."
        )
        # Refresh matching if active
        if self.current_segment_index >= 0:
            seg = self.file_handler.segments[self.current_segment_index]
            self.update_fuzzy_matches(seg["source"])
    except Exception as e:
        QMessageBox.critical(self, "Import Error", str(e))

def clear_tm(self):
    reply = QMessageBox.question(
        self, "Confirm Delete", "Are you sure you want to clear the entire Translation Memory?",
        QMessageBox.Yes | QMessageBox.No
    )
    if reply == QMessageBox.Yes:
        self.tm.clear()
        self.status_label.setText("Translation Memory cleared.")
        if self.current_segment_index >= 0:
            seg = self.file_handler.segments[self.current_segment_index]
            self.update_fuzzy_matches(seg["source"])

def _toggle_fuzzy(self):
    if self.current_segment_index >= 0:
        self.fuzzy_checkbox.toggle()

def _go_first_segment(self):

```

```

        if self.file_handler.segments:
            self.table.selectRow(0)
            self._edit_target_cell(0)

def _go_last_segment(self):
    if self.file_handler.segments:
        last = len(self.file_handler.segments) - 1
        self.table.selectRow(last)
        self._edit_target_cell(last)

def show_shortcuts(self):
    dlg = ShortcutsDialog(self)
    dlg.exec_()

def _open_dicts_page(self):
    import webbrowser
    webbrowser.open("https://extensions.libreoffice.org/en?Tags%5B%5D=50")

def navigate_to_segment(self, segment_index):
    if 0 <= segment_index < len(self.file_handler.segments):
        self.table.selectRow(segment_index)
        self._edit_target_cell(segment_index)

def run_qa(self):
    if not self.file_handler.segments:
        QMessageBox.warning(self, "No Data", "Open a file first before running QA.")
        return
    issues = qa.run_all_checks(self.file_handler.segments, glossary=self.glossary)
    dlg = QADialog(issues, self)
    dlg.exec_()

def import_dictionary(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Import Dictionary", "",
        "Dictionary (*.oxf *.zip *.dic);;All Files (*)"
    )
    if not path:
        return
    if path.endswith(".dic"):
        ok = self._spellchecker.load_dictionary(path)
    else:
        ok = self._spellchecker.load_from_archive(path)
    if ok:
        names = self._spellchecker.loaded_names
        QMessageBox.information(
            self, "Success",
            f"Dictionary imported.\nLoaded: {' '.join(names)}"
        )
    else:
        QMessageBox.warning(self, "Error", "Failed to load dictionary.\n\n"
            "Make sure the file contains valid .dic/.aff files.")

def _load_spelling_dict_for_language(self, lang_code):
    if not lang_code:
        return
    from cat_tool.spellcheck import DICTS_DIR
    code = lang_code.lower()
    found = None
    if os.path.isdir(DICTS_DIR):
        for name in os.listdir(DICTS_DIR):
            if name.lower().startswith(code):
                found = name
                break
    if found:
        if found not in self._spellchecker.loaded_names:
            path = os.path.join(DICTS_DIR, found)
            for f in os.listdir(path):
                if f.endswith(".dic"):
                    base = os.path.join(path, os.path.splitext(f)[0])
                    try:
                        from spylls.hunspell import Dictionary
                        d = Dictionary.from_files(base)
                        self._spellchecker.dictionaries.append(d)
                        self._spellchecker._loaded_names.append(found)
                    except Exception:
                        pass
            break
        self.status_label.setText(
            f"Loaded spelling dictionary: {found}"

```

```

    )
else:
    self.status_label.setText(
        f"No spelling dictionary found for '{lang_code}'. "
        f"Use Tools &gt; Import Dictionary to add one."
    )

def run_spellcheck(self):
    if not self.file_handler.segments:
        QMessageBox.warning(self, "No Data", "Open a file first before spell check.")
        return
    dlg = SpellCheckDialog(self.file_handler.segments, self._spellchecker, self)
    dlg.exec_()
    self.populate_table(self.file_handler.segments)

# ----- STATS UPDATE -----
def update_stats(self):
    if not self.file_handler.segments:
        self.progress_bar.setValue(0)
        self.progress_bar.setFormat("No file loaded")
        return

    total = len(self.file_handler.segments)
    translated = 0
    fuzzy = 0
    total_words = 0
    total_chars = 0
    translated_words = 0

    for seg in self.file_handler.segments:
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        total_words += len(src.split())
        total_chars += len(src)
        if seg["fuzzy"]:
            fuzzy += 1
        elif tgt.strip():
            translated += 1
            translated_words += len(tgt.split())

    percent = int((translated / total) * 100) if total > 0 else 0
    self.progress_bar.setValue(percent)
    self.progress_bar.setFormat(f"%p% ({translated}/{total})")

    proj_name = self.project.name if self.project else "No Project"
    self.status_label.setText(
        f"[{proj_name}] "
        f"Active File: {os.path.basename(self.file_handler.current_file_path or 'None')} | "
        f"Translated: {translated} | Fuzzy: {fuzzy} | Total: {total} | "
        f"Words: {total_words} | Chars: {total_chars}"
    )
    title = f"mcat - {proj_name}"
    if self.file_handler.current_file_path:
        title += f" [{os.path.basename(self.file_handler.current_file_path)}]"
    self.setWindowTitle(title)

# ----- PROJECT OPERATIONS -----
def close_project(self):
    if not self.project:
        return
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Save before closing?",
            QMessageBox.Yes | QMessageBox.No | QMessageBox.Cancel
        )
        if reply == QMessageBox.Cancel:
            return
        if reply == QMessageBox.Yes:
            self.save_project()

    proj_name = self.project.name
    self.project = None
    self.file_handler = CATFileHandler()
    self.current_segment_index = -1
    self.modified = False
    self.table.setRowCount(0)
    self.tm_list.clear()

```

```

self.glossary_table.setRowCount(0)
self.status_label.setText(f"Project closed: {proj_name}")

def new_project(self):
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Start a new project anyway?",
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.No:
            return

    dial = NewProjectDialog(self)
    if dial.exec_() != QDialog.Accepted:
        return

    name, src_lang, tgt_lang, src_files = dial.get_data()
    if not name:
        QMessageBox.warning(self, "Invalid", "Project name cannot be empty.")
        return

    self.project = Project()
    self.project.new(name, src_lang, tgt_lang, src_files)
    self.file_handler = CATFileHandler()
    self.current_segment_index = -1
    self.modified = False

    if src_files:
        self._open_project_source_file(0)
    else:
        self.table.setRowCount(0)

    self.update_stats()
    self._load_spelling_dict_for_language(tgt_lang)
    self.status_label.setText(f"Created project: {name}")

def open_project(self):
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Open a different project anyway?",
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.No:
            return

    filter_str = "CAT Project (*.mcatproj);;OmegaT Project (folder with omegat.project);;memoQ Project (folder with manifest.xml)"
    file_path, selected_filter = QFileDialog.getOpenFileName(
        self, "Open Project", "", filter_str
    )
    if not file_path:
        return

    if "OmegaT" in selected_filter:
        dir_path = QFileDialog.getExistingDirectory(
            self, "Select OmegaT Project Folder (contains omegat.project)"
        )
        if not dir_path:
            return
        proj_file = os.path.join(dir_path, "omegat.project")
        if not os.path.exists(proj_file):
            QMessageBox.warning(self, "Invalid Project",
                               "Selected folder does not contain omegat.project")
            return
        self._load_omegat(dir_path)
    elif "memoQ" in selected_filter:
        dir_path = QFileDialog.getExistingDirectory(
            self, "Select memoQ Project Folder (contains manifest.xml)"
        )
        if not dir_path:
            return
        manifest_file = os.path.join(dir_path, "manifest.xml")
        if not os.path.exists(manifest_file):
            QMessageBox.warning(self, "Invalid Project",
                               "Selected folder does not contain manifest.xml")
            return
        self._load_memoq(dir_path)
    else:

```

```

        self._load_project(file_path)

def _load_project(self, file_path):
    try:
        self.project = Project.load(file_path)
        self.file_handler = CATFileHandler()
        self.current_segment_index = -1
        self.modified = False

        if self.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)

        Project.add_recent(file_path, self.project.name)
        self._update_recent_menu()
        self.update_stats()
        self._load_spelling_dict_for_language(self.project.target_lang)
        self.status_label.setText(f"Opened project: {self.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Opening Project", str(e))

def _load_omegat(self, dir_path):
    try:
        self.project = Project.load_omegat(dir_path)
        self.file_handler = CATFileHandler()
        self.current_segment_index = -1
        self.modified = False

        if self.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)

        Project.add_recent(dir_path, self.project.name)
        self._update_recent_menu()
        self.update_stats()
        self._load_spelling_dict_for_language(self.project.target_lang)
        self.status_label.setText(f"Opened OmegaT project: {self.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Opening Project", str(e))

def project_settings(self):
    if not self.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return

    dial = ProjectSettingsDialog(self.project, self)
    if dial.exec_() != QDialog.Accepted:
        return

    dial.apply()
    self.modified = True
    self.update_stats()
    self._load_spelling_dict_for_language(self.project.target_lang)
    self.status_label.setText("Project settings updated.")

def _open_project_source_file(self, index):
    if not self.project or index >= len(self.project.source_files):
        return
    file_path = self.project.source_files[index]
    if not os.path.exists(file_path):
        QMessageBox.warning(self, "File Not Found", f"Source file not found:\n{file_path}")
        return

    ext = os.path.splitext(file_path)[1].lower()
    is_po = ext in (".po", ".pot")
    try:
        if is_po:
            self.file_handler.load_file(file_path)
        else:
            po_path = file_path + ".po"
            convert_to_po(file_path, po_path)
            self.file_handler.load_file(po_path)

        self.populate_table(self.file_handler.segments)
        if self.file_handler.segments:
            self.table.selectRow(0)
            self.update_stats()

```

```

        label = os.path.basename(file_path)
        if not is_po:
            label += " (auto-converted to PO)"
        self.status_label.setText(f"Loaded: {label}")
    except Exception as e:
        QMessageBox.critical(self, "Error Loading Source File", str(e))

def save_project(self):
    if not self.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return
    if not self.project.file_path:
        self.save_project_as()
        return
    try:
        self.project.save()
        self.modified = False
        Project.add_recent(self.project.file_path, self.project.name)
        self._update_recent_menu()
        self.update_stats()
        self.status_label.setText(f"Project saved: {self.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Saving Project", str(e))

def save_project_as(self):
    if not self.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return
    filter_str = "CAT Project (*.mcatproj);;OmegaT Project folder;;memoQ Project folder"
    file_path, selected_filter = QFileDialog.getSaveFileName(
        self, "Save Project As", f"{self.project.name}.mcatproj", filter_str
    )
    if not file_path:
        return

    if "OmegaT" in selected_filter:
        dir_path = QFileDialog.getExistingDirectory(
            self, "Select folder to create OmegaT project in",
            os.path.dirname(file_path) if os.path.dirname(file_path) else ""
        )
        if not dir_path:
            return
        target_dir = os.path.join(dir_path, self.project.name)
        try:
            self.project.save_as_omegat(target_dir)
            self.modified = False
            self.project.file_path = target_dir
            Project.add_recent(target_dir, self.project.name)
            self._update_recent_menu()
            self.update_stats()
            self.status_label.setText(
                f"OmegaT project saved: {target_dir}"
            )
            QMessageBox.information(
                self, "Success",
                f"Project exported to OmegaT format:\n{target_dir}\n\n"
                f"Source files copied to '{target_dir}\\source\\'.\n\n"
                f"You can open this folder directly in OmegaT."
            )
        except Exception as e:
            QMessageBox.critical(self, "Export Error", str(e))
    elif "memoQ" in selected_filter:
        dir_path = QFileDialog.getExistingDirectory(
            self, "Select folder to create memoQ project in",
            os.path.dirname(file_path) if os.path.dirname(file_path) else ""
        )
        if not dir_path:
            return
        target_dir = os.path.join(dir_path, self.project.name)
        try:
            self.project.save_as_memoq(target_dir, self.file_handler)
            self.modified = False
            self.project.file_path = target_dir
            Project.add_recent(target_dir, self.project.name)
            self._update_recent_menu()
            self.update_stats()
            self.status_label.setText(
                f"memoQ project saved: {target_dir}"
            )
        )

```

```

        QMessageBox.information(
            self, "Success",
            f"Project exported to memoQ format:\n{target_dir}\n\n"
            f"Source files copied to '{target_dir}\\source\\'.\n\n"
            f"Translations saved as MQXLIFF in '{target_dir}\\target\\'.\n\n"
            f"You can open the .mqxliff file directly in memoQ."
        )
    except Exception as e:
        QMessageBox.critical(self, "Export Error", str(e))
else:
    if not file_path.endswith(".mcatproj"):
        file_path += ".mcatproj"
    try:
        self.project.save(file_path)
        self.modified = False
        Project.add_recent(file_path, self.project.name)
        self.update_recent_menu()
        self.update_stats()
        self.status_label.setText(f"Project saved: {self.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Saving Project", str(e))

def _current_lang_code(self):
    if self.project and self.project.source_lang:
        return self.project.source_lang
    return "all"

def _load_segmentation_rules(self):
    from cat_tool.segmentation import LANG_NAME_TO_CODE
    all_rules = []
    for code in set(LANG_NAME_TO_CODE.values()):
        loaded = load_rules(code)
        if loaded:
            all_rules.extend(loaded)
    if all_rules:
        self.segmentation_rules = all_rules
    else:
        self.segmentation_rules = [dict(r) for r in DEFAULT_RULES]

@property
def _tm_sl(self):
    return self.project.source_lang if self.project else ""

@property
def _tm_tl(self):
    return self.project.target_lang if self.project else ""

def open_segmentation_rules(self):
    code = self._current_lang_code()
    dlg = AdvancedRulesDialog(self, self.segmentation_rules, lang_code=code)
    if dlg.exec_() == QDialog.Accepted:
        self.segmentation_rules = dlg.get_rules()
        self.status_label.setText(
            f"Segmentation rules saved to {code}.json ({len(self.segmentation_rules)} rules)"
        )

def import_omegat_project(self):
    dir_path = QFileDialog.getExistingDirectory(
        self, "Select OmegaT Project Folder (contains omegat.project)"
    )
    if not dir_path:
        return
    proj_file = os.path.join(dir_path, "omegat.project")
    if not os.path.exists(proj_file):
        QMessageBox.warning(self, "Invalid Project",
            "Selected folder does not contain omegat.project")
        return
    try:
        self.project = Project.load_omegat(dir_path)
        self.file_handler = CATFileHandler()
        self.current_segment_index = -1
        self.modified = False
        if self.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)
        Project.add_recent(dir_path, self.project.name)
        self.update_recent_menu()
        self.update_stats()

```

```

        self.status_label.setText(f"Imported OmegaT project: {self.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Import Error", str(e))

def export_omegat_project(self):
    if not self.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return
    dir_path = QFileDialog.getExistingDirectory(
        self, "Select folder to export OmegaT project into"
    )
    if not dir_path:
        return
    target_dir = os.path.join(dir_path, self.project.name)
    try:
        self.project.save_as_omegat(target_dir)
        QMessageBox.information(
            self, "Success",
            f"OmegaT project exported to:\n{target_dir}\n\n"
            f"Source files copied to '{target_dir}\\source\\'.\n"
            f"Open the folder in OmegaT to translate."
        )
        self.status_label.setText(f"Exported OmegaT project: {target_dir}")
    except Exception as e:
        QMessageBox.critical(self, "Export Error", str(e))

# ----- MEMOQ PROJECTS -----
def import_memoq_project(self):
    dir_path = QFileDialog.getExistingDirectory(
        self, "Select memoQ Project Folder (contains manifest.xml)"
    )
    if not dir_path:
        return
    manifest_file = os.path.join(dir_path, "manifest.xml")
    if not os.path.exists(manifest_file):
        QMessageBox.warning(self, "Invalid Project",
            "Selected folder does not contain manifest.xml")
        return
    self._load_memoq(dir_path)

def export_memoq_project(self):
    if not self.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return
    dir_path = QFileDialog.getExistingDirectory(
        self, "Select folder to export memoQ project into"
    )
    if not dir_path:
        return
    target_dir = os.path.join(dir_path, self.project.name)
    try:
        self.project.save_as_memoq(target_dir, self.file_handler)
        QMessageBox.information(
            self, "Success",
            f"memoQ project exported to:\n{target_dir}\n\n"
            f"Source files copied to '{target_dir}\\source\\'.\n"
            f"Translations saved as MQXLIFF in '{target_dir}\\target\\'."
        )
        self.status_label.setText(f"Exported memoQ project: {target_dir}")
    except Exception as e:
        QMessageBox.critical(self, "Export Error", str(e))

def _load_memoq(self, dir_path):
    try:
        self.project = Project.load_memoq(dir_path)
        self.file_handler = CATFileHandler()
        self.current_segment_index = -1
        self.modified = False

        if self.project.mqxliiff_files:
            self.file_handler.load_file(self.project.mqxliiff_files[0])
            self.populate_table(self.file_handler.segments)
            if self.file_handler.segments:
                self.table.selectRow(0)
        elif self.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)
        Project.add_recent(dir_path, self.project.name)

```

```

        self._update_recent_menu()
        self.update_stats()
        self._load_spelling_dict_for_language(self.project.target_lang)
        self.status_label.setText(f"Opened memoQ project: {self.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Opening memoQ Project", str(e))

# ----- SDL TRADOS -----
def import_sdlxliff(self):
    file_path, _ = QFileDialog.getOpenFileName(
        self, "Open SDL Trados SDLXLIFF", "", "SDLXLIFF (*.sdlxliff);;All Files (*)"
    )
    if not file_path:
        return
    try:
        segs = self.file_handler.load_file(file_path)
        self.populate_table(segs)
        self.modified = False
        self.update_stats()
        self.status_label.setText(f"Loaded SDLXLIFF: {os.path.basename(file_path)}")
        if segs:
            self.table.selectRow(0)
    except Exception as e:
        QMessageBox.critical(self, "Error Loading SDLXLIFF", str(e))

def export_sdlxliff(self):
    if not self.file_handler.segments:
        QMessageBox.information(self, "Nothing to Export", "No file loaded to export.")
        return
    file_path, _ = QFileDialog.getSaveFileName(
        self, "Export as SDL Trados SDLXLIFF", "", "SDLXLIFF (*.sdlxliff);;All Files (*)"
    )
    if not file_path:
        return
    if not file_path.endswith(".sdlxliff"):
        file_path += ".sdlxliff"
    try:
        src_lang = self.project.source_lang if self.project else "en"
        tgt_lang = self.project.target_lang if self.project else "es"
        from cat_tool.formats.sdlxliff import SdlxliffHandler
        SdlxliffHandler().save(self.file_handler.segments, file_path, src_lang, tgt_lang)
        self.status_label.setText(f"Exported SDLXLIFF: {os.path.basename(file_path)}")
        QMessageBox.information(self, "Success", f"Exported to SDLXLIFF:\n{file_path}")
    except Exception as e:
        QMessageBox.critical(self, "Export Error", str(e))

def _update_recent_menu(self):
    self.recent_menu.clear()
    recents = Project.get_recent()
    if not recents:
        action = self.recent_menu.addAction("(No recent projects)")
        action.setEnabled(False)
        return
    for r in recents:
        path = r.get("path", "")
        name = r.get("name", os.path.basename(path))
        action = self.recent_menu.addAction(f"{name} [{os.path.basename(os.path.dirname(path))}]")
        action.setData(path)
        action.triggered.connect(lambda checked, p=path: self._load_project(p))

```

cat_tooltm.py — 89 lines

- Class **TranslationMemory** (line 6)
- Function **__init__** (line 7)
- Function **__init_db** (line 11)
- Function **add_translation** (line 32)
- Function **clear** (line 52)
- Function **get_fuzzy_matches** (line 59)

```

import sqlite3
import os
from datetime import datetime
from rapidfuzz import fuzz

class TranslationMemory:

```

```

def __init__(self, db_path="translation_memory.db"):
    self.db_path = db_path
    self._init_db()

def _init_db(self):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS translation_memory (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            source TEXT UNIQUE,
            target TEXT,
            source_lang TEXT DEFAULT '',
            target_lang TEXT DEFAULT '',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    for col in ("source_lang", "target_lang"):
        try:
            cursor.execute(f"ALTER TABLE translation_memory ADD COLUMN {col} TEXT DEFAULT ''")
        except sqlite3.OperationalError:
            pass
    conn.commit()
    conn.close()

def add_translation(self, source, target, source_lang="", target_lang=""):
    if not source or not source.strip():
        return
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    try:
        cursor.execute("""
            INSERT INTO translation_memory (source, target, source_lang, target_lang)
            VALUES (?, ?, ?, ?)
            ON CONFLICT(source) DO UPDATE SET
                target = excluded.target,
                source_lang = excluded.source_lang,
                target_lang = excluded.target_lang
        """, (source.strip(), target.strip(), source_lang.strip(), target_lang.strip()))
        conn.commit()
    except Exception as e:
        print(f"Error adding to TM: {e}")
    finally:
        conn.close()

def clear(self):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM translation_memory")
    conn.commit()
    conn.close()

def get_fuzzy_matches(self, query_source, min_score=50.0, limit=5, source_lang=None, target_lang=None):
    if not query_source or not query_source.strip():
        return []

    query_source = query_source.strip()
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT source, target, source_lang, target_lang FROM translation_memory")
    rows = cursor.fetchall()
    conn.close()

    q = query_source.lower()
    matches = []
    for source, target, sl, tl in rows:
        if source_lang and sl and sl != source_lang:
            continue
        if target_lang and tl and tl != target_lang:
            continue
        s = source.lower()
        ratio = fuzz.ratio(q, s)
        sort_ratio = fuzz.token_sort_ratio(q, s)
        score = max(ratio, sort_ratio)
        if score >= min_score:
            matches.append({
                "source": source,
                "target": target,
                "score": round(score, 1)
            })

```

```

    })

    matches.sort(key=lambda x: x["score"], reverse=True)
    return matches[:limit]

```

cat_toolglossary.py — 360 lines

- Class **Glossary** (line 6)
- Function **__init__** (line 7)
- Function **_init_db** (line 11)
- Function **add_term** (line 48)
- Function **remove_term** (line 69)
- Function **get_all_terms** (line 77)
- Function **_tokenize** (line 85)
- Function **peek_tmx** (line 88)
- Function **import_tmx** (line 119)
- Function **export_tmx** (line 160)
- Function **create_collection** (line 198)
- Function **delete_collection** (line 211)
- Function **get_all_collections** (line 219)
- Function **add_term_to_collection** (line 227)
- Function **remove_term_from_collection** (line 238)
- Function **get_collection_terms** (line 246)
- Function **export_collection_tmx** (line 260)
- Function **import_collection_tmx** (line 289)
- Function **check_segment** (line 329)

```

import sqlite3
import os
import re
import xml.etree.ElementTree as ET

class Glossary:
    def __init__(self, db_path="glossary.db"):
        self.db_path = db_path
        self._init_db()

    def _init_db(self):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS glossary (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                source_term TEXT UNIQUE,
                target_translation TEXT,
                description TEXT,
                source_lang TEXT DEFAULT '',
                target_lang TEXT DEFAULT ''
            )
        """)
        for col in ("source_lang", "target_lang"):
            try:
                cursor.execute(f"ALTER TABLE glossary ADD COLUMN {col} TEXT DEFAULT ''")
            except sqlite3.OperationalError:
                pass
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS collections (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT UNIQUE,
                description TEXT DEFAULT '',
                created_at TEXT DEFAULT CURRENT_TIMESTAMP
            )
        """)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS collection_terms (
                collection_id INTEGER,
                term_source TEXT,
                PRIMARY KEY (collection_id, term_source),
                FOREIGN KEY (collection_id) REFERENCES collections(id) ON DELETE CASCADE

```

```

    )
    """
    conn.commit()
    conn.close()

def add_term(self, source, target, description="", source_lang="", target_lang=""):
    if not source or not target:
        return
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    try:
        cursor.execute("""
            INSERT INTO glossary (source_term, target_translation, description, source_lang, target_lang)
            VALUES (?, ?, ?, ?, ?)
            ON CONFLICT(source_term) DO UPDATE SET
                target_translation = excluded.target_translation,
                description = excluded.description,
                source_lang = excluded.source_lang,
                target_lang = excluded.target_lang
            """, (source.strip(), target.strip(), description.strip(), source_lang.strip(), target_lang.strip()))
        conn.commit()
    except Exception as e:
        print(f"Error adding to Glossary: {e}")
    finally:
        conn.close()

def remove_term(self, source):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM glossary WHERE source_term = ?", (source.strip(),))
    cursor.execute("DELETE FROM collection_terms WHERE term_source = ?", (source.strip(),))
    conn.commit()
    conn.close()

def get_all_terms(self):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT source_term, target_translation, description, source_lang, target_lang FROM glossary")
    rows = cursor.fetchall()
    conn.close()
    return [{"source": r[0], "target": r[1], "description": r[2], "source_lang": r[3] or "", "target_lang": r[4] or ""} for r in rows]

def _tokenize(self, text):
    return re.findall(r"\w+(?:'\w+)?", text.lower())

def peek_tmx(self, file_path):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"TMX file not found: {file_path}")
    tree = ET.parse(file_path)
    root = tree.getroot()
    XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"

    header = root.find("header")
    srclang = header.get("srclang", "") if header is not None else ""

    body = root.find("body")
    if body is None:
        return {"source_lang": srclang, "target_lang": "", "count": 0}

    all_langs = set()
    seen_langs = set()
    tuv_count = 0
    for tu in body.findall("tu"):
        for tuv in tu.findall("tuv"):
            lang = tuv.get(XML_LANG) or tuv.get("lang") or ""
            seg = tuv.find("seg")
            seg_text = seg.text.strip() if seg is not None and seg.text else ""
            if seg_text:
                seen_langs.add(lang)
                tuv_count += 1

    total_pairs = tuv_count // 2
    target_langs = seen_langs - {srclang} if srclang else seen_langs
    target_lang = next(iter(target_langs)) if len(target_langs) == 1 else ", ".join(sorted(target_langs)) if target_langs else ""
    return {"source_lang": srclang, "target_lang": target_lang, "count": total_pairs}

def import_tmx(self, file_path, source_lang=None):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"TMX file not found: {file_path}")

```

```

tree = ET.parse(file_path)
root = tree.getroot()
XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"

header = root.find("header")
if header is not None and source_lang is None:
    source_lang = header.get("srclang", "en")
if source_lang is None:
    source_lang = "en"

body = root.find("body")
if body is None:
    return 0

count = 0
for tu in body.findall("tu"):
    tuvs = tu.findall("tuv")
    source_text = None
    target_text = None
    target_lang = ""
    for tuv in tuvs:
        lang = tuv.get(XML_LANG) or tuv.get("lang") or ""
        seg = tuv.find("seg")
        seg_text = seg.text.strip() if seg is not None and seg.text else ""
        if not seg_text:
            continue
        if lang == source_lang:
            source_text = seg_text
        elif target_text is None:
            target_text = seg_text
            target_lang = lang
    if source_text and target_text:
        self.add_term(source_text, target_text,
                      f"Imported from TMX: {os.path.basename(file_path)}",
                      source_lang=source_lang, target_lang=target_lang)
        count += 1
    return count

def export_tmj(self, file_path, source_lang=None, target_lang=None):
    terms = self.get_all_terms()
    if not terms:
        return 0

    if source_lang is None:
        source_lang = terms[0].get("source_lang") or "en"
    if target_lang is None:
        target_lang = terms[0].get("target_lang") or ""

    XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"

    root = ET.Element("tmj", version="1.4")
    header = ET.SubElement(root, "header")
    header.set("srclang", source_lang)
    header.set("datatype", "plaintext")
    body = ET.SubElement(root, "body")

    for t in terms:
        tu = ET.SubElement(body, "tu")
        note = ET.SubElement(tu, "note")
        note.text = t.get("description", "")

        tuv_src = ET.SubElement(tu, "tuv")
        tuv_src.set(XML_LANG, t.get("source_lang") or source_lang)
        seg_src = ET.SubElement(tuv_src, "seg")
        seg_src.text = t["source"]

        t1 = t.get("target_lang") or target_lang
        tuv_tgt = ET.SubElement(tu, "tuv")
        tuv_tgt.set(XML_LANG, t1 if t1 else source_lang)
        seg_tgt = ET.SubElement(tuv_tgt, "seg")
        seg_tgt.text = t["target"]

    tree = ET.ElementTree(root)
    tree.write(file_path, encoding="utf-8", xml_declaration=True)
    return len(terms)

def create_collection(self, name, description=""):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

```

```

try:
    cursor.execute("INSERT INTO collections (name, description) VALUES (?, ?)",
                    (name.strip(), description.strip()))
    conn.commit()
    return cursor.lastrowid
except sqlite3.IntegrityError:
    return None
finally:
    conn.close()

def delete_collection(self, collection_id):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM collection_terms WHERE collection_id = ?", (collection_id,))
    cursor.execute("DELETE FROM collections WHERE id = ?", (collection_id,))
    conn.commit()
    conn.close()

def get_all_collections(self):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT id, name, description, created_at FROM collections ORDER BY name")
    rows = cursor.fetchall()
    conn.close()
    return [{"id": r[0], "name": r[1], "description": r[2], "created_at": r[3]} for r in rows]

def add_term_to_collection(self, collection_id, term_source):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    try:
        cursor.execute("INSERT OR IGNORE INTO collection_terms (collection_id, term_source) VALUES (?, ?)",
                        (collection_id, term_source.strip()))
        conn.commit()
        return cursor.rowcount > 0
    finally:
        conn.close()

def remove_term_from_collection(self, collection_id, term_source):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM collection_terms WHERE collection_id = ? AND term_source = ?",
                    (collection_id, term_source.strip()))
    conn.commit()
    conn.close()

def get_collection_terms(self, collection_id):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT g.source_term, g.target_translation, g.description, g.source_lang, g.target_lang
        FROM glossary g
        JOIN collection_terms ct ON g.source_term = ct.term_source
        WHERE ct.collection_id = ?
        ORDER BY g.source_term
    """, (collection_id,))
    rows = cursor.fetchall()
    conn.close()
    return [{"source": r[0], "target": r[1], "description": r[2], "source_lang": r[3] or "", "target_lang": r[4] or ""} for r in rows]

def export_collection_tmx(self, collection_id, file_path):
    terms = self.get_collection_terms(collection_id)
    if not terms:
        return 0
    source_lang = terms[0].get("source_lang") or "en"
    target_lang = terms[0].get("target_lang") or ""
    XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"
    root = ET.Element("tmx", version="1.4")
    header = ET.SubElement(root, "header")
    header.set("srclang", source_lang)
    header.set("datatype", "plaintext")
    body = ET.SubElement(root, "body")
    for t in terms:
        tu = ET.SubElement(body, "tu")
        note = ET.SubElement(tu, "note")
        note.text = t.get("description", "")
        tuv_src = ET.SubElement(tu, "tuv")
        tuv_src.set(XML_LANG, t.get("source_lang") or source_lang)
        seg_src = ET.SubElement(tuv_src, "seg")
        seg_src.text = t["source"]

```

```

        tl = t.get("target_lang") or target_lang
        tuv_tgt = ET.SubElement(tu, "tuv")
        tuv_tgt.set(XML_LANG, tl if tl else source_lang)
        seg_tgt = ET.SubElement(tuv_tgt, "seg")
        seg_tgt.text = t["target"]
    tree = ET.ElementTree(root)
    tree.write(file_path, encoding="utf-8", xml_declaration=True)
    return len(terms)

def import_collection_tmx(self, file_path, name=None):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"TMX file not found: {file_path}")
    tree = ET.parse(file_path)
    root = tree.getroot()
    XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"
    header = root.find("header")
    srclang = header.get("srclang", "en") if header is not None else "en"
    body = root.find("body")
    if body is None:
        return 0, None
    coll_name = name or os.path.splitext(os.path.basename(file_path))[0]
    coll_id = self.create_collection(coll_name, f"Imported from {os.path.basename(file_path)}")
    if coll_id is None:
        return 0, None
    count = 0
    for tu in body.findall("tu"):
        tuvs = tu.findall("tuv")
        source_text = None
        target_text = None
        target_lang = ""
        for tuv in tuvs:
            lang = tuv.get(XML_LANG) or tuv.get("lang") or ""
            seg = tuv.find("seg")
            seg_text = seg.text.strip() if seg is not None and seg.text else ""
            if not seg_text:
                continue
            if lang == srclang:
                source_text = seg_text
            elif target_text is None:
                target_text = seg_text
                target_lang = lang
        if source_text and target_text:
            self.add_term(source_text, target_text,
                          f"Imported from TMX: {os.path.basename(file_path)}",
                          source_lang=srclang, target_lang=target_lang)
            self.add_term_to_collection(coll_id, source_text)
            count += 1
    return count, coll_id

def check_segment(self, text, source_lang=None, target_lang=None):
    if not text or not text.strip():
        return []

    tokens = self._tokenize(text)
    terms = self.get_all_terms()
    matched_terms = []

    for term in terms:
        if source_lang and term.get("source_lang") and term["source_lang"] != source_lang:
            continue
        if target_lang and term.get("target_lang") and term["target_lang"] != target_lang:
            continue

        src_lower = term["source"].lower()
        term_tokens = self._tokenize(src_lower)

        if not term_tokens:
            continue

        n_tokens = len(tokens)
        n_term = len(term_tokens)
        is_match = False
        for i in range(n_tokens - n_term + 1):
            if tokens[i:i+n_term] == term_tokens:
                is_match = True
                break

        if is_match:
            matched_terms.append(term)

```

```
return matched_terms
```

cat_toolsegmentation.py — 380 lines

- Class **AdvancedRulesDialog** (line 92)
- Function **_segrules_dir** (line 31)
- Function **load_rules** (line 37)
- Function **save_rules** (line 48)
- Function **_rule_summary** (line 70)
- Function **advanced_segmenter** (line 284)
- Function **_is_exception** (line 366)
- Function **__init__** (line 93)
- Function **_build_ui** (line 105)
- Function **_rule_summary** (line 185)
- Function **_refresh_list** (line 188)
- Function **_on_selection_changed** (line 197)
- Function **_clear_form** (line 218)
- Function **_save_rule** (line 229)
- Function **_start_add** (line 249)
- Function **_remove_selected** (line 253)
- Function **_apply** (line 268)
- Function **get_rules** (line 280)

```
import re
import json
import os
from PyQt5.QtWidgets import (
    QDialog, QVBoxLayout, QHBoxLayout, QListWidget, QListWidgetItem,
    QLineEdit, QPushButton, QWidget, QCheckBox, QComboBox, QLabel,
    QGroupBox, QSplitter, QMessageBox,
)
from PyQt5.QtCore import Qt

LANGUAGES = [
    "All", "English", "Spanish", "French", "German", "Italian",
    "Portuguese", "Dutch", "Russian", "Arabic", "Chinese", "Japanese",
    "Korean", "Turkish", "Polish", "Swedish", "Danish", "Norwegian",
    "Finnish", "Greek", "Hebrew", "Hindi", "Thai", "Vietnamese",
    "Czech", "Slovak", "Hungarian", "Romanian", "Bulgarian", "Ukrainian",
]

LANG_NAME_TO_CODE = {
    "All": "all", "English": "en", "Spanish": "es", "French": "fr",
    "German": "de", "Italian": "it", "Portuguese": "pt", "Dutch": "nl",
    "Russian": "ru", "Arabic": "ar", "Chinese": "zh", "Japanese": "ja",
    "Korean": "ko", "Turkish": "tr", "Polish": "pl", "Swedish": "sv",
    "Danish": "da", "Norwegian": "no", "Finnish": "fi", "Greek": "el",
    "Hebrew": "he", "Hindi": "hi", "Thai": "th", "Vietnamese": "vi",
    "Czech": "cs", "Slovak": "sk", "Hungarian": "hu", "Romanian": "ro",
    "Bulgarian": "bg", "Ukrainian": "uk",
}

def _segrules_dir():
    d = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat", "segrules")
    os.makedirs(d, exist_ok=True)
    return d

def load_rules(lang_code):
    path = os.path.join(_segrules_dir(), f"{lang_code}.json")
    if os.path.exists(path):
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except (json.JSONDecodeError, OSError):
            pass
    return None

def save_rules(rules, lang_code):
    path = os.path.join(_segrules_dir(), f"{lang_code}.json")
```

```

with open(path, "w", encoding="utf-8") as f:
    json.dump(rules, f, indent=2, ensure_ascii=False)

DEFAULT_RULES = [
    {"pattern": ".", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": "!", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": "?", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": ";", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": ":", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": "Mr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Mrs", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Dr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Ms", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Prof", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Sr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Jr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
]

def _rule_summary(rule):
    parts = [f"\"{rule['pattern']}\""]
    ba = rule.get("break_after", False)
    bb = rule.get("break_before", False)
    if ba:
        parts.append("break after")
    elif bb:
        parts.append("break before")
    else:
        parts.append("no break")
    flags = []
    if rule.get("case_sensitive"):
        flags.append("case")
    if rule.get("whole_word"):
        flags.append("whole word")
    if flags:
        parts.append(", ".join(flags))
    lang = rule.get("language", "All")
    parts.append(f"[{lang}]")
    return " \u2014 ".join(parts)

class AdvancedRulesDialog(QDialog):
    def __init__(self, parent=None, rules=None, lang_code=None):
        super().__init__(parent)
        self.lang_code = lang_code or "all"
        title = f"Segmentation Rules - {self.lang_code}.json"
        self.setWindowTitle(title)
        self.setMinimumWidth(720)
        self.setMinimumHeight(500)
        self.rules = rules if rules is not None else [dict(r) for r in DEFAULT_RULES]
        self._modified_index = None
        self._build_ui()
        self._refresh_list()

    def _build_ui(self):
        outer = QVBoxLayout(self)

        splitter = QSplitter(Qt.Horizontal)

        left_panel = QWidget()
        left_layout = QVBoxLayout(left_panel)
        left_layout.setContentsMargins(0, 0, 0, 0)

        left_layout.addWidget(QLabel("Rules:"))
        self.rule_list = QListWidget()
        self.rule_list.currentRowChanged.connect(self._on_selection_changed)
        left_layout.addWidget(self.rule_list)

        list_btn_row = QHBoxLayout()
        self.add_btn = QPushButton("Add")
        self.add_btn.clicked.connect(self._start_add)
        self.remove_btn = QPushButton("Remove")
        self.remove_btn.clicked.connect(self._remove_selected)
        self.remove_btn.setEnabled(False)
        list_btn_row.addWidget(self.add_btn)
        list_btn_row.addWidget(self.remove_btn)
        list_btn_row.addStretch()
        left_layout.addLayout(list_btn_row)

```

```

splitter.addWidget(left_panel)

right_panel = QWidget()
right_layout = QVBoxLayout(right_panel)
right_layout.setContentsMargins(0, 0, 0, 0)

detail_group = QGroupBox("Rule Details")
detail_layout = QVBoxLayout(detail_group)

detail_layout.addWidget(QLabel("Pattern (word, punctuation, or phrase):"))
self.pattern_input = QLineEdit()
self.pattern_input.setPlaceholderText("e.g. Mr, ., !, ...")
detail_layout.addWidget(self.pattern_input)

detail_layout.addWidget(QLabel("Language:"))
self.lang_combo = QComboBox()
self.lang_combo.addItems(LANGUAGES)
detail_layout.addWidget(self.lang_combo)

detail_layout.addWidget(QLabel("Options:"))
self.break_after_cb = QCheckBox("Break after this pattern")
self.break_after_cb.setChecked(True)
self.break_before_cb = QCheckBox("Break before this pattern")
self.case_cb = QCheckBox("Case sensitive")
self.wv_cb = QCheckBox("Whole word only")
detail_layout.addWidget(self.break_after_cb)
detail_layout.addWidget(self.break_before_cb)
detail_layout.addWidget(self.case_cb)
detail_layout.addWidget(self.wv_cb)

detail_btn_row = QHBoxLayout()
self.save_btn = QPushButton("Add Rule")
self.save_btn.clicked.connect(self._save_rule)
self.save_btn.setStyleSheet("background-color: #0078d4; color: white;")
self.clear_btn = QPushButton("Clear")
self.clear_btn.clicked.connect(self._clear_form)
detail_btn_row.addWidget(self.save_btn)
detail_btn_row.addWidget(self.clear_btn)
detail_btn_row.addStretch()
detail_layout.addLayout(detail_btn_row)

detail_layout.addStretch()
right_layout.addWidget(detail_group)

apply_row = QHBoxLayout()
apply_row.addStretch()
self.apply_btn = QPushButton("Apply Rules")
self.apply_btn.clicked.connect(self._apply)
apply_row.addWidget(self.apply_btn)
right_layout.addLayout(apply_row)

splitter.addWidget(right_panel)
splitter.setSizes([300, 420])
outer.addWidget(splitter)

def _rule_summary(self, rule):
    return _rule_summary(rule)

def _refresh_list(self):
    self.rule_list.blockSignals(True)
    self.rule_list.clear()
    for rule in self.rules:
        item = QListWidgetItem(self._rule_summary(rule))
        self.rule_list.addItem(item)
    self.rule_list.blockSignals(False)
    self.remove_btn.setEnabled(self.rule_list.count() > 0)

def _on_selection_changed(self, row):
    if row < 0 or row >= len(self.rules):
        self._clear_form()
        self.remove_btn.setEnabled(False)
        return
    self.remove_btn.setEnabled(True)
    rule = self.rules[row]
    self._modified_index = row
    self.pattern_input.setText(rule["pattern"])
    lang = rule.get("language", "All")
    idx = self.lang_combo.findText(lang)
    if idx >= 0:

```

```

        self.lang_combo.setCurrentIndex(idx)
    else:
        self.lang_combo.setCurrentIndex(0)
    self.break_after_cb.setChecked(rule.get("break_after", False))
    self.break_before_cb.setChecked(rule.get("break_before", False))
    self.case_cb.setChecked(rule.get("case_sensitive", False))
    self.wv_cb.setChecked(rule.get("whole_word", False))
    self.save_btn.setText("Update Rule")

def _clear_form(self):
    self._modified_index = None
    self.pattern_input.clear()
    self.lang_combo.setCurrentIndex(0)
    self.break_after_cb.setChecked(True)
    self.break_before_cb.setChecked(False)
    self.case_cb.setChecked(False)
    self.wv_cb.setChecked(False)
    self.save_btn.setText("Add Rule")
    self.rule_list.clearSelection()

def _save_rule(self):
    pattern = self.pattern_input.text().strip()
    if not pattern:
        QMessageBox.warning(self, "Missing Pattern", "Please enter a pattern.")
        return
    rule = {
        "pattern": pattern,
        "break_after": self.break_after_cb.isChecked(),
        "break_before": self.break_before_cb.isChecked(),
        "case_sensitive": self.case_cb.isChecked(),
        "whole_word": self.wv_cb.isChecked(),
        "language": self.lang_combo.currentText(),
    }
    if self._modified_index is not None and 0 <= self._modified_index < len(self.rules):
        self.rules[self._modified_index] = rule
    else:
        self.rules.append(rule)
    self._refresh_list()
    self._clear_form()

def _start_add(self):
    self._clear_form()
    self.pattern_input.setFocus()

def _remove_selected(self):
    row = self.rule_list.currentRow()
    if row < 0 or row >= len(self.rules):
        return
    reply = QMessageBox.question(
        self, "Remove Rule",
        f"Remove rule: {self._rule_summary(self.rules[row])}?",
        QMessageBox.Yes | QMessageBox.No, QMessageBox.No,
    )
    if reply != QMessageBox.Yes:
        return
    self.rules.pop(row)
    self._refresh_list()
    self._clear_form()

def _apply(self):
    if self.pattern_input.text().strip():
        self._save_rule()
    by_lang = {}
    for rule in self.rules:
        lang_name = rule.get("language", "All")
        code = LANG_NAME_TO_CODE.get(lang_name, "all")
        by_lang.setdefault(code, []).append(rule)
    for code, lang_rules in by_lang.items():
        save_rules(lang_rules, code)
    self.accept()

def get_rules(self):
    return self.rules

def advanced_segmenter(text, custom_rules=None, language=None, lang_code=None):
    if not text:
        return []
    rules = custom_rules

```

```

if rules is None and lang_code:
    lang_rules = []
    loaded = load_rules(lang_code)
    if loaded:
        lang_rules.extend(loaded)
    loaded_all = load_rules("all")
    if loaded_all:
        lang_rules.extend(loaded_all)
    if lang_rules:
        rules = lang_rules
if rules is None:
    rules = DEFAULT_RULES

if language and language != "All":
    rules = [r for r in rules if r.get("language", "All") in ("All", language)]
    if not rules:
        return text.splitlines()

after_rules = [r for r in rules if r.get("break_after", False)]
before_rules = [r for r in rules if r.get("break_before", False)]
exceptions = [r for r in rules if not r.get("break_after", False) and not r.get("break_before", False)]

try:
    positions = []

    # --- break-after: single-char splitters ---
    after_chars = set()
    for r in after_rules:
        if len(r["pattern"]) == 1 and not r.get("whole_word", False):
            after_chars.add(r["pattern"])

    if after_chars:
        cc = "".join(re.escape(c) for c in sorted(after_chars))
        for m in re.finditer(rf"([{cc}])\s+", text):
            if not _is_exception(text[:m.start()].rstrip(), exceptions):
                positions.append((m.start(), m.end()))

    # --- break-before: single-char splitters ---
    before_chars = set()
    for r in before_rules:
        if len(r["pattern"]) == 1 and not r.get("whole_word", False):
            before_chars.add(r["pattern"])

    if before_chars:
        cc = "".join(re.escape(c) for c in sorted(before_chars))
        for m in re.finditer(rf"(?<=\s)[{cc}]", text):
            if not _is_exception(text[:m.start()].rstrip(), exceptions):
                positions.append((m.start(), m.start()))

    positions.sort(key=lambda x: (x[0], x[1]))

    # Merge overlapping / adjacent positions
    merged = []
    for p in positions:
        if merged and p[0] < merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], p[1]))
        else:
            merged.append(p)
    positions = merged

    if not positions:
        return [text]

    segments = []
    prev = 0
    for start, end in positions:
        if start > prev:
            segments.append(text[prev:start].strip())
        prev = end
    segments.append(text[prev:].strip())

    return [s for s in segments if s]

except re.error:
    return text.splitlines()

def _is_exception(prefix, exceptions):
    for r in exceptions:

```

```

pat = r["pattern"]
flags = 0 if r.get("case_sensitive", False) else re.IGNORECASE
ww = r.get("whole_word", False)
try:
    if ww:
        if re.search(rf"\b{re.escape(pat)}\b$", prefix, flags):
            return True
    else:
        if re.search(rf"{re.escape(pat)}$", prefix, flags):
            return True
except re.error:
    continue
return False

```

cat_toolsuggest.py — 138 lines

- Class **CompletionDelegate** (line 31)
- Function **__init__** (line 32)
- Function **set_source_texts** (line 42)
- Function **createEditor** (line 45)
- Function **setModelData** (line 136)
- Function **update_model** (line 73)
- Function **on_text_changed** (line 119)
- Function **on_typing** (line 126)

```

from PyQt5.QtWidgets import QStyledItemDelegate, QLineEdit, QCompleter, QListView
from PyQt5.QtCore import QStringListModel
from PyQt5.QtCore import Qt, QTimer

```

```

VSCODE_POPUP = """
QListView {
    background-color: #252526;
    border: 1px solid #3c3c3c;
    outline: none;
    padding: 2px;
    font-family: "Segoe UI", Consolas, "Courier New", monospace;
    font-size: 12px;
    color: #d4d4d4;
}
QListView::item {
    padding: 4px 8px;
    min-height: 20px;
    border: none;
}
QListView::item:selected {
    background-color: #094771;
    color: white;
}
QListView::item:hover {
    background-color: #2a2d2e;
}
"""

```

```

class CompletionDelegate(QStyledItemDelegate):
    def __init__(self, parent, tm, glossary, spellcheck=None, settings=None, source_lang=None, target_lang=None):
        super().__init__(parent)
        self.tm = tm
        self.glossary = glossary
        self.spellcheck = spellcheck
        self.settings = settings
        self.source_lang = source_lang
        self.target_lang = target_lang
        self._source_texts = []

    def set_source_texts(self, texts):
        self._source_texts = texts

    def createEditor(self, parent, option, index):
        editor = QLineEdit(parent)
        editor.setStyleSheet("font-size: 14px; padding: 2px;")
        completer = QCompleter()
        completer.setCaseSensitivity(Qt.CaseInsensitive)

```

```

completer.setFilterMode(Qt.MatchContains)

popup = QListView()
popup.setStyleSheet(VSCODE_POPUP)
popup.setWindowFlags(Qt.ToolTip)
popup.setFocusPolicy(Qt.StrongFocus)
completer.setPopup(popup)
completer.setMaxVisibleItems(15)
editor.setCompleter(completer)

source_text = ""
row = index.row()
if row < len(self._source_texts):
    source_text = self._source_texts[row]

min_score = 40.0
if self.settings:
    min_score = self.settings.get("fuzzy_match_min_score", 40.0)

debounce = QTimer(editor)
debounce.setSingleShot(True)
debounce.setInterval(180)

def update_model(current_text=None):
    all_items = []
    seen_targets = set()

    if source_text:
        tm_matches = self.tm.get_fuzzy_matches(source_text, min_score=min_score, limit=15, source_lang=self.source_lang, target_lang=self.target_lang)
        for m in tm_matches:
            target = m.get("target", "")
            score = m.get("score", 0)
            src = m.get("source", "")
            if not current_text or current_text.lower() in target.lower():
                if target.lower() not in seen_targets:
                    seen_targets.add(target.lower())
                    label = f"{int(score):>2}% {target}"
                    all_items.append((score, 0, label, target))

    if self.glossary and source_text:
        gl_terms = self.glossary.check_segment(source_text, source_lang=self.source_lang, target_lang=self.target_lang) if hasattr(self, 'glossary') else []
        for term in gl_terms:
            target = term.get("target", "")
            src = term.get("source", "")
            if not current_text or current_text.lower() in target.lower():
                if target.lower() not in seen_targets:
                    seen_targets.add(target.lower())
                    label = f"GLOS {target} ({src})"
                    all_items.append((100, 1, label, target))

    if current_text and self.spellcheck and hasattr(self.spellcheck, 'suggest'):
        last_word = current_text.strip().split()[-1] if current_text.strip() else ""
        if last_word and len(last_word) > 2:
            sp_suggestions = self.spellcheck.suggest(last_word)
            for s in sp_suggestions[:5]:
                replacement = current_text.rstrip(last_word) + s
                if replacement.lower() not in seen_targets:
                    seen_targets.add(replacement.lower())
                    label = f"SPELL {replacement}"
                    all_items.append((90, 2, label, replacement))

    all_items.sort(key=lambda x: (-x[0], x[1]))
    suggestions = [item[2] for item in all_items[:30]]

    model = QStringListModel(suggestions)
    completer.setModel(model)
    if suggestions:
        completer.complete()

def on_text_changed(text):
    debounce.stop()
    if text:
        debounce.start()
    else:
        update_model(text)

def on_typing():
    update_model(editor.text())
debounce.timeout.connect(on_typing)

```

```

        editor.textChanged.connect(on_text_changed)

    QTimer.singleShot(50, lambda: update_model())

    return editor

def setModelData(self, editor, model, index):
    text = editor.text()
    model.setData(index, text, Qt.EditRole)

```

cat_toolspellcheck.py — 135 lines

- Class **SpellChecker** (line 10)
- Function **__init__** (line 11)
- Function **auto_load** (line 16)
- Function **_lang_id_from_archive** (line 34)
- Function **load_from_archive** (line 47)
- Function **load_dictionary** (line 70)
- Function **_load_from_dir** (line 85)
- Function **has_dict** (line 101)
- Function **loaded_names** (line 105)
- Function **check** (line 108)
- Function **suggest** (line 116)
- Function **check_text** (line 123)

```

import os
import re
import shutil
import zipfile
from spylls.hunspell import Dictionary

DICTS_DIR = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat", "dicts")

class SpellChecker:
    def __init__(self):
        self.dictionaries = []
        self._loaded_names = []
        os.makedirs(DICTS_DIR, exist_ok=True)

    def auto_load(self):
        for name in os.listdir(DICTS_DIR):
            dir_path = os.path.join(DICTS_DIR, name)
            if not os.path.isdir(dir_path):
                continue
            for f in os.listdir(dir_path):
                if f.endswith(".dic"):
                    base = os.path.join(dir_path, os.path.splitext(f)[0])
                    aff = base + ".aff"
                    if os.path.exists(aff):
                        try:
                            d = Dictionary.from_files(base)
                            self.dictionaries.append(d)
                            self._loaded_names.append(name)
                        except Exception:
                            pass
            break

    def _lang_id_from_archive(self, archive_path):
        candidates = []
        with zipfile.ZipFile(archive_path) as z:
            for name in z.namelist():
                if name.endswith(".dic"):
                    base = os.path.splitext(os.path.basename(name))[0]
                    if "_" in base or "-" in base:
                        candidates.append(base)
        if not candidates:
            base = os.path.splitext(os.path.basename(archive_path))[0]
            return base.replace(" ", "_").lower()
        return candidates[0]

    def load_from_archive(self, archive_path):

```

```

try:
    lang_id = self._lang_id_from_archive(archive_path)
except Exception:
    return False
target_dir = os.path.join(DICTS_DIR, lang_id)
os.makedirs(target_dir, exist_ok=True)

extracted = False
with zipfile.ZipFile(archive_path) as z:
    for name in z.namelist():
        if name.endswith(".dic") or name.endswith(".aff"):
            dest = os.path.join(target_dir, os.path.basename(name))
            if not os.path.exists(dest):
                with z.open(name) as src, open(dest, "wb") as dst:
                    shutil.copyfileobj(src, dst)
            extracted = True

if not extracted:
    return False

return self._load_from_dir(target_dir, lang_id)

def load_dictionary(self, dic_path, aff_path=None):
    if aff_path is None:
        aff_path = os.path.splitext(dic_path)[0] + ".aff"
    if not os.path.exists(aff_path):
        return False

    base_name = os.path.splitext(os.path.basename(dic_path))[0]
    target_dir = os.path.join(DICTS_DIR, base_name)
    os.makedirs(target_dir, exist_ok=True)

    shutil.copy2(dic_path, os.path.join(target_dir, os.path.basename(dic_path)))
    shutil.copy2(aff_path, os.path.join(target_dir, os.path.basename(aff_path)))

    return self._load_from_dir(target_dir, base_name)

def _load_from_dir(self, dir_path, name):
    if name in self._loaded_names:
        return True
    for f in os.listdir(dir_path):
        if f.endswith(".dic"):
            base = os.path.join(dir_path, os.path.splitext(f)[0])
            try:
                d = Dictionary.from_files(base)
                self.dictionaries.append(d)
                self._loaded_names.append(name)
                return True
            except Exception:
                return False
    return False

@property
def has_dict(self):
    return len(self.dictionaries) > 0

@property
def loaded_names(self):
    return list(self._loaded_names)

def check(self, word):
    if not self.dictionaries:
        return True
    for d in self.dictionaries:
        if d.lookup(word):
            return True
    return False

def suggest(self, word):
    suggestions = set()
    for d in self.dictionaries:
        for s in d.suggest(word):
            suggestions.add(s)
    return sorted(suggestions)

def check_text(self, text):
    if not text:
        return []
    words = re.findall(r"\b[^\W\d_]+\b", text, re.UNICODE)

```

```

errors = []
seen = set()
for word in words:
    if word.lower() in seen:
        continue
    if not self.check(word):
        seen.add(word.lower())
        errors.append((word, self.suggest(word)))
return errors

```

cat_toolqa.py — 179 lines

- Class **QAIssue** (line 5)
- Function **_extract_numbers** (line 17)
- Function **_count_tags** (line 21)
- Function **check_missing_translations** (line 28)
- Function **check_unedited_fuzzy** (line 41)
- Function **check_inconsistent_translations** (line 53)
- Function **check_tag_mismatch** (line 74)
- Function **check_number_mismatch** (line 90)
- Function **check_whitespace** (line 113)
- Function **check_terminology** (line 147)
- Function **run_all_checks** (line 169)
- Function **__init__** (line 8)

```

import re
from collections import defaultdict

```

```

class QAIssue:
    __slots__ = ("severity", "check_name", "segment_index", "source", "target", "description")

```

```

    def __init__(self, severity, check_name, segment_index, source, target, description):
        self.severity = severity
        self.check_name = check_name
        self.segment_index = segment_index
        self.source = source
        self.target = target
        self.description = description

```

```

def _extract_numbers(text):
    return set(re.findall(r"\d+(?:\.\d+)?", text))

```

```

def _count_tags(text):
    if not text:
        return 0
    from cat_tool.formats.tag_utils import PH_L, PH_R
    return text.count(PH_L)

```

```

def check_missing_translations(segments):
    issues = []
    for idx, seg in enumerate(segments):
        target = seg.get("target", "") or ""
        if not target.strip():
            issues.append(QAIssue(
                "error", "Missing translation", idx,
                seg.get("source", ""), "",
                "Segment has no translation."
            ))
    return issues

```

```

def check_unedited_fuzzy(segments):
    issues = []
    for idx, seg in enumerate(segments):
        if seg.get("fuzzy", False):
            issues.append(QAIssue(
                "warning", "Unedited fuzzy match", idx,
                seg.get("source", ""), seg.get("target", ""),

```

```

        "Segment is still marked as fuzzy/needs review."
    ))
    return issues

def check_inconsistent_translations(segments):
    source_map = defaultdict(set)
    for seg in segments:
        src = seg.get("source", "").strip()
        tgt = seg.get("target", "").strip()
        if src and tgt:
            source_map[src].add(tgt)

    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "").strip()
        tgt = seg.get("target", "").strip()
        if src and tgt and len(source_map.get(src, set())) > 1:
            issues.append(QAIssue(
                "warning", "Inconsistent translation", idx,
                src, tgt,
                f"Source appears {len(source_map[src])} times with different translations: {' '.join(repr(t) for t in source_map[src])}"
            ))
    return issues

def check_tag_mismatch(segments):
    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        src_count = _count_tags(src)
        tgt_count = _count_tags(tgt)
        if src_count != tgt_count:
            issues.append(QAIssue(
                "error", "Tag mismatch", idx,
                src, tgt,
                f"Source has {src_count} tag(s), target has {tgt_count} tag(s)."
            ))
    return issues

def check_number_mismatch(segments):
    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        src_nums = _extract_numbers(src)
        tgt_nums = _extract_numbers(tgt)
        if src_nums and src_nums != tgt_nums:
            missing = src_nums - tgt_nums
            extra = tgt_nums - src_nums
            parts = []
            if missing:
                parts.append(f"missing: {' '.join(sorted(missing))}")
            if extra:
                parts.append(f"extra: {' '.join(sorted(extra))}")
            issues.append(QAIssue(
                "error", "Number mismatch", idx,
                src, tgt,
                "; ".join(parts)
            ))
    return issues

def check_whitespace(segments):
    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        if not tgt.strip():
            continue
        if src.startswith(" ") and not tgt.startswith(" "):
            issues.append(QAIssue(
                "warning", "Leading whitespace", idx,
                src, tgt,
                "Source starts with whitespace but target does not."
            ))
        elif not src.startswith(" ") and tgt.startswith(" "):

```

```

        issues.append(QAIssue(
            "warning", "Leading whitespace", idx,
            src, tgt,
            "Target starts with whitespace but source does not."
        ))
    if src.endswith(" ") and not tgt.endswith(" "):
        issues.append(QAIssue(
            "warning", "Trailing whitespace", idx,
            src, tgt,
            "Source ends with whitespace but target does not."
        ))
    elif not src.endswith(" ") and tgt.endswith(" "):
        issues.append(QAIssue(
            "warning", "Trailing whitespace", idx,
            src, tgt,
            "Target ends with whitespace but source does not."
        ))
    return issues

def check_terminology(segments, glossary):
    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        if not src.strip() or not tgt.strip():
            continue
        terms = glossary.check_segment(src)
        for term in terms:
            tgt_lower = tgt.lower()
            term_target = term.get("target", "").lower()
            if term_target and term_target not in tgt_lower:
                desc = term.get("description") or ""
                extra = f" ({desc})" if desc else ""
                issues.append(QAIssue(
                    "warning", "Terminology", idx,
                    src, tgt,
                    f"Glossary term '{term['source']}' should be '{term['target']}'{extra}."
                ))
    return issues

def run_all_checks(segments, glossary=None):
    issues = []
    issues.extend(check_missing_translations(segments))
    issues.extend(check_unedited_fuzzy(segments))
    issues.extend(check_inconsistent_translations(segments))
    issues.extend(check_tag_mismatch(segments))
    issues.extend(check_number_mismatch(segments))
    issues.extend(check_whitespace(segments))
    if glossary is not None:
        issues.extend(check_terminology(segments, glossary))
    return issues

```

cat_tool/settings.py — 46 lines

- Class **AppSettings** (line 15)
- Function **__init__** (line 16)
- Function **load** (line 19)
- Function **save** (line 28)
- Function **__getitem__** (line 33)
- Function **__setitem__** (line 36)
- Function **__contains__** (line 39)
- Function **get** (line 42)
- Function **as_dict** (line 45)

```

import json
import os

SETTINGS_FILE = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat", "settings.json")

DEFAULTS = {
    "source_font_size": 13,
    "target_font_size": 13,

```

```

        "auto_save_on_segment_change": False,
        "fuzzy_match_min_score": 40.0,
        "show_tag_placeholders": True,
    }

class AppSettings:
    def __init__(self):
        self._data = dict(DEFAULTS)

    def load(self):
        try:
            with open(SETTINGS_FILE, encoding="utf-8") as f:
                stored = json.load(f)
                self._data = {**DEFAULTS, **stored}
        except (FileNotFoundError, json.JSONDecodeError):
            self._data = dict(DEFAULTS)
        return self

    def save(self):
        os.makedirs(os.path.dirname(SETTINGS_FILE), exist_ok=True)
        with open(SETTINGS_FILE, "w", encoding="utf-8") as f:
            json.dump(self._data, f, indent=2)

    def __getitem__(self, key):
        return self._data[key]

    def __setitem__(self, key, value):
        self._data[key] = value

    def __contains__(self, key):
        return key in self._data

    def get(self, key, default=None):
        return self._data.get(key, default)

    def as_dict(self):
        return dict(self._data)

```

cat_toolproject.py — 217 lines

- Class **Project** (line 11)
- Function **__init__** (line 12)
- Function **default_dir** (line 22)
- Function **new** (line 27)
- Function **to_dict** (line 37)
- Function **from_dict** (line 49)
- Function **save** (line 60)
- Function **load** (line 71)
- Function **get_recent** (line 77)
- Function **add_recent** (line 88)
- Function **save_as_omegat** (line 100)
- Function **load_omegat** (line 126)
- Function **save_as_memoq** (line 157)
- Function **load_memoq** (line 184)
- Function **__str__** (line 216)

```

import json
import os
import shutil
import xml.etree.ElementTree as ET
from datetime import datetime

RECENT_FILE = "recent_projects.json"

class Project:
    def __init__(self):
        self.name = ""
        self.source_lang = ""

```

```

        self.target_lang = ""
        self.source_files = []
        self.file_path = None
        self.created_at = ""
        self.modified_at = ""

    @staticmethod
    def default_dir():
        d = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat")
        os.makedirs(d, exist_ok=True)
        return d

    def new(self, name, source_lang, target_lang, source_files):
        now = datetime.now().isoformat()
        self.name = name
        self.source_lang = source_lang
        self.target_lang = target_lang
        self.source_files = list(source_files)
        self.file_path = None
        self.created_at = now
        self.modified_at = now

    def to_dict(self):
        return {
            "version": 1,
            "name": self.name,
            "source_lang": self.source_lang,
            "target_lang": self.target_lang,
            "source_files": self.source_files,
            "created_at": self.created_at,
            "modified_at": self.modified_at,
        }

    @staticmethod
    def from_dict(data, file_path=None):
        p = Project()
        p.name = data.get("name", "")
        p.source_lang = data.get("source_lang", "")
        p.target_lang = data.get("target_lang", "")
        p.source_files = data.get("source_files", [])
        p.file_path = file_path
        p.created_at = data.get("created_at", "")
        p.modified_at = data.get("modified_at", "")
        return p

    def save(self, file_path=None):
        if file_path:
            self.file_path = file_path
        if not self.file_path:
            raise ValueError("No file path set for project save")
        self.modified_at = datetime.now().isoformat()
        with open(self.file_path, "w", encoding="utf-8") as f:
            json.dump(self.to_dict(), f, indent=2, ensure_ascii=False)
        return self.file_path

    @staticmethod
    def load(file_path):
        with open(file_path, "r", encoding="utf-8") as f:
            data = json.load(f)
        return Project.from_dict(data, file_path)

    @staticmethod
    def get_recent():
        path = os.path.join(Project.default_dir(), RECENT_FILE)
        if not os.path.exists(path):
            return []
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            return []

    @staticmethod
    def add_recent(file_path, name):
        recents = Project.get_recent()
        recents = [r for r in recents if r.get("path") != file_path]
        recents.insert(0, {"path": file_path, "name": name, "opened": datetime.now().isoformat()})
        recents = recents[:10]
        path = os.path.join(Project.default_dir(), RECENT_FILE)

```

```

try:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(recents, f, indent=2)
except Exception:
    pass

def save_as_omegat(self, dir_path):
    self.modified_at = datetime.now().isoformat()
    os.makedirs(dir_path, exist_ok=True)
    src_dir = os.path.join(dir_path, "source")
    tgt_dir = os.path.join(dir_path, "target")
    tm_dir = os.path.join(dir_path, "tm")
    glos_dir = os.path.join(dir_path, "glossary")
    for d in (src_dir, tgt_dir, tm_dir, glos_dir):
        os.makedirs(d, exist_ok=True)

    for f in self.source_files:
        if os.path.exists(f):
            shutil.copy2(f, os.path.join(src_dir, os.path.basename(f)))

    root = ET.Element("omegat")
    proj_el = ET.SubElement(root, "project")
    ET.SubElement(proj_el, "source_lang").text = self.source_lang.upper()
    ET.SubElement(proj_el, "target_lang").text = self.target_lang.upper()
    ET.SubElement(proj_el, "source_dir").text = "source"
    ET.SubElement(proj_el, "target_dir").text = "target"
    ET.SubElement(proj_el, "tm_dir").text = "tm"
    ET.SubElement(proj_el, "glossary_dir").text = "glossary"
    tree = ET.ElementTree(root)
    tree.write(os.path.join(dir_path, "omegat.project"), encoding="utf-8", xml_declaration=True)

@staticmethod
def load_omegat(dir_path):
    proj_file = os.path.join(dir_path, "omegat.project")
    if not os.path.exists(proj_file):
        raise FileNotFoundError(f"Not an OmegaT project: {dir_path}")
    tree = ET.parse(proj_file)
    root = tree.getroot()
    pe = root.find("project")
    if pe is None:
        raise ValueError("Invalid omegat.project: missing <project> element")

    p = Project()
    p.name = os.path.basename(dir_path)
    p.file_path = dir_path
    p.created_at = datetime.now().isoformat()
    p.modified_at = datetime.now().isoformat()

    raw_sl = (pe.findtext("source_lang") or "").strip()
    raw_tl = (pe.findtext("target_lang") or "").strip()
    p.source_lang = raw_sl.lower()
    p.target_lang = raw_tl.lower()

    source_dir_name = (pe.findtext("source_dir") or "source").strip()
    source_dir = os.path.join(dir_path, source_dir_name)
    p.source_files = []
    if os.path.isdir(source_dir):
        for entry in sorted(os.listdir(source_dir)):
            full = os.path.join(source_dir, entry)
            if os.path.isfile(full):
                p.source_files.append(full)
    return p

def save_as_memoq(self, dir_path, handler):
    self.modified_at = datetime.now().isoformat()
    os.makedirs(dir_path, exist_ok=True)
    src_dir = os.path.join(dir_path, "source")
    tgt_dir = os.path.join(dir_path, "target")
    os.makedirs(src_dir, exist_ok=True)
    os.makedirs(tgt_dir, exist_ok=True)

    for f in self.source_files:
        if os.path.exists(f):
            shutil.copy2(f, os.path.join(src_dir, os.path.basename(f)))

    manifest = ET.Element("memoQProject")
    ET.SubElement(manifest, "name").text = self.name
    ET.SubElement(manifest, "source_lang").text = self.source_lang
    ET.SubElement(manifest, "target_lang").text = self.target_lang

```

```

ET.SubElement(manifest, "created_at").text = self.created_at
ET.SubElement(manifest, "modified_at").text = self.modified_at
ET.SubElement(manifest, "tool").text = "mcat"
ET.indent(manifest)
tree = ET.ElementTree(manifest)
tree.write(os.path.join(dir_path, "manifest.xml"), encoding="utf-8", xml_declaration=True)

mqxliff_path = os.path.join(tgt_dir, f"{self.name}.mqxliff")
handler.save_as_mqxliif(mqxliff_path, self.source_lang, self.target_lang)

@staticmethod
def load_memoq(dir_path):
    manifest_file = os.path.join(dir_path, "manifest.xml")
    if not os.path.exists(manifest_file):
        raise FileNotFoundError(f"Not a memoQ project: {dir_path}")

    tree = ET.parse(manifest_file)
    root = tree.getroot()
    p = Project()
    p.name = (root.findtext("name") or "").strip()
    p.file_path = dir_path
    p.source_lang = (root.findtext("source_lang") or "en").strip()
    p.target_lang = (root.findtext("target_lang") or "es").strip()
    p.created_at = root.findtext("created_at") or ""
    p.modified_at = root.findtext("modified_at") or ""

    src_dir = os.path.join(dir_path, "source")
    p.source_files = []
    if os.path.isdir(src_dir):
        for entry in sorted(os.listdir(src_dir)):
            full = os.path.join(src_dir, entry)
            if os.path.isfile(full):
                p.source_files.append(full)

    tgt_dir = os.path.join(dir_path, "target")
    p.mqxliff_files = []
    if os.path.isdir(tgt_dir):
        for entry in sorted(os.listdir(tgt_dir)):
            if entry.endswith(".mqxliff"):
                p.mqxliff_files.append(os.path.join(tgt_dir, entry))

    return p

def __str__(self):
    return f"Project({self.name}, {self.source_lang}-&gt;{self.target_lang}, files={len(self.source_files)})"

```

cat_toolfile_handler.py — 113 lines

- Class **CATFileHandler** (line 13)
- Function **convert_to_po** (line 102)
- Function **__init__** (line 14)
- Function **load_file** (line 19)
- Function **_atomic_save** (line 46)
- Function **save_file** (line 57)
- Function **save_as_mqxliif** (line 85)
- Function **render_translated** (line 90)
- Function **do_save** (line 65)

```

import os

from cat_tool.formats import FORMAT_HANDLERS, FACTORY_EXTS
from cat_tool.formats.tag_utils import PH_L, PH_R, TAG_RE, extract_all_tags, apply_tags, restore_tags
from cat_tool.formats.factory import FactoryHandler

SUPPORTED_MSG = (
    "Supported: MCAT.DB, PO, POT, XLIFF, TS, TMX, TBX, CSV, TXT, "
    "PDF, DOCX, PPTX, XLSX, XLS, ODT, ODS, ODP, MQXLIFF"
)

class CATFileHandler:
    def __init__(self):
        self.current_file_path = None
        self.segments = []

```

```

self._handler = None

def load_file(self, file_path):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File not found: {file_path}")

    ext = os.path.splitext(file_path)[1].lower()
    if file_path.lower().endswith(".mcat.db"):
        ext = ".mcat.db"
    self.current_file_path = file_path

    handler_cls = FORMAT_HANDLERS.get(ext)
    if handler_cls is not None:
        self._handler = handler_cls()
        self.segments = self._handler.load(file_path)
        return self.segments

    if ext in FACTORY_EXTS:
        self._handler = FactoryHandler()
        self.segments = self._handler.load(file_path)
        return self.segments

    try:
        self._handler = FactoryHandler()
        self.segments = self._handler.load(file_path)
        return self.segments
    except Exception:
        raise ValueError(f"Unsupported file format: {ext}\n{SUPPORTED_MSG}")

def _atomic_save(self, save_fn, target_path):
    dir_name = os.path.dirname(target_path) or "."
    tmp_path = os.path.join(dir_name, f".mcat_tmp_{os.getpid()}_{id(self)}")
    try:
        save_fn(tmp_path)
        os.replace(tmp_path, target_path)
    except BaseException:
        if os.path.exists(tmp_path):
            os.remove(tmp_path)
        raise

def save_file(self, target_path=None):
    if not self.segments:
        raise ValueError("No file loaded to save.")
    save_path = target_path or self.current_file_path
    ext = os.path.splitext(save_path)[1].lower()
    if save_path.lower().endswith(".mcat.db"):
        ext = ".mcat.db"

    def do_save(dest):
        if ext == ".mqxliff":
            from cat_tool.formats.mqxliff import MqxliffHandler
            MqxliffHandler().save(self.segments, dest)
        elif ext == ".sdlxliif":
            from cat_tool.formats.sdlxliif import SdlxliifHandler
            SdlxliifHandler().save(self.segments, dest)
        elif ext == ".mcat.db":
            from cat_tool.formats.mcatdb import McatDbHandler
            McatDbHandler().save(self.segments, dest)
        elif ext in FACTORY_EXTS:
            FactoryHandler().save(self.segments, dest)
        elif self._handler is not None and hasattr(self._handler, "save"):
            self._handler.save(self.segments, dest)
        else:
            raise ValueError(f"Cannot save to format: {ext}")

    self._atomic_save(do_save, save_path)
    self.current_file_path = save_path

def save_as_mqxliff(self, output_path, src_lang="en", tgt_lang="es"):
    from cat_tool.formats.mqxliff import MqxliffHandler
    MqxliffHandler().save(self.segments, output_path, src_lang, tgt_lang)
    self.current_file_path = output_path

def render_translated(self, output_path):
    if not self.segments:
        raise ValueError("No segments to render.")
    if not self.current_file_path:
        raise ValueError("No original file path; cannot render.")
    if self._handler is None:

```

```

        raise ValueError("No handler available for rendering.")
    if not hasattr(self._handler, "render"):
        raise ValueError(f"{type(self._handler).__name__} does not support rendering.")
    self._handler.render(self.current_file_path, self.segments, output_path)

def convert_to_po(source_path, output_path):
    from translate.storage.po import pofile
    handler = CATFileHandler()
    segments = handler.load_file(source_path)
    store = pofile()
    for seg in segments:
        unit = store.addsourceunit(seg["source"])
        unit.target = seg.get("target", "")
        if seg.get("fuzzy"):
            unit.markfuzzy()
    store.savefile(output_path)
    return output_path

```

cat_toolmcat_format_manager.py — 151 lines

- Class **MCatProjectManager** (line 43)
- Function **_connect** (line 46)
- Function **create_project_file** (line 55)
- Function **import_extracted_segments** (line 81)
- Function **get_editor_stream** (line 117)
- Function **update_segment_translation** (line 137)

```

import sqlite3
import os
from datetime import datetime, timezone

SCHEMA_SQL = """
CREATE TABLE IF NOT EXISTS project_meta (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS translation_units (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    file_id TEXT NOT NULL,
    source_text TEXT NOT NULL,
    target_text TEXT DEFAULT '',
    status TEXT DEFAULT 'Untranslated',
    match_score INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS format_tags (
    unit_id INTEGER NOT NULL,
    tag_index TEXT NOT NULL,
    raw_xml_tag TEXT NOT NULL,
    PRIMARY KEY (unit_id, tag_index),
    FOREIGN KEY(unit_id) REFERENCES translation_units(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS project_glossary (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    source_term TEXT NOT NULL,
    target_term TEXT NOT NULL,
    definition TEXT DEFAULT ''
);

PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA foreign_keys = ON;
"""

ENGINE_VERSION = "1.0.0"

class MCatProjectManager:

    @staticmethod
    def _connect(file_path):
        conn = sqlite3.connect(file_path)

```

```

conn.row_factory = sqlite3.Row
conn.execute("PRAGMA journal_mode = WAL")
conn.execute("PRAGMA synchronous = NORMAL")
conn.execute("PRAGMA foreign_keys = ON")
return conn

@staticmethod
def create_project_file(file_path, src_lang, tgt_lang):
    if not file_path.endswith(".mcat.db"):
        file_path += ".mcat.db"
    os.makedirs(os.path.dirname(os.path.abspath(file_path)), exist_ok=True)

    conn = MCatProjectManager._connect(file_path)
    try:
        conn.executescript(SCHEMA_SQL)
        now = datetime.now(timezone.utc).isoformat()
        meta = [
            ("source_language", src_lang),
            ("target_language", tgt_lang),
            ("created_at", now),
            ("engine_version", ENGINE_VERSION),
        ]
        conn.executemany(
            "INSERT OR REPLACE INTO project_meta (key, value) VALUES (?, ?)", meta
        )
        conn.commit()
    except BaseException:
        conn.rollback()
        raise
    finally:
        conn.close()

@staticmethod
def import_extracted_segments(file_path, segments_list):
    conn = MCatProjectManager._connect(file_path)
    try:
        with conn:
            for seg in segments_list:
                cursor = conn.execute(
                    """INSERT INTO translation_units
                    (file_id, source_text, target_text, status, match_score)
                    VALUES (?, ?, ?, ?, ?)""",
                    (
                        seg.get("file_id", ""),
                        seg.get("source", ""),
                        seg.get("target", ""),
                        seg.get("status", "Untranslated"),
                        seg.get("match_score", 0),
                    ),
                )
                unit_id = cursor.lastrowid
                tags = seg.get("all_tags", [])
                if tags:
                    tag_rows = [
                        (unit_id, tag.get("index", f"{{{i}}}" ), tag.get("raw", ""))
                        for i, tag in enumerate(tags)
                    ]
                    conn.executemany(
                        """INSERT OR IGNORE INTO format_tags
                        (unit_id, tag_index, raw_xml_tag) VALUES (?, ?, ?)""",
                        tag_rows,
                    )
    except BaseException:
        conn.rollback()
        raise
    finally:
        conn.close()

@staticmethod
def get_editor_stream(file_path, filter_status=None):
    conn = MCatProjectManager._connect(file_path)
    try:
        if filter_status:
            rows = conn.execute(
                "SELECT * FROM translation_units WHERE status = ? ORDER BY id",
                (filter_status,),
            ).fetchall()
        else:
            rows = conn.execute(

```

```

        "SELECT * FROM translation_units ORDER BY id"
    ).fetchall()
    return [dict(r) for r in rows]
except BaseException:
    conn.rollback()
    raise
finally:
    conn.close()

@staticmethod
def update_segment_translation(file_path, unit_id, target_text, status="Confirmed"):
    conn = MCatProjectManager._connect(file_path)
    try:
        conn.execute(
            """UPDATE translation_units
               SET target_text = ?, status = ?
               WHERE id = ?""",
            (target_text, status, unit_id),
        )
        conn.commit()
    except BaseException:
        conn.rollback()
        raise
    finally:
        conn.close()

```

4. Format Handlers

cat_toolformats__init__.py — 30 lines

```
from cat_tool.formats.tag_utils import PH_L, PH_R, TAG_RE, extract_all_tags, apply_tags, restore_tags
from cat_tool.formats.factory import FactoryHandler
from cat_tool.formats.mqxliif import MqxliifHandler
from cat_tool.formats.sdlxliif import SdlxliifHandler
from cat_tool.formats.text import TextHandler
from cat_tool.formats.pdf import PdfHandler
from cat_tool.formats.docx import DocxHandler
from cat_tool.formats.pptx import PptxHandler
from cat_tool.formats.xlsx import XlsxHandler
from cat_tool.formats.xls import XlsHandler
from cat_tool.formats.odf import OdfHandler
from cat_tool.formats.mcatdb import McatDbHandler

FORMAT_HANDLERS = {
    ".txt": TextHandler,
    ".pdf": PdfHandler,
    ".docx": DocxHandler,
    ".pptx": PptxHandler,
    ".ppsx": PptxHandler,
    ".xlsx": XlsxHandler,
    ".xls": XlsHandler,
    ".odt": OdfHandler,
    ".ods": OdfHandler,
    ".odp": OdfHandler,
    ".mqxliif": MqxliifHandler,
    ".sdlxliif": SdlxliifHandler,
    ".mcat.db": McatDbHandler,
}

FACTORY_EXTS = frozenset({"po", ".pot", ".mo", ".xlf", ".xliff", ".ts", ".tmx", ".tbx", ".csv"})
```

cat_toolformats\factory.py — 55 lines

- Class **FactoryHandler** (line 4)
- Function **load** (line 5)
- Function **save** (line 41)
- Function **render** (line 54)

```
from cat_tool.formats.tag_utils import extract_all_tags, apply_tags

class FactoryHandler:
    def load(self, file_path):
        from translate.storage.factory import getclass
        store_class = getclass(file_path)
        if not store_class:
            raise ValueError(f"Unsupported translation format: {file_path}")
        store = store_class.parsefile(file_path)
        segments = []
        for idx, unit in enumerate(store.units):
            if unit.isheader():
                continue
            locations = unit.getlocations()
            loc_str = ", ".join(locations) if locations else ""
            source_text = str(unit.source) if unit.source else ""
            target_text = str(unit.target) if unit.target else ""

            src_tags = extract_all_tags(source_text)
            tgt_tags = extract_all_tags(target_text)
            all_tags = list(src_tags)
            for t in tgt_tags:
                if t not in all_tags:
                    all_tags.append(t)
            source_clean = apply_tags(source_text, all_tags)
            target_clean = apply_tags(target_text, all_tags)

            segments.append({
                "index": idx, "source": source_text, "target": target_text,
                "source_clean": source_clean, "target_clean": target_clean,
```

```

        "all_tags": all_tags,
        "notes": unit.getnotes() or "",
        "fuzzy": unit.isfuzzy(), "translated": unit.istranslated(),
        "locations": loc_str,
        "source_location": {"unit_index": idx},
        "store": store, "unit": unit,
    })
    return segments

def save(self, segments, target_path):
    if not segments:
        raise ValueError("No segments to save.")
    store = segments[0].get("store")
    if store is None:
        raise ValueError("No store object available; cannot save factory format.")
    for seg in segments:
        unit = seg["unit"]
        unit.target = seg["target"]
        if hasattr(unit, "markfuzzy"):
            unit.markfuzzy(seg["fuzzy"])
    store.savefile(target_path)

def render(self, original_path, segments, output_path):
    self.save(segments, output_path)

```

cat_toolformats\text_base.py — 27 lines

- Function **make_segments_from_texts** (line 5)

```

from translate.storage.po import pofile
from cat_tool.formats.tag_utils import extract_all_tags, apply_tags

def make_segments_from_texts(texts, notes_prefix="", source_locations=None):
    store = pofile()
    segments = []
    for idx, text in enumerate(texts):
        if not text or not text.strip():
            continue
        all_tags = extract_all_tags(text)
        source_clean = apply_tags(text, all_tags)
        unit = store.addsourceunit(text)
        unit.source = text
        unit.target = ""
        loc = source_locations[idx] if source_locations and idx < len(source_locations) else {}
        segments.append({
            "index": idx, "source": text, "target": "",
            "source_clean": source_clean, "target_clean": "",
            "all_tags": all_tags,
            "notes": f"{notes_prefix}{idx + 1}" if notes_prefix else "",
            "fuzzy": False, "translated": False,
            "locations": f"{notes_prefix.strip().lower()}:{idx + 1}" if notes_prefix else "",
            "source_location": loc,
            "store": store, "unit": unit,
        })
    return segments

```

cat_toolformats>tag_utils.py — 34 lines

- Function **extract_all_tags** (line 8)
- Function **apply_tags** (line 17)
- Function **restore_tags** (line 27)
- Function **_repl** (line 21)
- Function **_repl** (line 31)

```

import re

TAG_RE = re.compile(r'<[^\>]+\>')
PH_L = '\u27e6'
PH_R = '\u27e7'

def extract_all_tags(text):
    seen = []

```

```

for m in TAG_RE.finditer(text or ""):
    t = m.group(0)
    if t not in seen:
        seen.append(t)
return seen

def apply_tags(text, tag_list):
    if not text or not tag_list:
        return text
    tag_to_idx = {t: i for i, t in enumerate(tag_list)}
    def _repl(m):
        idx = tag_to_idx.get(m.group(0))
        return f'{PH_L}{idx}{PH_R}' if idx is not None else m.group(0)
    return TAG_RE.sub(_repl, text)

def restore_tags(text, tag_list):
    if not text or not tag_list:
        return text
    pat = re.escape(PH_L) + r'(\d+)' + re.escape(PH_R)
    def _repl(m):
        idx = int(m.group(1))
        return tag_list[idx] if idx < len(tag_list) else m.group(0)
    return re.sub(pat, _repl, text)

```

cat_tool\formats\text.py — 16 lines

- Class **TextHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 11)

```

from cat_tool.formats._text_base import make_segments_from_texts

class TextHandler:
    def load(self, file_path):
        with open(file_path, "r", encoding="utf-8", errors="replace") as f:
            lines = [l.strip() for l in f.readlines()]
            locations = [{"line": i} for i in range(len(lines))]
            return make_segments_from_texts(lines, "Line ", source_locations=locations)

    def render(self, original_path, segments, output_path):
        translated = []
        for seg in segments:
            translated.append(seg.get("target", "").strip() or seg["source"])
        with open(output_path, "w", encoding="utf-8") as f:
            f.write("\n".join(translated))

```

cat_tool\formats\pdf.py — 77 lines

- Class **PdfHandler** (line 8)
- Function **load** (line 9)
- Function **render** (line 37)

```

from cat_tool.formats._text_base import make_segments_from_texts
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
from pypdf import PdfReader, PdfWriter
import io

class PdfHandler:
    def load(self, file_path):
        import pdfplumber
        texts = []
        locations = []
        with pdfplumber.open(file_path) as pdf:
            for page_num, page in enumerate(pdf.pages):
                chars = page.chars
                if not chars:
                    continue
                chars.sort(key=lambda c: (c["top"], c["x0"]))
                lines = []

```

```

        current_line = []
        current_top = chars[0]["top"]
        for c in chars:
            if abs(c["top"] - current_top) > max(c.get("height", 8) * 0.5, 3):
                if current_line:
                    lines.append("".join(current_line))
                    current_line = []
                    current_top = c["top"]
                current_line.append(c.get("text", ""))
            if current_line:
                lines.append("".join(current_line))
        block_text = " ".join(line.strip() for line in lines if line.strip())
        if block_text.strip():
            texts.append(block_text.strip())
            locations.append({"page": page_num, "block": len(locations)})
        return make_segments_from_texts(texts, "Page ", source_locations=locations)

def render(self, original_path, segments, output_path):
    reader = PdfReader(original_path)
    writer = PdfWriter()

    page_data = {}
    for seg in segments:
        loc = seg.get("source_location", {})
        if not loc:
            continue
        page_num = loc.get("page", 0)
        target = seg.get("target", "").strip()
        source = seg.get("source", "").strip()
        page_data.setdefault(page_num, []).append({
            "source": source,
            "target": target or source,
            "index": seg.get("index", 0),
        })

    num_pages = len(reader.pages)
    for page_num in range(num_pages):
        packet = io.BytesIO()
        w, h = letter
        c = canvas.Canvas(packet, pagesize=letter)
        c.setFont("Helvetica", 10)
        y = h - 50
        segs = page_data.get(page_num, [])
        if segs:
            for s in segs:
                c.drawString(50, y, f"{s['target']}")
                y -= 16
        else:
            c.drawString(50, y, "(no translated content)")
        c.save()
        packet.seek(0)
        overlay = PdfReader(packet)
        page = reader.pages[page_num]
        page.merge_page(overlay.pages[0])
        writer.add_page(page)

    with open(output_path, "wb") as f:
        writer.write(f)

```

cat_tool\formats\docx.py — 47 lines

- Class **DocxHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 24)

```

from cat_tool.formats._text_base import make_segments_from_texts

class DocxHandler:
    def load(self, file_path):
        from docx import Document
        doc = Document(file_path)
        texts = []
        locations = []
        for i, p in enumerate(doc.paragraphs):
            t = p.text.strip()

```

```

        if t:
            texts.append(t)
            locations.append({"type": "paragraph", "index": i})
    for ti, table in enumerate(doc.tables):
        for ri, row in enumerate(table.rows):
            for ci, cell in enumerate(row.cells):
                t = cell.text.strip()
                if t:
                    texts.append(t)
                    locations.append({"type": "table_cell", "table_index": ti, "row_index": ri, "col_index": ci})
    return make_segments_from_texts(texts, "Para ", source_locations=locations)

def render(self, original_path, segments, output_path):
    from docx import Document
    doc = Document(original_path)
    seg_iter = iter(segments)
    for p in doc.paragraphs:
        t = p.text.strip()
        if t:
            seg = next(seg_iter, None)
            if seg and seg.get("source_location", {}).get("type") == "paragraph":
                target = seg.get("target", "").strip()
                if target:
                    p.clear()
                    p.add_run(target)
    for ti, table in enumerate(doc.tables):
        for ri, row in enumerate(table.rows):
            for ci, cell in enumerate(row.cells):
                t = cell.text.strip()
                if t:
                    seg = next(seg_iter, None)
                    if seg and seg.get("source_location", {}).get("type") == "table_cell":
                        target = seg.get("target", "").strip()
                        if target:
                            cell.text = target
    doc.save(output_path)

```

cat_tool\formats\pptx.py — 43 lines

- Class **PptxHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 27)

```

from cat_tool.formats._text_base import make_segments_from_texts

class PptxHandler:
    def load(self, file_path):
        from pptx import Presentation
        prs = Presentation(file_path)
        texts = []
        locations = []
        for slide_num, slide in enumerate(prs.slides):
            for shape_idx, shape in enumerate(slide.shapes):
                if shape.has_text_frame:
                    for para_idx, para in enumerate(shape.text_frame.paragraphs):
                        t = para.text.strip()
                        if t:
                            texts.append(t)
                            locations.append({"slide": slide_num, "shape": shape_idx, "paragraph": para_idx})
                if shape.has_table:
                    for ri, row in enumerate(shape.table.rows):
                        for ci, cell in enumerate(row.cells):
                            t = cell.text.strip()
                            if t:
                                texts.append(t)
                                locations.append({"slide": slide_num, "shape": shape_idx, "table_row": ri, "table_col": ci})
        return make_segments_from_texts(texts, "Slide ", source_locations=locations)

    def render(self, original_path, segments, output_path):
        from pptx import Presentation
        prs = Presentation(original_path)
        for seg in segments:
            loc = seg.get("source_location", {})
            if not loc:
                continue

```

```

        target = seg.get("target", "").strip()
        if not target:
            continue
        slide = prs.slides[loc["slide"]]
        shape = slide.shapes[loc["shape"]]
        if "paragraph" in loc:
            shape.text_frame.paragraphs[loc["paragraph"]].text = target
        elif "table_row" in loc:
            shape.table.cell(loc["table_row"], loc["table_col"]).text = target
    prs.save(output_path)

```

cat_toolformats\xlsx.py — 33 lines

- Class **XlsxHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 21)

```

from cat_tool.formats._text_base import make_segments_from_texts

class XlsxHandler:
    def load(self, file_path):
        from openpyxl import load_workbook
        wb = load_workbook(file_path, read_only=True, data_only=True)
        texts = []
        locations = []
        for si, ws in enumerate(wb.worksheets):
            for ri, row in enumerate(ws.iter_rows(values_only=True)):
                for ci, cell in enumerate(row):
                    if cell is not None:
                        t = str(cell).strip()
                        if t:
                            texts.append(t)
                            locations.append({"sheet": si, "row": ri, "col": ci})
        wb.close()
        return make_segments_from_texts(texts, "Cell ", source_locations=locations)

    def render(self, original_path, segments, output_path):
        from openpyxl import load_workbook
        wb = load_workbook(original_path)
        for seg in segments:
            loc = seg.get("source_location", {})
            if not loc:
                continue
            target = seg.get("target", "").strip()
            if not target:
                continue
            ws = wb.worksheets[loc["sheet"]]
            ws.cell(row=loc["row"] + 1, column=loc["col"] + 1, value=target)
        wb.save(output_path)

```

cat_toolformats\xls.py — 39 lines

- Class **XlsHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 22)

```

from cat_tool.formats._text_base import make_segments_from_texts

class XlsHandler:
    def load(self, file_path):
        import xlrd
        wb = xlrd.open_workbook(file_path)
        texts = []
        locations = []
        for si in range(wb.nsheets):
            ws = wb.sheet_by_index(si)
            for ri in range(ws.nrows):
                for ci in range(ws.ncols):
                    cell = ws.cell(ri, ci)
                    if cell.ctype != xlrd.XL_CELL_EMPTY:
                        t = str(cell.value).strip()
                        if t:

```

```

        texts.append(t)
        locations.append({"sheet": si, "row": ri, "col": ci})
    return make_segments_from_texts(texts, "Cell ", source_locations=locations)

def render(self, original_path, segments, output_path):
    try:
        import xlrd
        from xlutils.copy import copy
    except ImportError:
        raise ImportError("xlutils is required to render .xls files. Install it with: pip install xlutils")
    rb = xlrd.open_workbook(original_path, formatting_info=True)
    wb = copy(rb)
    for seg in segments:
        loc = seg.get("source_location", {})
        if not loc:
            continue
        target = seg.get("target", "").strip()
        if not target:
            continue
        ws = wb.get_sheet(loc["sheet"])
        ws.write(loc["row"], loc["col"], target)
    wb.save(output_path)

```

cat_tool\formats\odf.py — 102 lines

- Class **OdfHandler** (line 4)
- Function **_is_in_table_cell** (line 6)
- Function **load** (line 15)
- Function **render** (line 52)

```

from cat_tool.formats._text_base import make_segments_from_texts

class OdfHandler:
    @staticmethod
    def _is_in_table_cell(element):
        from odf.table import TableCell
        parent = getattr(element, 'parent', None)
        while parent is not None:
            if isinstance(parent, TableCell):
                return True
            parent = getattr(parent, 'parent', None)
        return False

    def load(self, file_path):
        from odf.opendocument import load
        from odf.text import P, H
        from odf.table import Table, TableRow, TableCell
        from odf.element import Text as OdfText
        doc = load(file_path)
        texts = []
        locations = []
        para_counter = 0
        heading_counter = 0
        table_counter = 0

        for p in doc.getElementsByType(P):
            if self._is_in_table_cell(p):
                continue
            t = "".join(n.data for n in p.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
            if t:
                texts.append(t)
                locations.append({"type": "paragraph", "index": para_counter})
                para_counter += 1
        for h in doc.getElementsByType(H):
            t = "".join(n.data for n in h.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
            if t:
                texts.append(t)
                locations.append({"type": "heading", "index": heading_counter})
                heading_counter += 1
        for table in doc.getElementsByType(Table):
            for ri, row in enumerate(table.getElementsByType(TableRow)):
                for ci, cell in enumerate(row.getElementsByType(TableCell)):
                    t = "".join(n.data for n in cell.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
                    if t:
                        texts.append(t)

```

```

        locations.append({"type": "table_cell", "table_index": table_counter, "row_index": ri, "col_index": ci})
        table_counter += 1

    return make_segments_from_texts(texts, "ODF ", source_locations=locations)

def render(self, original_path, segments, output_path):
    from odf.opendocument import load
    from odf.text import P, H
    from odf.table import Table, TableRow, TableCell
    from odf.element import Text as OdfText
    from copy import deepcopy

    doc = load(original_path)
    seg_iter = iter(segments)

    for p in doc.getElementsByType(P):
        if self._is_in_table_cell(p):
            continue
        t = "".join(n.data for n in p.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
        if t:
            seg = next(seg_iter, None)
            if seg and seg.get("source_location", {}).get("type") == "paragraph":
                target = seg.get("target", "").strip()
                if target:
                    for child in list(p.childNodes):
                        if child.nodeType == OdfText.TEXT_NODE:
                            child.data = target
                            break

    for h in doc.getElementsByType(H):
        t = "".join(n.data for n in h.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
        if t:
            seg = next(seg_iter, None)
            if seg and seg.get("source_location", {}).get("type") == "heading":
                target = seg.get("target", "").strip()
                if target:
                    for child in list(h.childNodes):
                        if child.nodeType == OdfText.TEXT_NODE:
                            child.data = target
                            break

    for table in doc.getElementsByType(Table):
        for ri, row in enumerate(table.getElementsByType(TableRow)):
            for ci, cell in enumerate(row.getElementsByType(TableCell)):
                t = "".join(n.data for n in cell.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
                if t:
                    seg = next(seg_iter, None)
                    if seg and seg.get("source_location", {}).get("type") == "table_cell":
                        target = seg.get("target", "").strip()
                        if target:
                            for child in list(cell.childNodes):
                                if child.nodeType == OdfText.TEXT_NODE:
                                    child.data = target
                                    break

    doc.save(output_path)

```

cat_toolformats\mqxliff.py — 64 lines

- Class **MqxliffHandler** (line 6)
- Function **load** (line 7)
- Function **save** (line 39)

```

import os
import xml.etree.ElementTree as ET
from cat_tool.formats.tag_utils import extract_all_tags, apply_tags

```

```

class MqxliffHandler:
    def load(self, file_path):
        ns = "urn:oasis:names:tc:xliff:document:1.2"
        segments = []
        tree = ET.parse(file_path)
        root = tree.getroot()
        body = root.find(f".//{{{ns}}}body")
        if body is None:

```

```

        raise ValueError("Invalid MQXLIFF: no <body> element")
    for idx, unit_el in enumerate(body.findall(f"{{{ns}}}trans-unit")):
        src_el = unit_el.find(f"{{{ns}}}source")
        tgt_el = unit_el.find(f"{{{ns}}}target")
        src_text = src_el.text or "" if src_el is not None else ""
        tgt_text = tgt_el.text or "" if tgt_el is not None else ""

        src_tags = extract_all_tags(src_text)
        tgt_tags = extract_all_tags(tgt_text)
        all_tags = list(src_tags)
        for t in tgt_tags:
            if t not in all_tags:
                all_tags.append(t)
        source_clean = apply_tags(src_text, all_tags)
        target_clean = apply_tags(tgt_text, all_tags)

        segments.append({
            "index": idx, "source": src_text, "target": tgt_text,
            "source_clean": source_clean, "target_clean": target_clean,
            "all_tags": all_tags, "notes": "",
            "fuzzy": False, "translated": bool(tgt_text.strip()),
            "locations": "", "store": None, "unit": None,
        })
    return segments

def save(self, segments, output_path, src_lang="en", tgt_lang="es"):
    ns = "urn:oasis:names:tc:xliff:document:1.2"
    xsi_ns = "http://www.w3.org/2001/XMLSchema-instance"
    ET.register_namespace("", ns)
    ET.register_namespace("xsi", xsi_ns)

    root = ET.Element(f"{{{ns}}}xliff", {"version": "1.2"})
    root.set(f"{{{xsi_ns}}}schemaLocation", f"{{{ns}}}xliff-core-1.2-transitional.xsd")

    file_el = ET.SubElement(root, f"{{{ns}}}file", {
        "original": os.path.basename(output_path),
        "source-language": src_lang,
        "target-language": tgt_lang,
        "datatype": "x-memoq",
    })

    body = ET.SubElement(file_el, f"{{{ns}}}body")
    for idx, seg in enumerate(segments):
        unit_el = ET.SubElement(body, f"{{{ns}}}trans-unit", {"id": str(idx + 1)})
        src_el = ET.SubElement(unit_el, f"{{{ns}}}source")
        src_el.text = seg["source"]
        tgt_el = ET.SubElement(unit_el, f"{{{ns}}}target")
        tgt_el.text = seg["target"] if seg["target"] else ""

    tree = ET.ElementTree(root)
    tree.write(output_path, encoding="utf-8", xml_declaration=True)

```

cat_toolformats\sdlxliff.py — 111 lines

- Class **SdlxliffHandler** (line 10)
- Function **load** (line 11)
- Function **save** (line 69)

```

import os
import xml.etree.ElementTree as ET
from cat_toolformats.tag_utils import extract_all_tags, apply_tags

```

```

XLF_NS = "urn:oasis:names:tc:xliff:document:1.2"
SDL_NS = "http://sdl.com/FileTypes/SdlXliff/1.0"

```

```

class SdlxliffHandler:
    def load(self, file_path):
        segments = []
        tree = ET.parse(file_path)
        root = tree.getroot()

        for idx, unit_el in enumerate(root.findall(f".//{{{XLF_NS}}}trans-unit")):
            if unit_el.get("translate") == "no":
                continue

```

```

src_text = ""
tgt_text = ""

seg_source_el = unit_el.find(f"{{{XLF_NS}}}seg-source")
target_el = unit_el.find(f"{{{XLF_NS}}}target")
source_el = unit_el.find(f"{{{XLF_NS}}}source")

if seg_source_el is not None:
    src_parts = []
    for mrk in seg_source_el.findall(f"{{{XLF_NS}}}mrk"):
        txt = "".join(mrk.itertext()) if mrk.text else (mrk.text or "")
        src_parts.append(txt.strip())
    src_text = " ".join(p for p in src_parts if p)
elif source_el is not None:
    src_text = "".join(source_el.itertext()).strip()

if target_el is not None:
    mrks = target_el.findall(f"{{{XLF_NS}}}mrk")
    if mrks:
        tgt_parts = []
        for mrk in mrks:
            txt = "".join(mrk.itertext()) if mrk.text else (mrk.text or "")
            tgt_parts.append(txt.strip())
        tgt_text = " ".join(p for p in tgt_parts if p)
    else:
        tgt_text = "".join(target_el.itertext()).strip()

if not src_text:
    continue

src_tags = extract_all_tags(src_text)
tgt_tags = extract_all_tags(tgt_text)
all_tags = list(src_tags)
for t in tgt_tags:
    if t not in all_tags:
        all_tags.append(t)
source_clean = apply_tags(src_text, all_tags)
target_clean = apply_tags(tgt_text, all_tags)

segments.append({
    "index": idx, "source": src_text, "target": tgt_text,
    "source_clean": source_clean, "target_clean": target_clean,
    "all_tags": all_tags, "notes": "",
    "fuzzy": False, "translated": bool(tgt_text.strip()),
    "locations": "", "store": None, "unit": None,
})

return segments

def save(self, segments, output_path, src_lang="en", tgt_lang="es"):
    ET.register_namespace("", XLF_NS)
    ET.register_namespace("sdl", SDL_NS)
    ET.register_namespace("xsi", "http://www.w3.org/2001/XMLSchema-instance")

    root = ET.Element(f"{{{XLF_NS}}}xliff", {
        "version": "1.2",
        f"{{{SDL_NS}}}version": "1.0",
    })

    file_el = ET.SubElement(root, f"{{{XLF_NS}}}file", {
        "original": os.path.basename(output_path),
        "datatype": "x-sdlfilterframework2",
        "source-language": src_lang,
        "target-language": tgt_lang,
    })

    header = ET.SubElement(file_el, f"{{{XLF_NS}}}header")
    body = ET.SubElement(file_el, f"{{{XLF_NS}}}body")

    for idx, seg in enumerate(segments):
        unit_el = ET.SubElement(body, f"{{{XLF_NS}}}trans-unit", {"id": str(idx + 1)})

        src_el = ET.SubElement(unit_el, f"{{{XLF_NS}}}source")
        src_el.text = seg["source"]

        seg_src = ET.SubElement(unit_el, f"{{{XLF_NS}}}seg-source")
        mrk_src = ET.SubElement(seg_src, f"{{{XLF_NS}}}mrk", {"mtype": "seg", "mid": "1"})
        mrk_src.text = seg["source"]
        tgt_el = ET.SubElement(unit_el, f"{{{XLF_NS}}}target")

```

```

    if seg["target"]:
        mrk_tgt = ET.SubElement(tgt_el, f"{{{XLF_NS}}}mrk", {"mtype": "seg", "mid": "1"})
        mrk_tgt.text = seg["target"]

    seg_defs = ET.SubElement(unit_el, f"{{{SDL_NS}}}seg-defs")
    seg_def = ET.SubElement(seg_defs, f"{{{SDL_NS}}}seg", {
        "id": "1",
        "conf": "Translated" if seg["target"] else "Draft",
    })

tree = ET.ElementTree(root)
tree.write(output_path, encoding="utf-8", xml_declaration=True)

```

cat_tool\formats\mcatdb.py — 125 lines

- Class **McatDbHandler** (line 25)
- Function **_app_status_from_mcat** (line 9)
- Function **_mcat_status_from_app** (line 17)
- Function **load** (line 26)
- Function **save** (line 63)
- Function **render** (line 124)

```

import os
import shutil
import sqlite3
from datetime import datetime, timezone
from cat_tool.mcat_format_manager import MCatProjectManager, SCHEMA_SQL
from cat_tool.formats.tag_utils import extract_all_tags

```

```

def _app_status_from_mcat(status_text, has_target):
    if status_text == "Confirmed":
        return {"translated": True, "fuzzy": False}
    if status_text == "Draft":
        return {"translated": bool(has_target), "fuzzy": True}
    return {"translated": False, "fuzzy": False}

```

```

def _mcat_status_from_app(fuzzy, translated):
    if fuzzy:
        return "Draft"
    if translated:
        return "Confirmed"
    return "Untranslated"

```

```

class McatDbHandler:
    def load(self, file_path):
        rows = MCatProjectManager.get_editor_stream(file_path)
        segments = []
        for r in rows:
            source = r["source_text"]
            target = r["target_text"]
            src_tags = extract_all_tags(source)
            tgt_tags = extract_all_tags(target)
            all_tags = list(src_tags)
            for t in tgt_tags:
                if t not in all_tags:
                    all_tags.append(t)
            source_clean = source
            target_clean = target
            if all_tags:
                from cat_tool.formats.tag_utils import apply_tags
                source_clean = apply_tags(source, all_tags)
                target_clean = apply_tags(target, all_tags)

            status = _app_status_from_mcat(r["status"], target)
            segments.append({
                "index": r["id"],
                "source": source,
                "target": target,
                "source_clean": source_clean,
                "target_clean": target_clean,
                "all_tags": all_tags,
                "notes": "",
            })

```

```

        "fuzzy": status["fuzzy"],
        "translated": status["translated"],
        "locations": r["file_id"],
        "source_location": {"unit_id": r["id"]},
        "store": None,
        "unit": None,
    })
    return segments

def save(self, segments, target_path):
    if not segments:
        raise ValueError("No segments to save.")

    if not os.path.exists(target_path):
        tmp_conn = sqlite3.connect(target_path)
        try:
            tmp_conn.executescript(SCHEMA_SQL)
            now = datetime.now(timezone.utc).isoformat()
            tmp_conn.execute(
                "INSERT OR REPLACE INTO project_meta (key, value) VALUES (?, ?)",
                ("created_at", now),
            )
            tmp_conn.commit()
        except BaseException:
            tmp_conn.rollback()
            raise
        finally:
            tmp_conn.close()
        conn = None
    else:
        conn = None

    conn = MCatProjectManager._connect(target_path)
    try:
        conn.execute("DELETE FROM format_tags")
        conn.execute("DELETE FROM translation_units")
        for seg in segments:
            status = _mcat_status_from_app(seg.get("fuzzy", False),
                                           bool(seg.get("target", "")))

            cursor = conn.execute(
                """INSERT INTO translation_units
                (file_id, source_text, target_text, status, match_score)
                VALUES (?, ?, ?, ?, ?)""",
                (
                    seg.get("locations", ""),
                    seg.get("source", ""),
                    seg.get("target", ""),
                    status,
                    seg.get("match_score", 0),
                ),
            )
            unit_id = cursor.lastrowid
            tags = seg.get("all_tags", [])
            if tags:
                tag_rows = [
                    (unit_id, f"{{{i}}}", tag if isinstance(tag, str) else tag.get("raw", str(tag)))
                    for i, tag in enumerate(tags)
                ]
                conn.executemany(
                    """INSERT INTO format_tags (unit_id, tag_index, raw_xml_tag)
                    VALUES (?, ?, ?)""",
                    tag_rows,
                )
            conn.commit()
        except BaseException:
            conn.rollback()
            raise
        finally:
            conn.close()

def render(self, original_path, segments, output_path):
    shutil.copy2(original_path, output_path)

```

5. Tests

tests/test_cat_logic.py — 101 lines

- Class **TestCATTool** (line 8)
- Function **setUp** (line 9)
- Function **tearDown** (line 15)
- Function **test_translation_memory** (line 18)
- Function **test_glossary** (line 35)
- Function **test_file_handler** (line 58)

```
import unittest
import os
import tempfile
from cat_tool.tm import TranslationMemory
from cat_tool.glossary import Glossary
from cat_tool.file_handler import CATFileHandler

class TestCATTool(unittest.TestCase):
    def setUp(self):
        # Create temp files for databases and PO files
        self.temp_dir = tempfile.TemporaryDirectory()
        self.tm_db = os.path.join(self.temp_dir.name, "test_tm.db")
        self.glossary_db = os.path.join(self.temp_dir.name, "test_glossary.db")

    def tearDown(self):
        self.temp_dir.cleanup()

    def test_translation_memory(self):
        tm = TranslationMemory(self.tm_db)
        tm.add_translation("The quick brown fox jumps over the lazy dog.", "El zorro marrón rápido salta sobre el perro perezoso.")
        tm.add_translation("Hello world", "Hola mundo")

        # Exact match
        matches = tm.get_fuzzy_matches("Hello world")
        self.assertTrue(len(matches) > 0)
        self.assertEqual(matches[0]["target"], "Hola mundo")
        self.assertEqual(matches[0]["score"], 100.0)

        # Fuzzy match
        matches_fuzzy = tm.get_fuzzy_matches("Hello worlds")
        self.assertTrue(len(matches_fuzzy) > 0)
        self.assertGreater(matches_fuzzy[0]["score"], 80)
        self.assertEqual(matches_fuzzy[0]["target"], "Hola mundo")

    def test_glossary(self):
        gl = Glossary(self.glossary_db)
        gl.add_term("cat", "gato", "feline companion")
        gl.add_term("quick brown fox", "zorro marrón rápido", "a fast fox")

        terms = gl.get_all_terms()
        self.assertEqual(len(terms), 2)

        # Token match single word
        matches = gl.check_segment("My cat is sleeping.")
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches[0]["source"], "cat")
        self.assertEqual(matches[0]["target"], "gato")

        # Multi-word match
        matches_multi = gl.check_segment("The quick brown fox is fast.")
        self.assertEqual(len(matches_multi), 1)
        self.assertEqual(matches_multi[0]["source"], "quick brown fox")

        # Non-match due to boundaries
        matches_none = gl.check_segment("Concatenation is a concept.")
        self.assertEqual(len(matches_none), 0)

    def test_file_handler(self):
        # Create a basic mock PO file content
        po_content = """
msgid ""
msgstr ""
```

```

"Project-Id-Version: Test\\n"
"MIME-Version: 1.0\\n"
"Content-Type: text/plain; charset=UTF-8\\n"
"Content-Transfer-Encoding: 8bit\\n"

#. Location of segment 1
#: src/main.c:10
msgid "Hello World!"
msgstr ""

#. Location of segment 2
#: src/main.c:20
msgid "Goodbye World!"
msgstr "Adiós Mundo!"
"""

    po_file_path = os.path.join(self.temp_dir.name, "test.po")
    with open(po_file_path, "w", encoding="utf-8") as f:
        f.write(po_content.strip())

    handler = CATFileHandler()
    segments = handler.load_file(po_file_path)

    self.assertEqual(len(segments), 2)
    self.assertEqual(segments[0]["source"], "Hello World!")
    self.assertEqual(segments[0]["target"], "")
    self.assertEqual(segments[1]["source"], "Goodbye World!")
    self.assertEqual(segments[1]["target"], "Adiós Mundo!")

    # Modify first segment target
    segments[0]["target"] = "Hola Mundo!"
    handler.save_file()

    # Reload and check
    handler2 = CATFileHandler()
    segments2 = handler2.load_file(po_file_path)
    self.assertEqual(segments2[0]["target"], "Hola Mundo!")

if __name__ == "__main__":
    unittest.main()

```

tests/test_comprehensive.py — 610 lines

- Class **TestQAModule** (line 46)
- Class **TestTranslationMemory** (line 172)
- Class **TestGlossary** (line 217)
- Class **TestSegmentation** (line 280)
- Class **TestTagUtils** (line 318)
- Class **TestMakeSegmentsFromTexts** (line 369)
- Class **TestFileHandler** (line 393)
- Class **TestMcatProjectManager** (line 469)
- Class **TestTextHandler** (line 522)
- Class **TestMcatDbHandler** (line 547)
- Function **make_seg** (line 23)
- Function **setUp** (line 47)
- Function **test_missing_translations** (line 56)
- Function **test_missing_translations_none_target** (line 63)
- Function **test_unedited_fuzzy** (line 68)
- Function **test_inconsistent_translations** (line 73)
- Function **test_inconsistent_translations_no_dupes** (line 79)
- Function **test_tag_mismatch** (line 87)
- Function **test_number_mismatch** (line 97)
- Function **test_whitespace** (line 108)
- Function **test_terminology** (line 120)
- Function **test_terminology_missing_target** (line 136)
- Function **test_run_all_checks** (line 145)
- Function **test_run_all_checks_no_glossary** (line 153)
- Function **test_extract_numbers** (line 157)

- Function **test_empty_segments** (line 162)
- Function **test_qa_issue_slots** (line 166)
- Function **setUp** (line 173)
- Function **tearDown** (line 178)
- Function **test_add_and_fuzzy_match** (line 181)
- Function **test_fuzzy_match_partial** (line 188)
- Function **test_no_match** (line 194)
- Function **test_clear** (line 199)
- Function **test_add_empty_source** (line 205)
- Function **test_add_duplicate** (line 210)
- Function **setUp** (line 218)
- Function **tearDown** (line 223)
- Function **test_add_and_check** (line 226)
- Function **test_multi_word_term** (line 233)
- Function **test_word_boundary_no_match** (line 239)
- Function **test_case_insensitive_match** (line 244)
- Function **test_remove_term** (line 249)
- Function **test_get_all_terms** (line 255)
- Function **test_check_empty_text** (line 261)
- Function **test_add_empty_term** (line 267)
- Function **test_update_existing** (line 273)
- Function **test_basic_split** (line 281)
- Function **test_exception_no_split** (line 285)
- Function **test_multiple_exceptions** (line 289)
- Function **test_empty_text** (line 293)
- Function **test_no_splitters** (line 297)
- Function **test_custom_rules** (line 301)
- Function **test_semicolon_split** (line 309)
- Function **test_colon_split** (line 313)
- Function **test_extract_all_tags** (line 319)
- Function **test_extract_no_tags** (line 324)
- Function **test_extract_none_text** (line 328)
- Function **test_apply_and_restore** (line 332)
- Function **test_restore_no_tags** (line 341)
- Function **test_restore_none_text** (line 345)
- Function **test_apply_no_text** (line 349)
- Function **test_apply_no_taglist** (line 353)
- Function **test_unknown_tag_index** (line 357)
- Function **test_unique_tags** (line 364)
- Function **test_basic** (line 370)
- Function **test_empty_text_skipped** (line 377)
- Function **test_with_locations** (line 381)
- Function **test_tag_extraction** (line 387)
- Function **setUp** (line 394)
- Function **tearDown** (line 398)
- Function **_make_po** (line 401)
- Function **test_load_po** (line 407)
- Function **test_load_nonexistent** (line 416)
- Function **test_load_unsupported** (line 420)
- Function **test_save_po** (line 427)
- Function **test_save_no_segments** (line 440)
- Function **test_atomic_save** (line 444)
- Function **test_convert_to_po** (line 453)
- Function **test_render_no_segments** (line 464)
- Function **setUp** (line 470)

- Function **tearDown** (line 474)
- Function **test_create_project** (line 477)
- Function **test_create_project_auto_extension** (line 481)
- Function **test_import_and_get_stream** (line 486)
- Function **test_get_stream_filtered** (line 497)
- Function **test_update_segment** (line 508)
- Function **setUp** (line 523)
- Function **tearDown** (line 526)
- Function **test_load_and_render** (line 529)
- Function **setUp** (line 548)
- Function **tearDown** (line 553)
- Function **test_load** (line 556)
- Function **test_load_with_tags** (line 569)
- Function **test_save** (line 581)
- Function **test_save_empty_raises** (line 597)
- Function **test_render_copies** (line 602)

```
import unittest
import os
import tempfile
import json
import sqlite3

from cat_tool.qa import (
    QAIssue, run_all_checks,
    check_missing_translations, check_unedited_fuzzy,
    check_inconsistent_translations, check_tag_mismatch,
    check_number_mismatch, check_whitespace, check_terminology,
    _extract_numbers, _count_tags,
)

from cat_tool.tm import TranslationMemory
from cat_tool.glossary import Glossary
from cat_tool.segmentation import advanced_segmenter, DEFAULT_RULES
from cat_tool.formats.tag_utils import PH_L, PH_R, extract_all_tags, apply_tags, restore_tags
from cat_tool.file_handler import CATFileHandler, convert_to_po
from cat_tool.mcat_format_manager import MCatProjectManager, SCHEMA_SQL
from cat_tool.formats._text_base import make_segments_from_texts

def make_seg(index=0, source="Hello world", target="", fuzzy=False, translated=False, tags=None, source_location=None):
    seg = {
        "index": index,
        "source": source,
        "target": target,
        "source_clean": source,
        "target_clean": target,
        "all_tags": tags or [],
        "notes": "",
        "fuzzy": fuzzy,
        "translated": translated,
        "locations": "",
        "source_location": source_location or {},
        "store": None,
        "unit": None,
    }
    if tags:
        from cat_tool.formats.tag_utils import apply_tags
        seg["source_clean"] = apply_tags(source, tags)
        seg["target_clean"] = apply_tags(target, tags)
    return seg

class TestQAModule(unittest.TestCase):
    def setUp(self):
        self.segments = [
            make_seg(0, "Hello world", "Hola mundo", translated=True),
            make_seg(1, "Good morning", "", fuzzy=True),
            make_seg(2, "How are you?", "¿Cómo estás?", translated=True),
            make_seg(3, "Goodbye", ""),
            make_seg(4, "Hello world", "Hola mundo!", translated=True),
        ]
```

```

def test_missing_translations(self):
    issues = check_missing_translations(self.segments)
    self.assertEqual(len(issues), 2)
    indices = {i.segment_index for i in issues}
    self.assertIn(1, indices)
    self.assertIn(3, indices)

def test_missing_translations_none_target(self):
    segs = [make_seg(0, "test", None)]
    issues = check_missing_translations(segs)
    self.assertEqual(len(issues), 1)

def test_unedited_fuzzy(self):
    issues = check_unedited_fuzzy(self.segments)
    self.assertEqual(len(issues), 1)
    self.assertEqual(issues[0].segment_index, 1)

def test_inconsistent_translations(self):
    issues = check_inconsistent_translations(self.segments)
    self.assertEqual(len(issues), 2)
    self.assertEqual(issues[0].segment_index, 0)
    self.assertEqual(issues[1].segment_index, 4)

def test_inconsistent_translations_no_dupes(self):
    segs = [
        make_seg(0, "Hello", "Hola", translated=True),
        make_seg(1, "World", "Mundo", translated=True),
    ]
    issues = check_inconsistent_translations(segs)
    self.assertEqual(len(issues), 0)

def test_tag_mismatch(self):
    segs = [
        make_seg(0, f"Start {PH_L}{PH_R} end", f"Inicio {PH_L}{PH_R} fin", tags=["<b>"]),
        make_seg(1, f"Hello {PH_L}{PH_R} world", "Hola mundo", tags=["<b>"]),
        make_seg(2, "No tags", "Sin etiquetas"),
    ]
    issues = check_tag_mismatch(segs)
    self.assertEqual(len(issues), 1)
    self.assertEqual(issues[0].segment_index, 1)

def test_number_mismatch(self):
    segs = [
        make_seg(0, "Version 2.0 released", "Versión 2.0 lanzada", translated=True),
        make_seg(1, "File 1 of 3", "Archivo 1 de 3", translated=True),
        make_seg(2, "Cost: $50.00", "Costo: $99.00", translated=True),
        make_seg(3, "No numbers here", "Sin números"),
    ]
    issues = check_number_mismatch(segs)
    self.assertEqual(len(issues), 1)
    self.assertEqual(issues[0].segment_index, 2)

def test_whitespace(self):
    segs = [
        make_seg(0, "Leading spaces", "Leading spaces", translated=True),
        make_seg(1, "No leading space", "Has leading space", translated=True),
        make_seg(2, "Trailing space", "Trailing space", translated=True),
        make_seg(3, "No trailing", "Has trailing ", translated=True),
        make_seg(4, "Both sides ", "Both sides ", translated=True),
        make_seg(5, "Good", "", translated=False),
    ]
    issues = check_whitespace(segs)
    self.assertEqual(len(issues), 4)

def test_terminology(self):
    segs = [
        make_seg(0, "The cat is sleeping.", "El gato duerme.", translated=True),
        make_seg(1, "Quick brown fox.", "Zorro rápido.", translated=True),
        make_seg(2, "No terms here.", "Sin términos.", translated=True),
    ]
    tmp_dir = tempfile.TemporaryDirectory()
    gl = Glossary(os.path.join(tmp_dir.name, "test_gl.db"))
    gl.add_term("cat", "gato", "feline")
    gl.add_term("quick brown fox", "zorro marrón rápido", "fast fox")

    issues = check_terminology(segs, gl)
    self.assertEqual(len(issues), 1)
    self.assertEqual(issues[0].segment_index, 1)
    tmp_dir.cleanup()

```

```

def test_terminology_missing_target(self):
    segs = [make_seg(0, "The cat is here.", "", fuzzy=True)]
    tmp_dir = tempfile.TemporaryDirectory()
    gl = Glossary(os.path.join(tmp_dir.name, "test_gl.db"))
    gl.add_term("cat", "gato")
    issues = check_terminology(segs, gl)
    self.assertEqual(len(issues), 0)
    tmp_dir.cleanup()

def test_run_all_checks(self):
    tmp_dir = tempfile.TemporaryDirectory()
    gl = Glossary(os.path.join(tmp_dir.name, "test_gl.db"))
    gl.add_term("cat", "gato")
    issues = run_all_checks(self.segments, glossary=gl)
    self.assertGreater(len(issues), 0)
    tmp_dir.cleanup()

def test_run_all_checks_no_glossary(self):
    issues = run_all_checks(self.segments)
    self.assertGreater(len(issues), 2)

def test_extract_numbers(self):
    self.assertEqual(_extract_numbers("Version 2.0"), {"2.0"})
    self.assertEqual(_extract_numbers("File 1 of 3"), {"1", "3"})
    self.assertEqual(_extract_numbers("No numbers"), set())

def test_empty_segments(self):
    issues = run_all_checks([])
    self.assertEqual(len(issues), 0)

def test_qa_issue_slots(self):
    issue = QAIssue("error", "Test", 0, "src", "tgt", "desc")
    self.assertEqual(issue.severity, "error")
    self.assertEqual(issue.segment_index, 0)

class TestTranslationMemory(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "tm.db")
        self.tm = TranslationMemory(self.db_path)

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_add_and_fuzzy_match(self):
        self.tm.add_translation("Hello world", "Hola mundo")
        matches = self.tm.get_fuzzy_matches("Hello world")
        self.assertGreater(len(matches), 0)
        self.assertEqual(matches[0]["target"], "Hola mundo")
        self.assertEqual(matches[0]["score"], 100.0)

    def test_fuzzy_match_partial(self):
        self.tm.add_translation("The quick brown fox", "El zorro marrón rápido")
        matches = self.tm.get_fuzzy_matches("The quick brown fox jumps", min_score=50.0)
        self.assertGreater(len(matches), 0)
        self.assertGreater(matches[0]["score"], 50.0)

    def test_no_match(self):
        self.tm.add_translation("Hello", "Hola")
        matches = self.tm.get_fuzzy_matches("Completely different text", min_score=90.0)
        self.assertEqual(len(matches), 0)

    def test_clear(self):
        self.tm.add_translation("Hello", "Hola")
        self.tm.clear()
        matches = self.tm.get_fuzzy_matches("Hello")
        self.assertEqual(len(matches), 0)

    def test_add_empty_source(self):
        self.tm.add_translation("", "vacío")
        matches = self.tm.get_fuzzy_matches("")
        self.assertEqual(len(matches), 0)

    def test_add_duplicate(self):
        self.tm.add_translation("Hello", "Hola")
        self.tm.add_translation("Hello", ";Hola!")
        matches = self.tm.get_fuzzy_matches("Hello")
        self.assertEqual(matches[0]["target"], ";Hola!")

```

```

class TestGlossary(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "glossary.db")
        self.gl = Glossary(self.db_path)

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_add_and_check(self):
        self.gl.add_term("cat", "gato", "feline companion")
        matches = self.gl.check_segment("My cat is sleeping.")
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches[0]["source"], "cat")
        self.assertEqual(matches[0]["target"], "gato")

    def test_multi_word_term(self):
        self.gl.add_term("quick brown fox", "zorro marrón rápido")
        matches = self.gl.check_segment("The quick brown fox jumps.")
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches[0]["source"], "quick brown fox")

    def test_word_boundary_no_match(self):
        self.gl.add_term("cat", "gato")
        matches = self.gl.check_segment("Concatenation is a concept.")
        self.assertEqual(len(matches), 0)

    def test_case_insensitive_match(self):
        self.gl.add_term("Cat", "Gato")
        matches = self.gl.check_segment("My CAT is sleeping.")
        self.assertEqual(len(matches), 1)

    def test_remove_term(self):
        self.gl.add_term("dog", "perro")
        self.gl.remove_term("dog")
        matches = self.gl.check_segment("My dog runs.")
        self.assertEqual(len(matches), 0)

    def test_get_all_terms(self):
        self.gl.add_term("a", "1")
        self.gl.add_term("b", "2")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 2)

    def test_check_empty_text(self):
        matches = self.gl.check_segment("")
        self.assertEqual(len(matches), 0)
        matches = self.gl.check_segment(None)
        self.assertEqual(len(matches), 0)

    def test_add_empty_term(self):
        self.gl.add_term("", "test")
        self.gl.add_term("test", "")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 0)

    def test_update_existing(self):
        self.gl.add_term("cat", "gato")
        self.gl.add_term("cat", "michi")
        matches = self.gl.check_segment("My cat is here.")
        self.assertEqual(matches[0]["target"], "michi")

class TestSegmentation(unittest.TestCase):
    def test_basic_split(self):
        result = advanced_segmenter("Hello. How are you? I'm fine.")
        self.assertGreaterEqual(len(result), 2)

    def test_exception_no_split(self):
        result = advanced_segmenter("Dr. Smith went to the store.")
        self.assertEqual(len(result), 1)

    def test_multiple_exceptions(self):
        result = advanced_segmenter("Mr. and Mrs. Jones arrived. Dr. Brown came too.")
        self.assertEqual(len(result), 2)

    def test_empty_text(self):
        result = advanced_segmenter("")
        self.assertEqual(result, [])

```

```

def test_no_splitters(self):
    result = advanced_segmenter("Hello world no punctuation here")
    self.assertEqual(len(result), 1)

def test_custom_rules(self):
    rules = [
        {"pattern": ".", "break_after": True, "case_sensitive": False, "whole_word": False},
        {"pattern": "Inc", "break_after": False, "case_sensitive": True, "whole_word": True},
    ]
    result = advanced_segmenter("Hello Inc. World. Next sentence.", rules)
    self.assertEqual(len(result), 2)

def test_semicolon_split(self):
    result = advanced_segmenter("First part; second part.")
    self.assertEqual(len(result), 2)

def test_colon_split(self):
    result = advanced_segmenter("Note: important text here.")
    self.assertEqual(len(result), 2)

class TestTagUtils(unittest.TestCase):
    def test_extract_all_tags(self):
        tags = extract_all_tags("Hello <b>world</b>!")
        self.assertIn("<b>", tags)
        self.assertIn("</b>", tags)

    def test_extract_no_tags(self):
        tags = extract_all_tags("Hello world")
        self.assertEqual(tags, [])

    def test_extract_none_text(self):
        tags = extract_all_tags(None)
        self.assertEqual(tags, [])

    def test_apply_and_restore(self):
        tags = ["<b>", "</b>"]
        text = "<b>Hello</b>"
        applied = apply_tags(text, tags)
        self.assertIn(PH_L, applied)
        self.assertIn(PH_R, applied)
        restored = restore_tags(applied, tags)
        self.assertEqual(restored, text)

    def test_restore_no_tags(self):
        result = restore_tags("Hello world", [])
        self.assertEqual(result, "Hello world")

    def test_restore_none_text(self):
        result = restore_tags(None, ["<b>"])
        self.assertIsNone(result)

    def test_apply_no_text(self):
        result = apply_tags("", ["<b>"])
        self.assertEqual(result, "")

    def test_apply_no_taglist(self):
        result = apply_tags("Hello", None)
        self.assertEqual(result, "Hello")

    def test_unknown_tag_index(self):
        tags = ["<b>"]
        applied = apply_tags("<i>text</i>", tags)
        restored = restore_tags(applied, tags)
        self.assertIn("<i>", restored)
        self.assertIn("</i>", restored)

    def test_unique_tags(self):
        tags = extract_all_tags("<b><b><i></b></i></b></i>")
        self.assertEqual(len(tags), 4)

class TestMakeSegmentsFromTexts(unittest.TestCase):
    def test_basic(self):
        segs = make_segments_from_texts(["Hello", "World"], notes_prefix="Line ")
        self.assertEqual(len(segs), 2)
        self.assertEqual(segs[0]["source"], "Hello")
        self.assertEqual(segs[0]["notes"], "Line 1")
        self.assertEqual(segs[1]["source"], "World")

```

```

def test_empty_text_skipped(self):
    segs = make_segments_from_texts(["Hello", "", "World"])
    self.assertEqual(len(segs), 2)

def test_with_locations(self):
    locs = [{"line": 0}, {"line": 2}]
    segs = make_segments_from_texts(["Hello", "World"], source_locations=locs)
    self.assertEqual(segs[0]["source_location"], {"line": 0})
    self.assertEqual(segs[1]["source_location"], {"line": 2})

def test_tag_extraction(self):
    segs = make_segments_from_texts(["<b>Hello</b>"])
    self.assertIn("<b>", segs[0]["all_tags"])
    self.assertIn("</b>", segs[0]["all_tags"])

class TestFileHandler(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.handler = CATFileHandler()

    def tearDown(self):
        self.tmp_dir.cleanup()

    def _make_po(self, content):
        path = os.path.join(self.tmp_dir.name, "test.po")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content.strip())
        return path

    def test_load_po(self):
        path = self._make_po("""msgid "Hello"
msgstr "Hola"
msgid "World"
msgstr "Mundo"
""")
        segments = self.handler.load_file(path)
        self.assertGreater(len(segments), 0)

    def test_load_nonexistent(self):
        with self.assertRaises(FileNotFoundError):
            self.handler.load_file("nonexistent.po")

    def test_load_unsupported(self):
        path = os.path.join(self.tmp_dir.name, "test.xyz")
        with open(path, "w") as f:
            f.write("data")
        with self.assertRaises(ValueError):
            self.handler.load_file(path)

    def test_save_po(self):
        path = self._make_po("""msgid "Hello"
msgstr ""
""")
        segments = self.handler.load_file(path)
        segments[0]["target"] = "Hola"
        save_path = os.path.join(self.tmp_dir.name, "output.po")
        self.handler.save_file(save_path)
        self.assertTrue(os.path.exists(save_path))
        with open(save_path, encoding="utf-8") as f:
            content = f.read()
        self.assertIn("Hola", content)

    def test_save_no_segments(self):
        with self.assertRaises(ValueError):
            self.handler.save_file("out.po")

    def test_atomic_save(self):
        path = self._make_po("""msgid "Hello"
msgstr ""
""")
        self.handler.load_file(path)
        save_path = os.path.join(self.tmp_dir.name, "atomic.po")
        self.handler.save_file(save_path)
        self.assertTrue(os.path.exists(save_path))

    def test_convert_to_po(self):
        txt_path = os.path.join(self.tmp_dir.name, "test.txt")
        with open(txt_path, "w", encoding="utf-8") as f:

```

```

        f.write("Hello\nWorld")
    po_out = os.path.join(self.tmp_dir.name, "out.po")
    convert_to_po(txt_path, po_out)
    self.assertTrue(os.path.exists(po_out))
    with open(po_out, encoding="utf-8") as f:
        content = f.read()
    self.assertIn("Hello", content)

def test_render_no_segments(self):
    with self.assertRaises(ValueError):
        self.handler.render_translated("out.txt")

class TestMCAProjectManager(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "test.mcat.db")

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_create_project(self):
        MCAProjectManager.create_project_file(self.db_path, "en", "es")
        self.assertTrue(os.path.exists(self.db_path))

    def test_create_project_auto_extension(self):
        path = os.path.join(self.tmp_dir.name, "test")
        MCAProjectManager.create_project_file(path, "en", "es")
        self.assertTrue(os.path.exists(path + ".mcat.db"))

    def test_import_and_get_stream(self):
        MCAProjectManager.create_project_file(self.db_path, "en", "es")
        segments = [
            {"file_id": "test.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
        ]
        MCAProjectManager.import_extracted_segments(self.db_path, segments)
        rows = MCAProjectManager.get_editor_stream(self.db_path)
        self.assertEqual(len(rows), 1)
        self.assertEqual(rows[0]["source_text"], "Hello")
        self.assertEqual(rows[0]["target_text"], "Hola")

    def test_get_stream_filtered(self):
        MCAProjectManager.create_project_file(self.db_path, "en", "es")
        segments = [
            {"file_id": "a.txt", "source": "Hello", "target": "", "status": "Untranslated", "match_score": 0},
            {"file_id": "a.txt", "source": "World", "target": "Mundo", "status": "Confirmed", "match_score": 100},
        ]
        MCAProjectManager.import_extracted_segments(self.db_path, segments)
        rows = MCAProjectManager.get_editor_stream(self.db_path, filter_status="Confirmed")
        self.assertEqual(len(rows), 1)
        self.assertEqual(rows[0]["source_text"], "World")

    def test_update_segment(self):
        MCAProjectManager.create_project_file(self.db_path, "en", "es")
        segments = [
            {"file_id": "a.txt", "source": "Hello", "target": "", "status": "Untranslated", "match_score": 0},
        ]
        MCAProjectManager.import_extracted_segments(self.db_path, segments)
        rows = MCAProjectManager.get_editor_stream(self.db_path)
        unit_id = rows[0]["id"]
        MCAProjectManager.update_segment_translation(self.db_path, unit_id, "Hola", "Confirmed")
        rows = MCAProjectManager.get_editor_stream(self.db_path)
        self.assertEqual(rows[0]["target_text"], "Hola")
        self.assertEqual(rows[0]["status"], "Confirmed")

class TestTextHandler(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_load_and_render(self):
        from cat_tool.formats.text import TextHandler
        txt_path = os.path.join(self.tmp_dir.name, "test.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Hello\nWorld\nFoo")
        handler = TextHandler()

```

```

        segments = handler.load(txt_path)
        self.assertEqual(len(segments), 3)
        segments[0]["target"] = "Hola"
        segments[1]["target"] = "Mundo"
        out_path = os.path.join(self.tmp_dir.name, "out.txt")
        handler.render(txt_path, segments, out_path)
        with open(out_path, encoding="utf-8") as f:
            content = f.read()
        self.assertIn("Hola", content)
        self.assertIn("Mundo", content)

class TestMcatDbHandler(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "test.mcat.db")
        McatProjectManager.create_project_file(self.db_path, "en", "es")

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_load(self):
        from cat_tool.formats.mcatdb import McatDbHandler
        McatProjectManager.import_extracted_segments(self.db_path, [
            {"file_id": "f.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
        ])
        handler = McatDbHandler()
        segments = handler.load(self.db_path)
        self.assertEqual(len(segments), 1)
        self.assertEqual(segments[0]["source"], "Hello")
        self.assertEqual(segments[0]["target"], "Hola")
        self.assertFalse(segments[0]["fuzzy"])
        self.assertTrue(segments[0]["translated"])

    def test_load_with_tags(self):
        from cat_tool.formats.mcatdb import McatDbHandler
        conn = sqlite3.connect(self.db_path)
        conn.executescript(SCHEMA_SQL)
        conn.execute("INSERT INTO translation_units (file_id, source_text, target_text) VALUES (?, ?, ?)",
            ("f.txt", "<b>Hello</b>", "<b>Hola</b>"))
        conn.commit()
        conn.close()
        segments = McatDbHandler().load(self.db_path)
        self.assertEqual(len(segments), 1)
        self.assertIn("<b>", segments[0]["all_tags"])

    def test_save(self):
        from cat_tool.formats.mcatdb import McatDbHandler
        segs = [{
            "index": 0, "source": "Hello", "target": "Hola",
            "all_tags": [], "source_clean": "Hello", "target_clean": "Hola",
            "fuzzy": False, "translated": True, "locations": "f.txt",
            "notes": "", "source_location": {}, "store": None, "unit": None,
            "match_score": 100,
        }]
        out_path = os.path.join(self.tmp_dir.name, "out.mcat.db")
        McatDbHandler().save(segs, out_path)
        self.assertTrue(os.path.exists(out_path))
        loaded = McatDbHandler().load(out_path)
        self.assertEqual(len(loaded), 1)
        self.assertEqual(loaded[0]["target"], "Hola")

    def test_save_empty_raises(self):
        from cat_tool.formats.mcatdb import McatDbHandler
        with self.assertRaises(ValueError):
            McatDbHandler().save([], "out.mcat.db")

    def test_render_copies(self):
        from cat_tool.formats.mcatdb import McatDbHandler
        out_path = os.path.join(self.tmp_dir.name, "copy.mcat.db")
        McatDbHandler().render(self.db_path, [], out_path)
        self.assertTrue(os.path.exists(out_path))

if __name__ == "__main__":
    unittest.main()

```

tests/test_remaining.py — 722 lines

- Class **TestProject** (line 18)
- Class **TestMqxcliffHandler** (line 155)
- Class **TestSdcliffHandler** (line 248)
- Class **TestGuiSmoke** (line 379)
- Class **TestFileHandlerIntegration** (line 505)
- Class **TestRealDocumentLoad** (line 612)
- Function **setUp** (line 19)
- Function **tearDown** (line 22)
- Function **test_new_project** (line 25)
- Function **test_save_and_load** (line 35)
- Function **test_save_no_path_raises** (line 46)
- Function **test_to_dict_roundtrip** (line 52)
- Function **test_default_dir** (line 62)
- Function **test_recent_projects** (line 66)
- Function **test_str** (line 73)
- Function **test_omegat_save_and_load** (line 80)
- Function **test_omegat_load_raises_on_missing** (line 96)
- Function **test_memoq_save_and_load** (line 100)
- Function **test_memoq_load_raises_on_missing** (line 123)
- Function **test_load_omegat_from_minimal_xml** (line 127)
- Function **test_load_omegat_missing_project_element** (line 142)
- Function **setUp** (line 156)
- Function **tearDown** (line 159)
- Function **_make_mqxcliff** (line 162)
- Function **test_load_simple** (line 168)
- Function **test_load_empty_target** (line 187)
- Function **test_load_missing_body_raises** (line 205)
- Function **test_save_and_reload** (line 215)
- Function **test_save_no_target** (line 231)
- Function **setUp** (line 249)
- Function **tearDown** (line 252)
- Function **_make_sdcliff** (line 255)
- Function **test_load_simple** (line 261)
- Function **test_load_skips_translate_no** (line 284)
- Function **test_load_skip_no_source** (line 308)
- Function **test_save_and_reload** (line 325)
- Function **test_save_without_target** (line 341)
- Function **test_target_without_mrk** (line 355)
- Function **setUp** (line 380)
- Function **tearDown** (line 383)
- Function **test_create_mainwindow** (line 386)
- Function **test_load_po_file_through_handler** (line 402)
- Function **test_load_po_and_translate** (line 428)
- Function **test_qa_dialog_creation** (line 453)
- Function **test_shortcuts_dialog_creation** (line 472)
- Function **test_glossary_dialog_creation** (line 487)
- Function **setUp** (line 506)
- Function **tearDown** (line 509)
- Function **_make_po** (line 512)
- Function **test_po_load_save_roundtrip** (line 518)
- Function **test_txt_load_save_roundtrip** (line 536)
- Function **test_po_convert_to_po_from_txt** (line 552)
- Function **test_factory_xliff_load** (line 564)
- Function **test_render_no_current_path_raises** (line 584)

- Function **test_load_mcatdb_and_save_mcatdb_roundtrip** (line 590)
- Function **setUp** (line 613)
- Function **tearDown** (line 616)
- Function **test_docx_load** (line 619)
- Function **test_docx_render** (line 633)
- Function **test_pptx_load** (line 650)
- Function **test_pptx_render** (line 665)
- Function **test_xlsx_load** (line 688)
- Function **test_xlsx_render** (line 702)

```
import unittest
import os
import tempfile
import json
import shutil
import sys

from cat_tool.project import Project
from cat_tool.file_handler import CATFileHandler
from cat_tool.formats.mqxliif import MqxliifHandler
from cat_tool.formats.sdxliif import SdxliifHandler
from cat_tool.formats.text import TextHandler
from cat_tool.formats._text_base import make_segments_from_texts

# ===== PROJECT TESTS =====

class TestProject(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_new_project(self):
        p = Project()
        p.new("Test Proj", "en", "es", ["a.txt", "b.txt"])
        self.assertEqual(p.name, "Test Proj")
        self.assertEqual(p.source_lang, "en")
        self.assertEqual(p.target_lang, "es")
        self.assertEqual(len(p.source_files), 2)
        self.assertTrue(p.created_at)
        self.assertTrue(p.modified_at)

    def test_save_and_load(self):
        p = Project()
        p.new("SaveTest", "fr", "de", [])
        proj_path = os.path.join(self.tmp.name, "test.mcatproj")
        p.save(proj_path)
        self.assertTrue(os.path.exists(proj_path))
        loaded = Project.load(proj_path)
        self.assertEqual(loaded.name, "SaveTest")
        self.assertEqual(loaded.source_lang, "fr")
        self.assertEqual(loaded.target_lang, "de")

    def test_save_no_path_raises(self):
        p = Project()
        p.new("NoPath", "en", "es", [])
        with self.assertRaises(ValueError):
            p.save()

    def test_to_dict_roundtrip(self):
        p = Project()
        p.new("DictTest", "en", "es", ["f1.txt"])
        d = p.to_dict()
        self.assertEqual(d["version"], 1)
        self.assertEqual(d["name"], "DictTest")
        p2 = Project.from_dict(d)
        self.assertEqual(p2.name, "DictTest")
        self.assertEqual(p2.source_lang, "en")

    def test_default_dir(self):
        d = Project.default_dir()
        self.assertTrue(os.path.isabs(d))
```

```

def test_recent_projects(self):
    Project.add_recent("/fake/path", "FakeProj")
    recents = Project.get_recent()
    found = any(r.get("path") == "/fake/path" for r in recents)
    self.assertTrue(found)
    self.assertLessEqual(len(recents), 10)

def test_str(self):
    p = Project()
    p.new("StrTest", "en", "es", [])
    s = str(p)
    self.assertIn("StrTest", s)
    self.assertIn("en", s)

def test_omegat_save_and_load(self):
    src_file = os.path.join(self.tmp.name, "doc.txt")
    with open(src_file, "w", encoding="utf-8") as f:
        f.write("Hello")
    p = Project()
    p.new("OmegaTTest", "en", "es", [src_file])
    omegat_dir = os.path.join(self.tmp.name, "OmegaTTest")
    p.save_as_omegat(omegat_dir)
    self.assertTrue(os.path.exists(os.path.join(omegat_dir, "omegat.project")))
    self.assertTrue(os.path.exists(os.path.join(omegat_dir, "source", "doc.txt")))
    loaded = Project.load_omegat(omegat_dir)
    self.assertEqual(loaded.name, "OmegaTTest")
    self.assertEqual(loaded.source_lang, "en")
    self.assertEqual(loaded.target_lang, "es")
    self.assertIn("doc.txt", loaded.source_files[0])

def test_omegat_load_raises_on_missing(self):
    with self.assertRaises(FileNotFoundError):
        Project.load_omegat("nonexistent_dir")

def test_memoq_save_and_load(self):
    src_file = os.path.join(self.tmp.name, "doc.txt")
    with open(src_file, "w", encoding="utf-8") as f:
        f.write("Hello")
    p = Project()
    p.new("MemoQTest", "en", "es", [src_file])

    handler = CATFileHandler()
    handler.segments = make_segments_from_texts(["Hello"], notes_prefix="")
    handler.segments[0]["target"] = "Hola"

    memoq_dir = os.path.join(self.tmp.name, "memoq_out")
    p.save_as_memoq(memoq_dir, handler)
    self.assertTrue(os.path.exists(os.path.join(memoq_dir, "manifest.xml")))
    self.assertTrue(os.path.exists(os.path.join(memoq_dir, "source", "doc.txt")))
    self.assertTrue(os.path.exists(os.path.join(memoq_dir, "target", "MemoQTest.mqxcliff")))

    loaded = Project.load_memoq(memoq_dir)
    self.assertEqual(loaded.name, "MemoQTest")
    self.assertEqual(loaded.source_lang, "en")
    self.assertEqual(loaded.target_lang, "es")
    self.assertGreater(len(loaded.mqxcliff_files), 0)

def test_memoq_load_raises_on_missing(self):
    with self.assertRaises(FileNotFoundError):
        Project.load_memoq("nonexistent_dir")

def test_load_omegat_from_minimal_xml(self):
    omegat_dir = os.path.join(self.tmp.name, "minimal_omegat")
    os.makedirs(omegat_dir)
    os.makedirs(os.path.join(omegat_dir, "source"))
    et = ET = __import__("xml.etree.ElementTree", fromlist=["ElementTree"])
    root = et.Element("omegat")
    pe = et.SubElement(root, "project")
    et.SubElement(pe, "source_lang").text = "EN-US"
    et.SubElement(pe, "target_lang").text = "FR"
    tree = et.ElementTree(root)
    tree.write(os.path.join(omegat_dir, "omegat.project"), encoding="utf-8", xml_declaration=True)
    loaded = Project.load_omegat(omegat_dir)
    self.assertEqual(loaded.source_lang, "en-us")
    self.assertEqual(loaded.target_lang, "fr")

def test_load_omegat_missing_project_element(self):
    omegat_dir = os.path.join(self.tmp.name, "bad_omegat")
    os.makedirs(omegat_dir)

```

```

et = ET = __import__("xml.etree.ElementTree", fromlist=["ElementTree"])
root = et.Element("omegat")
tree = et.ElementTree(root)
tree.write(os.path.join(omegat_dir, "omegat.project"), encoding="utf-8", xml_declaration=True)
with self.assertRaises(ValueError):
    Project.load_omegat(omegat_dir)

# ===== MQXLIFF TESTS =====

class TestMqxliffHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def _make_mqxliff(self, content):
        path = os.path.join(self.tmp.name, "test.mqxliff")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content)
        return path

    def test_load_simple(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <target>Hola</target>
      </trans-unit>
    </body>
  </file>
</xliff>"""
        path = self._make_mqxliff(xml)
        segs = MqxliffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["source"], "Hello")
        self.assertEqual(segs[0]["target"], "Hola")
        self.assertTrue(segs[0]["translated"])

    def test_load_empty_target(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <target></target>
      </trans-unit>
    </body>
  </file>
</xliff>"""
        path = self._make_mqxliff(xml)
        segs = MqxliffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["target"], "")
        self.assertFalse(segs[0]["translated"])

    def test_load_missing_body_raises(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    </file>
  </xliff>"""
        path = self._make_mqxliff(xml)
        with self.assertRaises(ValueError):
            MqxliffHandler().load(path)

    def test_save_and_reload(self):
        segs = [{
            "index": 0, "source": "Hello", "target": "Hola",
            "source_clean": "Hello", "target_clean": "Hola",
            "all_tags": [], "notes": "",
            "fuzzy": False, "translated": True,
            "locations": "", "store": None, "unit": None,
        }]
        out = os.path.join(self.tmp.name, "out.mqxliff")

```

```

MqxLiffHandler().save(segs, out, src_lang="en", tgt_lang="es")
self.assertTrue(os.path.exists(out))
reloaded = MqxLiffHandler().load(out)
self.assertEqual(len(reloaded), 1)
self.assertEqual(reloaded[0]["source"], "Hello")
self.assertEqual(reloaded[0]["target"], "Hola")

def test_save_no_target(self):
    segs = [{
        "index": 0, "source": "Hello", "target": "",
        "source_clean": "Hello", "target_clean": "",
        "all_tags": [], "notes": "",
        "fuzzy": False, "translated": False,
        "locations": "", "store": None, "unit": None,
    }]
    out = os.path.join(self.tmp.name, "empty.mqxLiff")
    MqxLiffHandler().save(segs, out)
    self.assertTrue(os.path.exists(out))
    reloaded = MqxLiffHandler().load(out)
    self.assertEqual(reloaded[0]["target"], "")

# ===== SDLXLIFF TESTS =====

class TestSdlxLiffHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def _make_sdlxLiff(self, content):
        path = os.path.join(self.tmp.name, "test.sdlxLiff")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content)
        return path

    def test_load_simple(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
<file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
<header/>
<body>
<trans-unit id="1">
<source>Hello</source>
<seg-source><mrk mtype="seg" mid="1">Hello</mrk></seg-source>
<target><mrk mtype="seg" mid="1">Hola</mrk></target>
<sdl:seg-defs><sdl:seg id="1" conf="Translated"/></sdl:seg-defs>
</trans-unit>
</body>
</file>
</xliff>"""
        path = self._make_sdlxLiff(xml)
        segs = SdlxLiffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["source"], "Hello")
        self.assertEqual(segs[0]["target"], "Hola")
        self.assertTrue(segs[0]["translated"])

    def test_load_skips_translate_no(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
<file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
<header/>
<body>
<trans-unit id="1" translate="no">
<source>Skip me</source>
</trans-unit>
<trans-unit id="2">
<source>Hello</source>
<seg-source><mrk mtype="seg" mid="1">Hello</mrk></seg-source>
<target><mrk mtype="seg" mid="1">Hola</mrk></target>
<sdl:seg-defs><sdl:seg id="1" conf="Translated"/></sdl:seg-defs>
</trans-unit>
</body>
</file>
</xliff>"""

```

```

        path = self._make_sdlxliff(xml)
        segs = SdlxliffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["source"], "Hello")

    def test_load_skip_no_source(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
    xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
    <file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
        <header/>
        <body>
            <trans-unit id="1">
                <seg-source><mrk mtype="seg" mid="1"></mrk></seg-source>
                </trans-unit>
            </body>
        </file>
    </xliff>"""
        path = self._make_sdlxliff(xml)
        segs = SdlxliffHandler().load(path)
        self.assertEqual(len(segs), 0)

    def test_save_and_reload(self):
        segs = [{
            "index": 0, "source": "Hello", "target": "Hola",
            "source_clean": "Hello", "target_clean": "Hola",
            "all_tags": [], "notes": "",
            "fuzzy": False, "translated": True,
            "locations": "", "store": None, "unit": None,
        }]
        out = os.path.join(self.tmp.name, "out.sdlxliff")
        SdlxliffHandler().save(segs, out, src_lang="en", tgt_lang="es")
        self.assertTrue(os.path.exists(out))
        reloaded = SdlxliffHandler().load(out)
        self.assertEqual(len(reloaded), 1)
        self.assertEqual(reloaded[0]["source"], "Hello")
        self.assertEqual(reloaded[0]["target"], "Hola")

    def test_save_without_target(self):
        segs = [{
            "index": 0, "source": "Hello", "target": "",
            "source_clean": "Hello", "target_clean": "",
            "all_tags": [], "notes": "",
            "fuzzy": False, "translated": False,
            "locations": "", "store": None, "unit": None,
        }]
        out = os.path.join(self.tmp.name, "empty.sdlxliff")
        SdlxliffHandler().save(segs, out)
        self.assertTrue(os.path.exists(out))
        reloaded = SdlxliffHandler().load(out)
        self.assertEqual(reloaded[0]["target"], "")

    def test_target_without_mrk(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
    xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
    <file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
        <header/>
        <body>
            <trans-unit id="1">
                <source>Hello</source>
                <seg-source><mrk mtype="seg" mid="1">Hello</mrk></seg-source>
                <target>Hola</target>
                <sdl:seg-defs><sdl:seg id="1" conf="Translated"></sdl:seg-defs>
            </trans-unit>
        </body>
    </file>
    </xliff>"""
        path = self._make_sdlxliff(xml)
        segs = SdlxliffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["target"], "Hola")

```

===== GUI SMOKE TEST =====

```

class TestGuiSmoke(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

```

```

def tearDown(self):
    self.tmp.cleanup()

def test_create_mainwindow(self):
    from PyQt5.QtWidgets import QApplication
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import MainWindow
        win = MainWindow()
        self.assertIsNotNone(win)
        self.assertEqual(win.windowTitle()[4:], "mcat")
    finally:
        if not existed:
            pass
        app = None

def test_load_po_file_through_handler(self):
    from PyQt5.QtWidgets import QApplication
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import MainWindow
        win = MainWindow()
        po_content = '''msgid "Hello"

msgstr "Hola"
msgid "World"
msgstr "Mundo"
'''
        po_path = os.path.join(self.tmp.name, "test.po")
        with open(po_path, "w", encoding="utf-8") as f:
            f.write(po_content)
        segs = win.file_handler.load_file(po_path)
        win.populate_table(segs)
        self.assertEqual(win.table.rowCount(), 2)
        self.assertEqual(win.table.item(0, 0).text(), "Hello")
    finally:
        if not existed:
            pass
        app = None

def test_load_po_and_translate(self):
    from PyQt5.QtWidgets import QApplication
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import MainWindow
        win = MainWindow()
        po_content = '''msgid "Hello"

msgstr ""
'''
        po_path = os.path.join(self.tmp.name, "test.po")
        with open(po_path, "w", encoding="utf-8") as f:
            f.write(po_content)
        segs = win.file_handler.load_file(po_path)
        win.populate_table(segs)
        win.file_handler.segments[0]["target"] = "Hola"
        win.update_table_row_status(0, win.file_handler.segments[0])
        self.assertEqual(win.file_handler.segments[0]["target"], "Hola")
    finally:
        if not existed:
            pass
        app = None

def test_qa_dialog_creation(self):
    from PyQt5.QtWidgets import QApplication, QDialog
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.qa import QAIssue
        from cat_tool.ui import QDialog

```

```

        issues = [
            QAIssue("error", "Missing translation", 0, "Hello", "", "No target"),
        ]
        dlg = QADialog(issues)
        self.assertIsInstance(dlg, QDialog)
    finally:
        if not existed:
            pass
        app = None

def test_shortcuts_dialog_creation(self):
    from PyQt5.QtWidgets import QApplication, QDialog
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import ShortcutsDialog
        dlg = ShortcutsDialog()
        self.assertIsInstance(dlg, QDialog)
    finally:
        if not existed:
            pass
        app = None

def test_glossary_dialog_creation(self):
    from PyQt5.QtWidgets import QApplication, QDialog
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import GlossaryDialog
        dlg = GlossaryDialog()
        self.assertIsInstance(dlg, QDialog)
    finally:
        if not existed:
            pass
        app = None

# ===== FORMAT HANDLER INTEGRATION =====

class TestFileHandlerIntegration(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def _make_po(self, content):
        path = os.path.join(self.tmp.name, "test.po")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content)
        return path

    def test_po_load_save_roundtrip(self):
        po = '''msgid "Hello"
msgstr "Hola"
msgid "World"
msgstr ""
'''
        path = self._make_po(po)
        fh = CATFileHandler()
        segs = fh.load_file(path)
        segs[0]["target"] = ";Hola!"
        segs[1]["target"] = "Mundo"
        out = os.path.join(self.tmp.name, "out.po")
        fh.save_file(out)
        fh2 = CATFileHandler()
        segs2 = fh2.load_file(out)
        self.assertEqual(segs2[0]["target"], ";Hola!")
        self.assertEqual(segs2[1]["target"], "Mundo")

    def test_txt_load_save_roundtrip(self):
        txt_path = os.path.join(self.tmp.name, "input.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Line one\nLine two\nLine three")
        fh = CATFileHandler()

```

```

        segs = fh.load_file(txt_path)
        self.assertEqual(len(segs), 3)
        segs[0]["target"] = "Línea uno"
        segs[1]["target"] = "Línea dos"
        out = os.path.join(self.tmp.name, "out.txt")
        fh.render_translated(out)
        with open(out, encoding="utf-8") as f:
            content = f.read()
        self.assertIn("Línea uno", content)
        self.assertIn("Línea dos", content)

    def test_po_convert_to_po_from_txt(self):
        from cat_tool.file_handler import convert_to_po
        txt_path = os.path.join(self.tmp.name, "source.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Hello\nWorld")
        po_out = os.path.join(self.tmp.name, "converted.po")
        convert_to_po(txt_path, po_out)
        self.assertTrue(os.path.exists(po_out))
        fh = CATFileHandler()
        segs = fh.load_file(po_out)
        self.assertEqual(len(segs), 2)

    def test_factory_xliff_load(self):
        xliff = """<?xml version="1.0" encoding="utf-8"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file source-language="en" target-language="es" datatype="plaintext" original="test.txt">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <target>Hola</target>
      </trans-unit>
    </body>
  </file>
</xliff>"""
        path = os.path.join(self.tmp.name, "test.xliff")
        with open(path, "w", encoding="utf-8") as f:
            f.write(xliff)
        fh = CATFileHandler()
        segs = fh.load_file(path)
        self.assertGreater(len(segs), 0)
        self.assertEqual(segs[0]["source"].strip(), "Hello")

    def test_render_no_current_path_raises(self):
        fh = CATFileHandler()
        fh.segments = [{"source": "Hello", "target": "Hola"}]
        with self.assertRaises(ValueError):
            fh.render_translated("out.txt")

    def test_load_mcatdb_and_save_mcatdb_roundtrip(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        db_path = os.path.join(self.tmp.name, "proj.mcat.db")
        MCatProjectManager.create_project_file(db_path, "en", "es")
        MCatProjectManager.import_extracted_segments(db_path, [
            {"file_id": "f.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
        ])
        fh = CATFileHandler()
        segs = fh.load_file(db_path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["source"], "Hello")
        self.assertEqual(segs[0]["target"], "Hola")
        out_db = os.path.join(self.tmp.name, "copy.mcat.db")
        fh.save_file(out_db)
        self.assertTrue(os.path.exists(out_db))
        fh2 = CATFileHandler()
        segs2 = fh2.load_file(out_db)
        self.assertEqual(segs2[0]["source"], "Hello")

# ===== REAL DOCUMENT LOAD TESTS =====

class TestRealDocumentLoad(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_docx_load(self):

```

```

from docx import Document
doc = Document()
doc.add_paragraph("Hello World")
doc.add_paragraph("Second paragraph")
docx_path = os.path.join(self.tmp.name, "test.docx")
doc.save(docx_path)
from cat_tool.formats.docx import DocxHandler
segs = DocxHandler().load(docx_path)
self.assertGreaterEqual(len(segs), 2)
texts = [s["source"] for s in segs]
self.assertIn("Hello World", texts)
self.assertIn("Second paragraph", texts)

def test_docx_render(self):
    from docx import Document
    doc = Document()
    doc.add_paragraph("Hello")
    docx_path = os.path.join(self.tmp.name, "test.docx")
    doc.save(docx_path)
    from cat_tool.formats.docx import DocxHandler
    handler = DocxHandler()
    segs = handler.load(docx_path)
    segs[0]["target"] = "Hola"
    out_path = os.path.join(self.tmp.name, "out.docx")
    handler.render(docx_path, segs, out_path)
    self.assertTrue(os.path.exists(out_path))
    doc2 = Document(out_path)
    texts = [p.text for p in doc2.paragraphs]
    self.assertIn("Hola", texts)

def test_pptx_load(self):
    from pptx import Presentation
    from pptx.util import Inches
    prs = Presentation()
    slide = prs.slides.add_slide(prs.slide_layouts[6])
    textBox = slide.shapes.add_textbox(Inches(1), Inches(1), Inches(8), Inches(2))
    tf = textBox.text_frame
    tf.text = "Hello from PowerPoint"
    pptx_path = os.path.join(self.tmp.name, "test.pptx")
    prs.save(pptx_path)
    from cat_tool.formats.pptx import PptxHandler
    segs = PptxHandler().load(pptx_path)
    texts = [s["source"] for s in segs]
    self.assertIn("Hello from PowerPoint", texts)

def test_pptx_render(self):
    from pptx import Presentation
    from pptx.util import Inches
    prs = Presentation()
    slide = prs.slides.add_slide(prs.slide_layouts[6])
    textBox = slide.shapes.add_textbox(Inches(1), Inches(1), Inches(8), Inches(2))
    tf = textBox.text_frame
    tf.text = "Hello"
    pptx_path = os.path.join(self.tmp.name, "test.pptx")
    prs.save(pptx_path)
    from cat_tool.formats.pptx import PptxHandler
    handler = PptxHandler()
    segs = handler.load(pptx_path)
    segs[0]["target"] = "Hola"
    out_path = os.path.join(self.tmp.name, "out.pptx")
    handler.render(pptx_path, segs, out_path)
    self.assertTrue(os.path.exists(out_path))
    prs2 = Presentation(out_path)
    for slide in prs2.slides:
        for shape in slide.shapes:
            if shape.has_text_frame:
                self.assertIn("Hola", shape.text_frame.text)

def test_xlsx_load(self):
    from openpyxl import Workbook
    wb = Workbook()
    ws = wb.active
    ws["A1"] = "Hello"
    ws["B1"] = "World"
    xlsx_path = os.path.join(self.tmp.name, "test.xlsx")
    wb.save(xlsx_path)
    from cat_tool.formats.xlsx import XlsxHandler
    segs = XlsxHandler().load(xlsx_path)
    texts = [s["source"] for s in segs]

```

```

        self.assertIn("Hello", texts)
        self.assertIn("World", texts)

    def test_xlsx_render(self):
        from openpyxl import Workbook
        wb = Workbook()
        ws = wb.active
        ws["A1"] = "Hello"
        xlsx_path = os.path.join(self.tmp.name, "test.xlsx")
        wb.save(xlsx_path)
        from cat_tool.formats.xlsx import XlsxHandler
        handler = XlsxHandler()
        segs = handler.load(xlsx_path)
        segs[0]["target"] = "Hola"
        out_path = os.path.join(self.tmp.name, "out.xlsx")
        handler.render(xlsx_path, segs, out_path)
        self.assertTrue(os.path.exists(out_path))
        from openpyxl import load_workbook
        wb2 = load_workbook(out_path)
        self.assertEqual(wb2.active["A1"].value, "Hola")

if __name__ == "__main__":
    unittest.main()

```

teststest_more.py — 1026 lines

- Class **TestSpellChecker** (line 12)
- Class **TestSpellCheckerConstants** (line 250)
- Class **TestAppSettings** (line 267)
- Class **TestQAModuleAdditional** (line 362)
- Class **TestSegmentationAdditional** (line 396)
- Class **TestTagUtilsAdditional** (line 446)
- Class **TestFileHandlerAdditional** (line 479)
- Class **TestXlsHandler** (line 544)
- Class **TestOdfHandler** (line 597)
- Class **TestMakeSegmentsFromTextsAdditional** (line 664)
- Class **TestConvertToPoAdditional** (line 683)
- Class **TestMcatProjectManagerAdditional** (line 717)
- Class **TestQACheckFunctions** (line 768)
- Class **TestTranslationMemoryAdditional** (line 799)
- Class **TestGlossaryAdditional** (line 829)
- Function **setUpClass** (line 14)
- Function **tearDownClass** (line 22)
- Function **setUp** (line 29)
- Function **_make_real_dic** (line 34)
- Function **test_empty_checker** (line 44)
- Function **test_auto_load_picks_up_dict** (line 54)
- Function **test_check_known_word** (line 67)
- Function **test_check_unknown_word** (line 80)
- Function **test_auto_load_no_dicts_dir** (line 92)
- Function **test_auto_load_empty_dicts_dir** (line 98)
- Function **test_auto_load_skips_non_dic_files** (line 105)
- Function **test_auto_load_skips_missing_aff** (line 114)
- Function **test_load_dictionary_missing_aff** (line 124)
- Function **test_load_dictionary_copies_and_loads** (line 130)
- Function **test_suggest_with_dict** (line 144)
- Function **test_check_text_deduplicates** (line 157)
- Function **test_check_text_catches_errors** (line 171)
- Function **test_lang_id_from_archive_with_underscore** (line 185)
- Function **test_lang_id_from_archive_uses_filename_fallback** (line 196)
- Function **test_lang_id_from_archive_with_dic_inside** (line 206)

- Function **test_load_from_archive_nonexistent_returns_false** (line 217)
- Function **test_load_from_archive_bad_zip_returns_false** (line 223)
- Function **test_load_from_archive_good_zip** (line 232)
- Function **test_dicts_dir_uses_appdata** (line 251)
- Function **setUp** (line 268)
- Function **tearDown** (line 275)
- Function **_settings_path** (line 281)
- Function **test_defaults** (line 284)
- Function **test_save_and_load** (line 290)
- Function **test_partial_overrides_keep_defaults** (line 304)
- Function **test_corrupted_json_falls_back_to_defaults** (line 313)
- Function **test_missing_file_falls_back_to_defaults** (line 321)
- Function **test_contains** (line 326)
- Function **test_get** (line 332)
- Function **test_as_dict** (line 339)
- Function **test_setitem_updates_dict** (line 347)
- Function **test_save_creates_directory** (line 353)
- Function **test_mixed_severity** (line 363)
- Function **test_empty_description** (line 370)
- Function **test_extract_numbers_edge_cases** (line 375)
- Function **test_whitespace_no_source** (line 381)
- Function **test_count_tags_empty** (line 387)
- Function **test_none_text** (line 397)
- Function **test_multiple_periods** (line 402)
- Function **test_exclamation_and_question** (line 407)
- Function **test_break_before_literal** (line 412)
- Function **test_break_before_combined** (line 421)
- Function **test_break_before_and_after** (line 432)
- Function **test_extract_single_tag** (line 447)
- Function **test_extract_nested_tags** (line 452)
- Function **test_extract_self_closing_xml** (line 460)
- Function **test_restore_with_none_tags** (line 466)
- Function **test_apply_with_none_text** (line 471)
- Function **setUp** (line 480)
- Function **tearDown** (line 484)
- Function **test_atomic_save_failure_cleans_temp** (line 487)
- Function **test_save_file_without_loading_raises** (line 499)
- Function **test_current_file_path_none_on_new** (line 504)
- Function **test_load_with_mcatdb_extension** (line 507)
- Function **test_render_without_handler_raises** (line 521)
- Function **test_save_file_mcatdb_preserves_path** (line 527)
- Function **setUp** (line 545)
- Function **tearDown** (line 548)
- Function **test_xls_load_basic** (line 551)
- Function **test_xls_load_empty_workbook** (line 570)
- Function **test_xls_load_nonexistent_raises** (line 586)
- Function **setUp** (line 598)
- Function **tearDown** (line 601)
- Function **test_load_odt** (line 604)
- Function **test_odf_load_nonexistent_raises** (line 620)
- Function **test_odf_render_updates_target** (line 628)
- Function **test_odf_render_no_segments_output_created** (line 648)
- Function **test_none_text_skipped** (line 665)
- Function **test_all_empty_skipped** (line 670)
- Function **test_normal_works** (line 675)

- Function **setUp** (line 684)
- Function **tearDown** (line 687)
- Function **test_convert_to_po_creates_file** (line 690)
- Function **test_convert_to_po_roundtrip** (line 703)
- Function **setUp** (line 718)
- Function **tearDown** (line 722)
- Function **test_get_editor_stream_empty_db** (line 725)
- Function **test_get_editor_stream_filter_nonexistent_status** (line 731)
- Function **test_update_segment_invalid_id** (line 737)
- Function **test_import_and_get_stream** (line 742)
- Function **test_update_segment** (line 753)
- Function **test_check_terminology_no_glossary** (line 769)
- Function **test_check_inconsistent_translations_empty_source** (line 775)
- Function **test_tag_mismatch_no_source_tags** (line 784)
- Function **test_count_tags_with_tags** (line 790)
- Function **setUp** (line 800)
- Function **tearDown** (line 806)
- Function **test_add_translation_none_target** (line 809)
- Function **test_get_fuzzy_matches_empty_db** (line 814)
- Function **test_get_fuzzy_matches_with_min_score** (line 818)
- Function **test_get_fuzzy_matches_extreme_min_score** (line 823)
- Function **setUp** (line 830)
- Function **tearDown** (line 836)
- Function **test_remove_nonexistent_term** (line 839)
- Function **test_check_segment_none_text** (line 844)
- Function **test_add_term_special_chars** (line 848)
- Function **test_add_term_with_langs** (line 853)
- Function **test_import_tmx_basic** (line 860)
- Function **test_import_tmx_nonexistent_file** (line 887)
- Function **test_import_tmx_empty_body** (line 891)
- Function **test_import_tmx_missing_body** (line 905)
- Function **test_import_tmx_no_srclang_header** (line 917)
- Function **test_import_tmx_with_explicit_source_lang** (line 935)
- Function **test_peek_tmx_basic** (line 957)
- Function **test_peek_tmx_no_srclang** (line 975)
- Function **test_export_tmx_empty** (line 992)
- Function **test_export_tmx_roundtrip** (line 997)
- Function **test_export_tmx_source_lang_override** (line 1014)
- Function **_fail** (line 492)

```
import unittest
import os
import sys
import tempfile
import json
import shutil
import importlib

# ===== SPELL CHECKER TESTS =====

class TestSpellChecker(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls._tmp = tempfile.TemporaryDirectory()
        cls._orig_appdata = os.environ.get("APPDATA")
        os.environ["APPDATA"] = cls._tmp.name
        import cat_tool.spellcheck
        importlib.reload(cat_tool.spellcheck)

    @classmethod
```

```

def tearDownClass(cls):
    if cls._orig_appdata:
        os.environ["APPDATA"] = cls._orig_appdata
    else:
        del os.environ["APPDATA"]
    cls._tmp.cleanup()

def setUp(self):
    dicts_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts")
    if os.path.isdir(dicts_dir):
        shutil.rmtree(dicts_dir)

def _make_real_dic(self, dic_path, aff_path, words=None):
    os.makedirs(os.path.dirname(dic_path), exist_ok=True)
    with open(aff_path, "w", encoding="utf-8") as f:
        f.write("SET UTF-8\n")
    words = words or ["hello", "world"]
    with open(dic_path, "w", encoding="utf-8") as f:
        f.write(f"{len(words)}\n")
        for w in words:
            f.write(f"{w}\n")

def test_empty_checker(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    self.assertFalse(sc.has_dict)
    self.assertEqual(sc.loaded_names, [])
    self.assertTrue(sc.check("anything"))
    self.assertEqual(sc.suggest("anything"), [])
    self.assertEqual(sc.check_text(""), [])
    self.assertEqual(sc.check_text(None), [])

def test_auto_load_picks_up_dict(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["Hello", "World"],
    )
    sc = SpellChecker()
    sc.auto_load()
    self.assertTrue(sc.has_dict)
    self.assertIn("en_US", sc.loaded_names)

def test_check_known_word(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["Hello", "World"],
    )
    sc = SpellChecker()
    sc.auto_load()
    self.assertTrue(sc.check("Hello"))
    self.assertTrue(sc.check("World"))

def test_check_unknown_word(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["Hello", "World"],
    )
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.check("xyzzzzz"))

def test_auto_load_no_dicts_dir(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.has_dict)

def test_auto_load_empty_dicts_dir(self):
    from cat_tool.spellcheck import SpellChecker
    os.makedirs(os.path.join(os.environ["APPDATA"], "mcat", "dicts"), exist_ok=True)

```

```

sc = SpellChecker()
sc.auto_load()
self.assertFalse(sc.has_dict)

def test_auto_load_skips_non_dic_files(self):
    from cat_tool.spellcheck import SpellChecker
    dicts_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "es")
    os.makedirs(dicts_dir, exist_ok=True)
    open(os.path.join(dicts_dir, "readme.txt"), "w").close()
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.has_dict)

def test_auto_load_skips_missing_aff(self):
    from cat_tool.spellcheck import SpellChecker
    dicts_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "fr")
    os.makedirs(dicts_dir, exist_ok=True)
    with open(os.path.join(dicts_dir, "fr.dic"), "w", encoding="utf-8") as f:
        f.write("l\nmot\n")
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.has_dict)

def test_load_dictionary_missing_aff(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    ok = sc.load_dictionary(r"C:\nonexistent.dic")
    self.assertFalse(ok)

def test_load_dictionary_copies_and_loads(self):
    from cat_tool.spellcheck import SpellChecker
    src_dir = os.path.join(self.tmp.name, "src")
    os.makedirs(src_dir, exist_ok=True)
    dic_path = os.path.join(src_dir, "custom.dic")
    aff_path = os.path.join(src_dir, "custom.aff")
    self._make_real_dic(dic_path, aff_path, ["foo", "bar"])
    sc = SpellChecker()
    ok = sc.load_dictionary(dic_path)
    self.assertTrue(ok)
    self.assertIn("custom", sc.loaded_names)
    self.assertTrue(sc.check("foo"))
    self.assertFalse(sc.check("baz"))

def test_suggest_with_dict(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["hello", "world"],
    )
    sc = SpellChecker()
    sc.auto_load()
    suggs = sc.suggest("worlt")
    self.assertIsInstance(suggs, list)

def test_check_text_deduplicates(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["hello"],
    )
    sc = SpellChecker()
    sc.auto_load()
    errors = sc.check_text("hello world world hello")
    err_words = [e[0] for e in errors]
    self.assertEqual(err_words.count("world"), 1)

def test_check_text_catches_errors(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["hello", "world", "test"],
    )
    sc = SpellChecker()

```

```

        sc.auto_load()
        errors = sc.check_text("hello worlt test")
        err_words = [e[0] for e in errors]
        self.assertIn("worlt", err_words)

    def test_lang_id_from_archive_with_underscore(self):
        import zipfile
        from cat_tool.spellcheck import SpellChecker
        sc = SpellChecker()
        archive_path = os.path.join(self._tmp.name, "dict_en_US.oxt")
        with zipfile.ZipFile(archive_path, "w") as z:
            z.writestr("en_US.dic", "l\nword\n")
            z.writestr("en_US.aff", "")
        lang_id = sc._lang_id_from_archive(archive_path)
        self.assertEqual(lang_id, "en_US")

    def test_lang_id_from_archive_uses_filename_fallback(self):
        import zipfile
        from cat_tool.spellcheck import SpellChecker
        sc = SpellChecker()
        archive_path = os.path.join(self._tmp.name, "spanish.oxt")
        with zipfile.ZipFile(archive_path, "w") as z:
            pass
        lang_id = sc._lang_id_from_archive(archive_path)
        self.assertEqual(lang_id, "spanish")

    def test_lang_id_from_archive_with_dic_inside(self):
        import zipfile
        from cat_tool.spellcheck import SpellChecker
        sc = SpellChecker()
        archive_path = os.path.join(self._tmp.name, "dicts.zip")
        with zipfile.ZipFile(archive_path, "w") as z:
            z.writestr("es_MX.dic", "l\nhola\n")
            z.writestr("es_MX.aff", "")
        lang_id = sc._lang_id_from_archive(archive_path)
        self.assertEqual(lang_id, "es_MX")

    def test_load_from_archive_nonexistent_returns_false(self):
        from cat_tool.spellcheck import SpellChecker
        sc = SpellChecker()
        ok = sc.load_from_archive(r"C:\nonexistent.zip")
        self.assertFalse(ok)

    def test_load_from_archive_bad_zip_returns_false(self):
        from cat_tool.spellcheck import SpellChecker
        sc = SpellChecker()
        bad_path = os.path.join(self._tmp.name, "bad.oxt")
        with open(bad_path, "w") as f:
            f.write("not a zip")
        ok = sc.load_from_archive(bad_path)
        self.assertFalse(ok)

    def test_load_from_archive_good_zip(self):
        import zipfile
        from cat_tool.spellcheck import SpellChecker
        sc = SpellChecker()
        dic_dir = os.path.join(self._tmp.name, "dictsrc")
        os.makedirs(dic_dir, exist_ok=True)
        with open(os.path.join(dic_dir, "es.dic"), "w", encoding="utf-8") as f:
            f.write("2\nhola\nmundo\n")
        with open(os.path.join(dic_dir, "es.aff"), "w", encoding="utf-8") as f:
            f.write("SET UTF-8\n")
        archive_path = os.path.join(self._tmp.name, "spanish.zip")
        with zipfile.ZipFile(archive_path, "w") as z:
            z.write(os.path.join(dic_dir, "es.dic"), "es.dic")
            z.write(os.path.join(dic_dir, "es.aff"), "es.aff")
        ok = sc.load_from_archive(archive_path)
        self.assertTrue(ok)

class TestSpellCheckerConstants(unittest.TestCase):
    def test_dicts_dir_uses_appdata(self):
        with tempfile.TemporaryDirectory() as tmp:
            old = os.environ.get("APPDATA")
            os.environ["APPDATA"] = tmp
            import cat_tool.spellcheck
            importlib.reload(cat_tool.spellcheck)
            expected = os.path.join(tmp, "mcat", "dicts")
            self.assertEqual(cat_tool.spellcheck.DICTS_DIR, expected)

```

```

        if old:
            os.environ["APPDATA"] = old
        else:
            del os.environ["APPDATA"]

# ===== SETTINGS TESTS =====

class TestAppSettings(unittest.TestCase):
    def setUp(self):
        self._tmp = tempfile.TemporaryDirectory()
        self._orig_appdata = os.environ.get("APPDATA")
        os.environ["APPDATA"] = self._tmp.name
        import cat_tool.settings
        importlib.reload(cat_tool.settings)

    def tearDown(self):
        if self._orig_appdata:
            os.environ["APPDATA"] = self._orig_appdata
        else:
            del os.environ["APPDATA"]

    def _settings_path(self):
        return os.path.join(self._tmp.name, "mcat", "settings.json")

    def test_defaults(self):
        from cat_tool.settings import AppSettings, DEFAULTS
        s = AppSettings().load()
        for k, v in DEFAULTS.items():
            self.assertEqual(s[k], v, f"Default {k} mismatch")

    def test_save_and_load(self):
        from cat_tool.settings import AppSettings
        s = AppSettings()
        s.load()
        s["source_font_size"] = 20
        s["target_font_size"] = 16
        s["auto_save_on_segment_change"] = True
        s.save()
        self.assertTrue(os.path.exists(self._settings_path()))
        s2 = AppSettings().load()
        self.assertEqual(s2["source_font_size"], 20)
        self.assertEqual(s2["target_font_size"], 16)
        self.assertTrue(s2["auto_save_on_segment_change"])

    def test_partial_overrides_keep_defaults(self):
        from cat_tool.settings import AppSettings
        os.makedirs(os.path.dirname(self._settings_path()), exist_ok=True)
        with open(self._settings_path(), "w") as f:
            json.dump({"source_font_size": 18}, f)
        s = AppSettings().load()
        self.assertEqual(s["source_font_size"], 18)
        self.assertEqual(s["target_font_size"], 13)

    def test_corrupted_json_falls_back_to_defaults(self):
        from cat_tool.settings import AppSettings, DEFAULTS
        os.makedirs(os.path.dirname(self._settings_path()), exist_ok=True)
        with open(self._settings_path(), "w") as f:
            f.write("not valid json")
        s = AppSettings().load()
        self.assertEqual(s["source_font_size"], DEFAULTS["source_font_size"])

    def test_missing_file_falls_back_to_defaults(self):
        from cat_tool.settings import AppSettings, DEFAULTS
        s = AppSettings().load()
        self.assertEqual(s["source_font_size"], DEFAULTS["source_font_size"])

    def test_contains(self):
        from cat_tool.settings import AppSettings
        s = AppSettings().load()
        self.assertIn("source_font_size", s)
        self.assertNotIn("nonexistent_key", s)

    def test_get(self):
        from cat_tool.settings import AppSettings
        s = AppSettings().load()
        self.assertEqual(s.get("source_font_size"), 13)
        self.assertIsNone(s.get("nonexistent"))
        self.assertEqual(s.get("nonexistent", 42), 42)

```

```

def test_as_dict(self):
    from cat_tool.settings import AppSettings
    s = AppSettings().load()
    d = s.as_dict()
    self.assertIsInstance(d, dict)
    self.assertIn("source_font_size", d)
    self.assertEqual(d["source_font_size"], 13)

def test_setitem_updates_dict(self):
    from cat_tool.settings import AppSettings
    s = AppSettings().load()
    s["source_font_size"] = 99
    self.assertEqual(s["source_font_size"], 99)

def test_save_creates_directory(self):
    from cat_tool.settings import AppSettings
    s = AppSettings().load()
    s.save()
    self.assertTrue(os.path.isdir(os.path.join(self._tmp.name, "mcat")))

# ===== ADDITIONAL QA EDGE CASES =====

class TestQAModuleAdditional(unittest.TestCase):
    def test_mixed_severity(self):
        from cat_tool.qa import QAIssue
        e = QAIssue("error", "E", 0, "s", "t", "d")
        w = QAIssue("warning", "W", 1, "s", "t", "d")
        self.assertEqual(e.severity, "error")
        self.assertEqual(w.severity, "warning")

    def test_empty_description(self):
        from cat_tool.qa import QAIssue
        issue = QAIssue("error", "test", 0, "src", "tgt", "")
        self.assertEqual(issue.description, "")

    def test_extract_numbers_edge_cases(self):
        from cat_tool.qa import _extract_numbers
        self.assertEqual(_extract_numbers(""), set())
        self.assertEqual(_extract_numbers("No digits here!"), set())
        self.assertEqual(_extract_numbers("Version 1.2.3"), {"1.2", "3"})

    def test_whitespace_no_source(self):
        from cat_tool.qa import check_whitespace
        segs = [{"source": None, "target": "Hello", "translated": True}]
        issues = check_whitespace(segs)
        self.assertEqual(len(issues), 0)

    def test_count_tags_empty(self):
        from cat_tool.qa import _count_tags
        self.assertEqual(_count_tags(""), 0)
        self.assertEqual(_count_tags(None), 0)
        self.assertEqual(_count_tags("plain text"), 0)

# ===== ADDITIONAL SEGMENTATION EDGE CASES =====

class TestSegmentationAdditional(unittest.TestCase):
    def test_none_text(self):
        from cat_tool.segmentation import advanced_segmenter
        result = advanced_segmenter(None)
        self.assertEqual(result, [])

    def test_multiple_periods(self):
        from cat_tool.segmentation import advanced_segmenter
        result = advanced_segmenter("Hello... World...")
        self.assertGreaterEqual(len(result), 2)

    def test_exclamation_and_question(self):
        from cat_tool.segmentation import advanced_segmenter
        result = advanced_segmenter("Wow! Really? Yes.")
        self.assertGreaterEqual(len(result), 3)

    def test_break_before_literal(self):
        from cat_tool.segmentation import advanced_segmenter
        rules = [
            {"pattern": "@", "break_after": False, "break_before": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
        ]

```

```

        result = advanced_segmenter("hello @world @test", rules)
        self.assertGreaterEqual(len(result), 3)

    def test_break_before_combined(self):
        from cat_tool.segmentation import advanced_segmenter
        rules = [
            {"pattern": ".", "break_after": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
            {"pattern": "@", "break_after": False, "break_before": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
        ]
        result = advanced_segmenter("hello. @world. @test", rules)
        self.assertGreaterEqual(len(result), 3)

    def test_break_before_and_after(self):
        from cat_tool.segmentation import advanced_segmenter
        rules = [
            {"pattern": ".", "break_after": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
            {"pattern": "@", "break_after": False, "break_before": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
        ]
        result = advanced_segmenter("Hello. Next @mention. Done.", rules)
        self.assertGreaterEqual(len(result), 3)

# ===== ADDITIONAL TAG UTILS EDGE CASES =====

class TestTagUtilsAdditional(unittest.TestCase):
    def test_extract_single_tag(self):
        from cat_tool.formats.tag_utils import extract_all_tags
        tags = extract_all_tags("<br/>")
        self.assertIn("<br/>", tags)

    def test_extract_nested_tags(self):
        from cat_tool.formats.tag_utils import extract_all_tags
        tags = extract_all_tags("<outer><inner>text</inner></outer>")
        self.assertIn("<outer>", tags)
        self.assertIn("<inner>", tags)
        self.assertIn("</inner>", tags)
        self.assertIn("</outer>", tags)

    def test_extract_self_closing_xml(self):
        from cat_tool.formats.tag_utils import extract_all_tags
        tags = extract_all_tags("<img src='x'> and <hr/>")
        self.assertIn("<img src='x'>", tags)
        self.assertIn("<hr/>", tags)

    def test_restore_with_none_tags(self):
        from cat_tool.formats.tag_utils import restore_tags
        result = restore_tags("Hello", None)
        self.assertEqual(result, "Hello")

    def test_apply_with_none_text(self):
        from cat_tool.formats.tag_utils import apply_tags
        result = apply_tags(None, ["<b>"])
        self.assertIsNone(result)

# ===== ADDITIONAL FILE HANDLER EDGE CASES =====

class TestFileHandlerAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.fh = __import__("cat_tool.file_handler", fromlist=["CATFileHandler"]).CATFileHandler()

    def tearDown(self):
        self.tmp.cleanup()

    def test_atomic_save_failure_cleans_temp(self):
        tmp_file = os.path.join(self.tmp.name, "test.po")
        with open(tmp_file, "w", encoding="utf-8") as f:
            f.write('msgid "Hello"\nmsgstr ""\n')
        self.fh.load_file(tmp_file)
        def _fail():
            raise RuntimeError("simulated failure")
        with self.assertRaises(RuntimeError):
            self.fh._atomic_save(_fail, tmp_file)
        temps = [f for f in os.listdir(self.tmp.name) if f.startswith(".mcat_tmp_")]

```

```

self.assertEqual(len(temps), 0)

def test_save_file_without_loading_raises(self):
    fh = __import__("cat_tool.file_handler", fromlist=["CATFileHandler"]).CATFileHandler()
    with self.assertRaises(ValueError):
        fh.save_file()

def test_current_file_path_none_on_new(self):
    self.assertIsNone(self.fh.current_file_path)

def test_load_with_mcatdb_extension(self):
    from cat_tool.formats.mcatdb import McatDbHandler
    path = os.path.join(self.tmp.name, "project.mcat.db")
    McatDbHandler().save([
        {
            "index": 0, "source": "Hello", "target": "Hola",
            "source_clean": "Hello", "target_clean": "Hola",
            "all_tags": [], "fuzzy": False, "translated": True,
            "locations": "", "notes": "", "source_location": {},
            "store": None, "unit": None,
        },
    ], path)
    segs = self.fh.load_file(path)
    self.assertEqual(len(segs), 1)
    self.assertEqual(segs[0]["source"], "Hello")

def test_render_without_handler_raises(self):
    self.fh.segments = [{"source": "Hello", "target": "Hola"}]
    self.fh.current_file_path = os.path.join(self.tmp.name, "dummy.txt")
    with self.assertRaises(ValueError):
        self.fh.render_translated(os.path.join(self.tmp.name, "out.txt"))

def test_save_file_mcatdb_preserves_path(self):
    from cat_tool.formats.mcatdb import McatDbHandler
    path = os.path.join(self.tmp.name, "project.mcat.db")
    McatDbHandler().save([
        {
            "index": 0, "source": "Hello", "target": "",
            "source_clean": "Hello", "target_clean": "",
            "all_tags": [], "fuzzy": False, "translated": False,
            "locations": "", "notes": "", "source_location": {},
            "store": None, "unit": None,
        },
    ], path)
    self.fh.load_file(path)
    self.fh.save_file()
    self.assertEqual(self.fh.current_file_path, path)

# ===== XLS HANDLER TESTS =====

class TestXlsHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_xls_load_basic(self):
        try:
            from cat_tool.formats.xls import XlsHandler
        except ImportError:
            self.skipTest("XlsHandler not available")
        try:
            import xlwt
        except ImportError:
            self.skipTest("xlwt not available")
        wb = xlwt.Workbook()
        ws = wb.add_sheet("Sheet1")
        ws.write(0, 0, "Hello")
        ws.write(0, 1, "World")
        path = os.path.join(self.tmp.name, "test.xls")
        wb.save(path)
        segs = XlsHandler().load(path)
        texts = [s["source"] for s in segs]
        self.assertIn("Hello", texts)

    def test_xls_load_empty_workbook(self):
        try:
            from cat_tool.formats.xls import XlsHandler
        except ImportError:
            self.skipTest("XlsHandler not available")
        try:

```

```

        import xlwt
    except ImportError:
        self.skipTest("xlwt not available")
    wb = xlwt.Workbook()
    wb.add_sheet("Empty")
    path = os.path.join(self.tmp.name, "empty.xls")
    wb.save(path)
    segs = XlsHandler().load(path)
    self.assertEqual(len(segs), 0)

def test_xls_load_nonexistent_raises(self):
    try:
        from cat_tool.formats.xls import XlsHandler
    except ImportError:
        self.skipTest("XlsHandler not available")
    with self.assertRaises(FileNotFoundError):
        XlsHandler().load(r"C:\nonexistent.xls")

# ===== ODF HANDLER TESTS =====

class TestOdfHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_load_odt(self):
        try:
            from odf.opendocument import OpenDocumentText
            from odf.text import P
        except ImportError:
            self.skipTest("odf not available")
        doc = OpenDocumentText()
        p = P(text="Hello ODF World")
        doc.text.addElement(p)
        path = os.path.join(self.tmp.name, "test.odt")
        doc.save(path)
        from cat_tool.formats.odf import OdfHandler
        segs = OdfHandler().load(path)
        texts = [s["source"] for s in segs]
        self.assertIn("Hello ODF World", texts)

    def test_odf_load_nonexistent_raises(self):
        try:
            from cat_tool.formats.odf import OdfHandler
        except ImportError:
            self.skipTest("odf not available")
        with self.assertRaises(FileNotFoundError):
            OdfHandler().load(r"C:\nonexistent.odt")

    def test_odf_render_updates_target(self):
        try:
            from odf.opendocument import OpenDocumentText
            from odf.text import P
        except ImportError:
            self.skipTest("odf not available")
        doc = OpenDocumentText()
        p = P(text="Hello")
        doc.text.addElement(p)
        path = os.path.join(self.tmp.name, "test.odt")
        doc.save(path)
        from cat_tool.formats.odf import OdfHandler
        handler = OdfHandler()
        segs = handler.load(path)
        segs[0]["target"] = "Hola"
        out_path = os.path.join(self.tmp.name, "out.odt")
        handler.render(path, segs, out_path)
        segs2 = handler.load(out_path)
        self.assertEqual(segs2[0]["source"], "Hola")

    def test_odf_render_no_segments_output_created(self):
        try:
            from odf.opendocument import OpenDocumentText
        except ImportError:
            self.skipTest("odf not available")
        doc = OpenDocumentText()
        path = os.path.join(self.tmp.name, "empty.odt")

```

```

        doc.save(path)
        from cat_tool.formats.odf import OdfHandler
        out_path = os.path.join(self.tmp.name, "out.odt")
        OdfHandler().render(path, [], out_path)
        self.assertTrue(os.path.exists(out_path))

# ===== ADDITIONAL FORMAT EDGE CASES =====

class TestMakeSegmentsFromTextsAdditional(unittest.TestCase):
    def test_none_text_skipped(self):
        from cat_tool.formats._text_base import make_segments_from_texts
        segs = make_segments_from_texts(["Hello", None, "World"])
        self.assertEqual(len(segs), 2)

    def test_all_empty_skipped(self):
        from cat_tool.formats._text_base import make_segments_from_texts
        segs = make_segments_from_texts(["", "", None])
        self.assertEqual(len(segs), 0)

    def test_normal_works(self):
        from cat_tool.formats._text_base import make_segments_from_texts
        segs = make_segments_from_texts(["Hello", "World"])
        self.assertEqual(len(segs), 2)

# ===== CONVERT TO PO EDGE CASES =====

class TestConvertToPoAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_convert_to_po_creates_file(self):
        from cat_tool.file_handler import convert_to_po
        txt_path = os.path.join(self.tmp.name, "source.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Hello\nWorld\nTest")
        out = os.path.join(self.tmp.name, "output.po")
        convert_to_po(txt_path, out)
        self.assertTrue(os.path.exists(out))
        with open(out, encoding="utf-8") as f:
            content = f.read()
        self.assertIn("Hello", content)
        self.assertIn("World", content)

    def test_convert_to_po_roundtrip(self):
        from cat_tool.file_handler import CATFileHandler, convert_to_po
        txt_path = os.path.join(self.tmp.name, "source.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Line one\nLine two")
        po_path = os.path.join(self.tmp.name, "conv.po")
        convert_to_po(txt_path, po_path)
        fh = CATFileHandler()
        segs = fh.load_file(po_path)
        self.assertEqual(len(segs), 2)

# ===== MCAT FORMAT MANAGER ADDITIONAL TESTS =====

class TestMcatProjectManagerAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp.name, "test.mcat.db")

    def tearDown(self):
        self.tmp.cleanup()

    def test_get_editor_stream_empty_db(self):
        from cat_tool.mcat_format_manager import McatProjectManager
        McatProjectManager.create_project_file(self.db_path, "en", "es")
        rows = McatProjectManager.get_editor_stream(self.db_path)
        self.assertEqual(len(rows), 0)

    def test_get_editor_stream_filter_nonexistent_status(self):
        from cat_tool.mcat_format_manager import McatProjectManager
        McatProjectManager.create_project_file(self.db_path, "en", "es")

```

```

        rows = MCatProjectManager.get_editor_stream(self.db_path, filter_status="NonExistent")
        self.assertEqual(len(rows), 0)

    def test_update_segment_invalid_id(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        MCatProjectManager.update_segment_translation(self.db_path, 999, "test", "Confirmed")

    def test_import_and_get_stream(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        segments = [
            {"file_id": "test.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
        ]
        MCatProjectManager.import_extracted_segments(self.db_path, segments)
        rows = MCatProjectManager.get_editor_stream(self.db_path)
        self.assertEqual(len(rows), 1)
        self.assertEqual(rows[0]["source_text"], "Hello")

    def test_update_segment(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        MCatProjectManager.import_extracted_segments(self.db_path, [
            {"file_id": "a.txt", "source": "Hello", "target": "", "status": "Untranslated", "match_score": 0},
        ])
        rows = MCatProjectManager.get_editor_stream(self.db_path)
        unit_id = rows[0]["id"]
        MCatProjectManager.update_segment_translation(self.db_path, unit_id, "Hola", "Confirmed")
        rows = MCatProjectManager.get_editor_stream(self.db_path)
        self.assertEqual(rows[0]["target_text"], "Hola")

# ===== DEDICATED QA CHECK TESTS =====

class TestQACheckFunctions(unittest.TestCase):
    def test_check_terminology_no_glossary(self):
        from cat_tool.qa import check_terminology
        segs = [{"source": "cat", "target": "gato", "translated": True}]
        with self.assertRaises(AttributeError):
            check_terminology(segs, None)

    def test_check_inconsistent_translations_empty_source(self):
        from cat_tool.qa import check_inconsistent_translations
        segs = [
            {"source": "", "target": "Hola", "translated": True},
            {"source": "", "target": "Hello", "translated": True},
        ]
        issues = check_inconsistent_translations(segs)
        self.assertEqual(len(issues), 0)

    def test_tag_mismatch_no_source_tags(self):
        from cat_tool.qa import check_tag_mismatch
        segs = [{"source": "Hello", "target": "Hola {0}", "all_tags": []}]
        issues = check_tag_mismatch(segs)
        self.assertEqual(len(issues), 0)

    def test_count_tags_with_tags(self):
        from cat_tool.qa import _count_tags
        from cat_tool.formats.tag_utils import PH_L, PH_R
        text = f"{PH_L}{0}{PH_R} and {PH_L}1{PH_R}"
        self.assertEqual(_count_tags(text), 2)

# ===== MEMORY / EDGE CASE TESTS =====

class TestTranslationMemoryAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp.name, "tm.db")
        from cat_tool.tm import TranslationMemory
        self.tm = TranslationMemory(self.db_path)

    def tearDown(self):
        self.tmp.cleanup()

    def test_add_translation_none_target(self):
        self.tm.add_translation("Hello", None)
        matches = self.tm.get_fuzzy_matches("Hello")
        self.assertEqual(len(matches), 0)

```

```

def test_get_fuzzy_matches_empty_db(self):
    matches = self.tm.get_fuzzy_matches("Anything")
    self.assertEqual(len(matches), 0)

def test_get_fuzzy_matches_with_min_score(self):
    self.tm.add_translation("Hello world", "Hola mundo")
    matches = self.tm.get_fuzzy_matches("Hello world!", min_score=90.0)
    self.assertGreater(len(matches), 0)

def test_get_fuzzy_matches_extreme_min_score(self):
    self.tm.add_translation("Hello", "Hola")
    matches = self.tm.get_fuzzy_matches("Completely different", min_score=0.0)
    self.assertGreater(len(matches), 0)

class TestGlossaryAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp.name, "glossary.db")
        from cat_tool.glossary import Glossary
        self.gl = Glossary(self.db_path)

    def tearDown(self):
        self.tmp.cleanup()

    def test_remove_nonexistent_term(self):
        self.gl.remove_term("nonexistent")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 0)

    def test_check_segment_none_text(self):
        matches = self.gl.check_segment(None)
        self.assertEqual(len(matches), 0)

    def test_add_term_special_chars(self):
        self.gl.add_term("cafe", "coffee")
        matches = self.gl.check_segment("I like cafe")
        self.assertEqual(len(matches), 1)

    def test_add_term_with_langs(self):
        self.gl.add_term("hello", "hola", source_lang="en", target_lang="es")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 1)
        self.assertEqual(terms[0]["source_lang"], "en")
        self.assertEqual(terms[0]["target_lang"], "es")

    def test_import_tmx_basic(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>
  <body>
    <tu>
      <tuv xml:lang="en"><seg>Hello</seg><tuv>
      <tuv xml:lang="es"><seg>Hola</seg><tuv>
    </tu>
    <tu>
      <tuv xml:lang="en"><seg>Goodbye</seg><tuv>
      <tuv xml:lang="es"><seg>Adiós</seg><tuv>
    </tu>
  </body>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "test.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path)
        self.assertEqual(count, 2)
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 2)
        for t in terms:
            self.assertEqual(t["source_lang"], "en")
            self.assertEqual(t["target_lang"], "es")

    def test_import_tmx_nonexistent_file(self):
        with self.assertRaises(FileNotFoundError):
            self.gl.import_tmx("nonexistent.tmx")

    def test_import_tmx_empty_body(self):
        import xml.etree.ElementTree as ET

```

```

        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>
  <body>
  </body>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "empty.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path)
        self.assertEqual(count, 0)

    def test_import_tmx_missing_body(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "nobody.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path)
        self.assertEqual(count, 0)

    def test_import_tmx_no_srclang_header(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header datatype="plaintext"/>
  <body>
    <tu>
      <tuv xml:lang="fr"><seg>Bonjour</seg></tuv>
      <tuv xml:lang="en"><seg>Hello</seg></tuv>
    </tu>
  </body>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "nosrclang.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path)
        self.assertEqual(count, 1)

    def test_import_tmx_with_explicit_source_lang(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>
  <body>
    <tu>
      <tuv xml:lang="en"><seg>Cat</seg></tuv>
      <tuv xml:lang="fr"><seg>Chat</seg></tuv>
    </tu>
  </body>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "explicit.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path, source_lang="fr")
        self.assertEqual(count, 1)
        terms = self.gl.get_all_terms()
        self.assertEqual(terms[0]["source"], "Chat")
        self.assertEqual(terms[0]["target"], "Cat")
        self.assertEqual(terms[0]["source_lang"], "fr")

    def test_peek_tmx_basic(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>
  <body>
    <tu><tuv xml:lang="en"><seg>Hello</seg></tuv><tuv xml:lang="es"><seg>Hola</seg></tuv>
    <tu><tuv xml:lang="en"><seg>Bye</seg></tuv><tuv xml:lang="es"><seg>Adiós</seg></tuv>
  </body>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "peek.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        info = self.gl.peek_tmx(tmx_path)
        self.assertEqual(info["source_lang"], "en")

```

```

        self.assertEqual(info["target_lang"], "es")
        self.assertEqual(info["count"], 2)

    def test_peek_tmx_no_srclang(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header datatype="plaintext"/>
  <body>
    <tu>
      <tu xml:lang="fr">
        <seg>Bonjour</seg>
      </tu>
    </body>
  </tmx>"""
        tmx_path = os.path.join(self.tmp.name, "peek2.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        info = self.gl.peek_tmx(tmx_path)
        self.assertEqual(info["source_lang"], "")
        self.assertIn("fr", info["target_lang"])
        self.assertEqual(info["count"], 1)

    def test_export_tmx_empty(self):
        tmx_path = os.path.join(self.tmp.name, "out.tmx")
        count = self.gl.export_tmx(tmx_path)
        self.assertEqual(count, 0)

    def test_export_tmx_roundtrip(self):
        from cat_tool.glossary import Glossary
        import xml.etree.ElementTree as ET
        self.gl.add_term("hello", "hola", source_lang="en", target_lang="es")
        self.gl.add_term("goodbye", "adiós", source_lang="en", target_lang="es")
        tmx_path = os.path.join(self.tmp.name, "roundtrip.tmx")
        count = self.gl.export_tmx(tmx_path)
        self.assertEqual(count, 2)

        self.gl2 = Glossary(os.path.join(self.tmp.name, "glossary2.db"))
        count2 = self.gl2.import_tmx(tmx_path)
        self.assertEqual(count2, 2)
        terms = self.gl2.get_all_terms()
        self.assertEqual(len(terms), 2)
        self.assertEqual(terms[0]["source_lang"], "en")
        self.assertEqual(terms[0]["target_lang"], "es")

    def test_export_tmx_source_lang_override(self):
        import xml.etree.ElementTree as ET
        self.gl.add_term("bonjour", "hello", source_lang="fr", target_lang="en")
        tmx_path = os.path.join(self.tmp.name, "override.tmx")
        self.gl.export_tmx(tmx_path, source_lang="fr")
        tree = ET.parse(tmx_path)
        root = tree.getroot()
        header = root.find("header")
        self.assertEqual(header.get("srclang"), "fr")

if __name__ == "__main__":
    unittest.main()

```